République Tunisienne
Ministère de l'Enseignement
Supérieur et de la Recherche
Scientifique

Université de Monastir
Institut Supérieur d'Informatique de Mahdia

MEMOIRE

N° d'ordre : MP-EXR-2-2026-007

MÉMOIRE

Présenté à

Institut Supérieur d'Informatique de Mahdia

Effectué à

ISIMA

En vue de l'obtention du diplôme de

Master Professionnel Computer Engineering : Expert Réseaux

Elaboré par :

Aziz Dhifaoui

Plateforme autonome de cyberdéfense pour la détection et la prévention des cyberattaques

Soutenu le : 14 octobre 2026
devant le jury composé de :

Président	: Walid Hassairi
Rapporteur	: Tarek M. Hamdani
Encadrant Académique	: Hamdi Aloulou
Encadrant Professionnel	: Isima Isima

Année universitaire 2026-2027

Remerciements

Tout d'abord, je remercie **Dieu Tout-Puissant** de m'avoir accordé la santé, la patience, la volonté et la force nécessaires pour mener à bien ce travail jusqu'à son terme.

Je tiens également à m'adresser à moi-même des remerciements sincères, pour la persévérance, la rigueur et la discipline personnelle dont j'ai dû faire preuve tout au long de ce projet, souvent exigeant et chronophage, afin de concevoir, implémenter et documenter une plateforme aussi complète que NG-IDPS.

Mes remerciements les plus chaleureux s'adressent ensuite à mon encadrant, **Mr. Hamdi Aloulou**, pour sa disponibilité, ses conseils avisés, son suivi rigoureux et la confiance qu'il m'a accordée tout au long de la réalisation de ce projet de fin d'études. Son accompagnement et ses remarques pertinentes ont grandement contribué à l'orientation et à la qualité de ce travail.

Enfin, j'exprime ma profonde gratitude à l'ensemble de mes **professeurs de l'Institut Supérieur d'Informatique de Mahdia (ISIMa)**, pour la qualité de la formation dispensée tout au long de mon cursus et pour les connaissances et compétences qu'ils m'ont transmises, qui ont constitué le socle indispensable à la réalisation de ce projet.

Aziz Dhifaoui

Dédicaces

À mes chers parents,

Aucun mot, aucune dédicace ne pourra jamais traduire la profondeur de ma gratitude envers vous, ni la valeur des sacrifices que vous avez consentis pour moi.

Vous avez porté mes doutes comme les vôtres, supporté mes moments de fatigue et de découragement, et n'avez jamais cessé d'y croire, même quand moi-même je n'y croyais plus.

Ce travail porte en lui vos nuits d'inquiétude, vos prières silencieuses, et toutes les fois où vous avez renoncé à quelque chose pour que, moi, je n'aie à renoncer à rien.

Si aujourd'hui j'arrive au bout de ce chapitre de ma vie, c'est parce que vous avez souffert, avec moi, à chaque étape, sans jamais me le faire peser.

Ce modeste travail est le vôtre avant d'être le mien.
Puisse Dieu vous accorder la santé, la paix et une vie longue à la hauteur de tout ce que vous m'avez donné.

À mon frère Ahmed et ma sœur Maryem,
pour leur soutien moral, leurs encouragements constants et leur présence à mes côtés tout au long de ce parcours.

À mes véritables amis,
ceux qui ont partagé mes doutes, mes nuits blanches et mes découragements sans jamais me laisser seul face à eux,
et qui ont su transformer la souffrance partagée en force pour avancer.

Ce travail n'est pas seulement l'aboutissement d'un effort personnel :
il est la preuve que rien de ce qui compte vraiment ne se construit seul.

À tous ceux qui ont souffert à mes côtés pour que j'arrive jusqu'ici,
je dédie ce travail, avec toute ma reconnaissance.

I LOVE YOU ALL.

LISTE DES ACRONYMES

Acronyme	**Signification**
AE	Autoencoder
AF_PACKET	Address Family Packet
ANOVA	Analysis of Variance
API	Application Programming Interface
APT	Advanced Persistent Threat
ARP	Address Resolution Protocol
AUC	Area Under the Curve
CLI	Command Line Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
CTI	Cyber Threat Intelligence
CVE	Common Vulnerabilities and Exposures
DDoS	Distributed Denial of Service
DNS	Domain Name System
DPDK	Data Plane Development Kit
ELBO	Evidence Lower BOund
ELK	Elasticsearch, Logstash, Kibana
ETL	Extract, Transform, Load
FPR	False Positive Rate
HTTP	HyperText Transfer Protocol
ICMP	Internet Control Message Protocol
IDS	Intrusion Detection System
IF	Isolation Forest
IOC	Indicator of Compromise

IoT	Internet of Things
IP	Internet Protocol
IPS	Intrusion Prevention System
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
KRaft	Kafka Raft
LLM	Large Language Model
LSTM	Long Short-Term Memory
MAC	Media Access Control (adresse physique de couche liaison)
ML	Machine Learning
MLOps	Machine Learning Operations
MSE	Mean Squared Error
NG-IDPS	Next-Generation Intrusion Detection and Prevention System
NIDS	Network Intrusion Detection System
OISF	Open Information Security Foundation
OSINT	Open-Source Intelligence
PPA	Personal Package Archive
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
Regex	Regular Expression
ROC	Receiver Operating Characteristic
SASL	Simple Authentication and Security Layer
SIEM	Security Information and Event Management
SOAR	Security Orchestration, Automation and Response
SOC	Security Operations Center
SSH	Secure Shell
TCP	Transmission Control Protocol
TI	Threat Intelligence
TLS	Transport Layer Security
TPR	True Positive Rate
UDP	User Datagram Protocol
VAE	Variational Autoencoder
VLAN	Virtual Local Area Network
VM	Virtual Machine
VRAM	Video Random Access Memory

YAML	YAML Ain't Markup Language
ZT	Zero Trust

TABLE DES MATIÈRES

Liste des acronymes 7
Table des figures 17
Liste des tableaux 19
Introduction Générale 20

1 Présentation du Projet et Étude de l'Existant **22**
1.1 Introduction 23
1.2 Champ d'application et Contexte Opérationnel 23
1.3 Présentation du Projet 24
1.3.1 Contexte 24
1.3.2 Problématique 24
1.3.3 Objectifs du Projet 25
1.4 Étude et Analyse de l'Existant 29
1.4.1 Évolution des Systèmes de Détection et de Réponse 29
1.4.2 Analyse des Approches Existantes 31
1.4.3 Analyse Comparative des Solutions Représentatives 33
1.4.3.1 Critères retenus. 33
1.4.3.2 Interprétation. 37
1.4.4 Limites des Solutions Existantes et Besoins Spécifiques du Projet . 38
1.4.5 Positionnement de NG-IDPS 40
1.5 Méthodologie adoptée 41
1.5.1 Méthodologie retenue 41
1.5.2 Méthodologie Scrum 41
1.5.3 Organisation des Sprints 42
1.5.4 Les rôles au sein de Scrum 43
1.5.5 Processus de développement adopté 44
1.6 Conclusion 44

2 Phase Préparatoire et Ingénierie Système 46
2.1 Introduction 48
2.2 Analyse et spécification des besoins 48
2.2.1 Identification des acteurs 49
2.2.2 Besoins fonctionnels 49
2.2.2.1 Besoins fonctionnels de l'Administrateur SOC 50
2.2.2.2 Besoins fonctionnels de l'Employé 50
2.2.2.3 Besoins fonctionnels de l'Agent autonome 51
2.2.3 Besoins non fonctionnels 51
2.3 Diagramme des cas d'utilisation globale 53
2.4 Backlog du produit 53
2.5 Planification des sprints 56
2.6 Environnement logiciel 57
2.6.1 Démarche de comparaison 57
2.6.2 Infrastructure de virtualisation et système d'exploitation 58
2.6.3 Détection d'intrusions réseau 59
2.6.4 Collecte, indexation et visualisation des journaux 60
2.6.5 Transport de données en flux 61
2.6.6 Détection d'anomalies par apprentissage profond 63
2.6.7 IA générative, RAG et renseignement sur les menaces 65
2.6.8 Prévention et réponse automatisée 67
2.6.9 Serveur applicatif et communication temps réel 68
2.6.10 Interface de supervision 69
2.6.11 Persistance des données 70
2.6.12 Gestion de projet, modélisation et rédaction 71
2.6.13 Synthèse de la pile technologique 72
2.7 Architecture générale 73
2.8 Conclusion 75

3 Détection - Ingestion parallèle et moteur stochastique (Release 1) . . 77
3.1 Introduction 79
3.2 Déploiement et Optimisation du Moteur d'Inspection Réseau (Sprint 1) . . 79
3.2.1 Backlog du Sprint 1 80
3.2.2 Analyse Fonctionnelle 80
3.2.3 Conception 81
3.2.4 Réalisation 82
3.2.5 Validation 82
3.2.6 Conclusion du Sprint 1 83
3.3 Intégration du Pipeline de Streaming Double Chemin (Sprint 2) 84

3.3.1 Justification Stratégique et Choix Architecturaux 84
3.3.2 Backlog du Sprint 2 84
3.3.3 Analyse Fonctionnelle : Routage et Enrichissement 85
3.3.4 Conception 86
3.3.5 Réalisation 87
3.3.6 Validation 87
3.3.6.1 Preuve de Tolérance aux Pannes (Kafka) 87
3.3.6.2 Preuve d'Enrichissement Sémantique (Logstash GeoIP) . . 88
3.3.6.3 Preuve de Scalabilité du Dashboard (Kibana) 88
3.3.7 Conclusion du Sprint 2 89
3.4 Implémentation du Modèle d'Apprentissage Profond LSTM-VAE (Sprint 3) 89
3.4.1 Backlog du Sprint 3 89
3.4.2 Analyse : Justification Stratégique et Choix Algorithmique 90
3.4.3 Conception : Ingénierie des Données (Data Preprocessing & Feature Engineering) 91
3.4.3.1 Prétraitement et Nettoyage 91
3.4.3.2 Sélection des Caractéristiques : de la Sélection Globale à la Sélection Équilibrée 91
3.4.3.3 Mise à l'Échelle et Normalisation (Scaling) 92
3.4.4 Conception 93
3.4.5 Réalisation : Entraînement et choix des hyperparamètres 95
3.4.6 Réalisation : Calibration Rigoureuse du Seuil de Décision 95
3.4.6.1 Diagnostic d'un Défaut de Recalibration 96
3.4.6.2 Méthodologie de Calibration Rigoureuse 96
3.4.6.3 Comparaison des Points de Fonctionnement Retenus . . . 97
3.4.7 Validation : Étude comparative des architectures de détection . . . 98
3.4.8 Validation : Tests sur données synthétiques 98
3.4.8.1 Test Synthétique par Profil Statistique Réel 99
3.4.8.2 Test d'Isolation du Cas PortScan 99
3.4.8.3 Test de Généralisation Hors-Distribution 99
3.4.9 Réalisation : Mécanismes de cyber-résilience et inférence temps-réel (Edge-to-ml) 99
3.4.9.1 Tampon Glissant Isolé (Stateful Windowing) 100
3.4.9.2 Pré-filtre Volumétrique Global (Bouclier Anti-DDoS Spoofing) 100
3.4.9.3 Persistance Temporelle et Liste Blanche (Whitelist) 100
3.4.10 Validation : Diagnostic et durcissement sécuritaire en conditions réelles 100
3.4.10.1 Validation Face à des Attaques Réellement Exécutées . . . 100

3.4.10.2 Durcissement contre l'Usurpation d'Adresse : Protection ARP . . . 102
3.4.11 Validation : Convergence, Latence et Faux Positifs en Conditions Opérationnelles . . . 102
3.4.12 Limites Identifiées et Perspectives . . . 104
3.4.13 Conclusion du Sprint 3 . . . 104
3.5 Conclusion . . . 104

4 Prévention, SOAR, agents IA et dashboard SOC (Release 2) . . . 106
4.1 Introduction . . . 108
4.2 Sprint 4 : Anticipation par le renseignement (agent CTI et triage Dual-RAG) 108
4.2.1 Backlog du sprint 4 . . . 108
4.2.2 Analyse Fonctionnelle . . . 109
4.2.3 Conception . . . 111
4.2.3.1 Architecture du sprint . . . 111
4.2.3.2 Diagramme de séquence . . . 111
4.2.4 Réalisation . . . 112
4.2.4.1 Environnement et composants . . . 112
4.2.4.2 Interfaces et résultats . . . 112
4.2.5 Validation . . . 114
4.2.6 Conclusion du sprint 4 . . . 116
4.3 Sprint 5 : Mitigation du risque humain (agent Gemini et RAG LMS) . . . 117
4.3.1 Backlog du sprint 5 . . . 117
4.3.2 Analyse Fonctionnelle . . . 118
4.3.3 Conception . . . 119
4.3.3.1 Architecture du sprint . . . 119
4.3.3.2 Diagramme de séquence . . . 119
4.3.4 Réalisation . . . 120
4.3.4.1 Composants . . . 120
4.3.4.2 Interfaces et résultats . . . 120
4.3.5 Validation . . . 123
4.3.6 Conclusion du sprint 5 . . . 125
4.4 Sprint 6 : Orchestration SOAR multi-paliers, agent post-attaque et dashboard SOC . . . 126
4.4.1 Backlog du sprint 6 . . . 126
4.4.2 Analyse Fonctionnelle . . . 127
4.4.3 Conception . . . 128
4.4.3.1 Architecture du sprint . . . 128
4.4.3.2 Deux moteurs de remédiation distincts . . . 130

4.4.3.3 Score composite de menace . . . 130
4.4.3.4 Notification en deux temps . . . 131
4.4.3.5 Diagrammes de séquence . . . 131
4.4.3.6 Diagramme d'états de la réponse . . . 132
4.4.4 Réalisation . . . 133
4.4.4.1 Moteur de corrélation autonome (`soar.py`) . . . 133
4.4.4.2 Agent IA post-attaque . . . 134
4.4.4.3 Politique Zero Trust et corrélation anti-faux-positif . . . 134
4.4.4.4 Bouclier anti-DDoS . . . 134
4.4.4.5 Mode auto-pilote et exécution supervisée . . . 134
4.4.4.6 Dashboard SOC, authentification et watchdog . . . 135
4.4.4.7 Interfaces et résultats . . . 135
4.4.5 Validation . . . 139
4.4.6 Conclusion du sprint 6 . . . 142
Conclusion . . . 142

Conclusion Générale . . . 143

Webographie . . . 146

TABLE DES FIGURES

2.1 Diagramme de cas d'utilisation global de NG-IDPS 54
2.2 Logo VMware Workstation Pro 59
2.3 Logo Suricata 60
2.4 Logo Elastic Stack 61
2.5 Logo Kafka 62
2.6 Modèle Deep LSTM-VAE 64
2.7 Logo Llama 3.2 67
2.8 Logo Python 68
2.9 Logo NodeJS - Express 69
2.10 Logo NextJS 70
2.11 Logo Mongo BD 71
2.12 Logo Github 71
2.13 Architecture de déploiement de NG-IDPS 74

3.1 Diagramme de séquence « Capture et Inspection Sémantique des Flux » . . 81
3.2 Validation de la génération des logs de détection 83
3.3 Diagramme de séquence et d'architecture « Routage Résilient et Enrichissement » 86
3.4 Validation de la communication asynchrone inter-nœuds via Apache Kafka 88
3.5 Validation de l'enrichissement GeoIP par Logstash 88
3.6 Tableau de bord SOC global 89
3.7 Matrice des scores ANOVA par type d'attaque ayant motivé la sélection équilibrée 92
3.8 Architecture et flux d'inférence probabiliste du modèle LSTM-VAE 94
3.9 Courbes ROC et Précision-Rappel sur le jeu de calibration 96
3.10 Logs de détection croisée Suricata/LSTM-VAE lors de l'attaque Nikto . . . 101
3.11 Logs de détection croisée Suricata/LSTM-VAE lors de l'attaque Port Scan 101

3.12 Logs de détection et d'auto-blocage lors de l'attaque Hydra (brute-force SSH) 102
3.13 Courbes de convergence algorithmique du modèle LSTM-VAE avec Early Stopping . . . 103

4.1 Architecture du sprint 4 . . . 111
4.2 Diagramme de séquence "triage et validation d'un article par l'agent CTI" 112
4.3 Journal d'exécution de l'agent CTI : ingestion et triage d'un article . . . 113
4.4 Génération et validation d'une règle Suricata candidate . . . 113
4.5 Fil d'actualités CTI affiché sur le dashboard, statut `PENDING` . . . 114
4.6 Article technique . . . 115
4.7 Article de sensibilisation . . . 115
4.8 Contenu publicitaire . . . 116
4.9 Article déjà présent en base . . . 116
4.10 Architecture du sprint 5 . . . 119
4.11 Diagramme de séquence « Génération résiliente de la formation » . . . 120
4.12 Module de cours généré par Gemini . . . 121
4.13 Module de cours généré par Mistral . . . 121
4.14 Module de cours généré par Ollama Local . . . 122
4.15 E-mail de notification personnalisé envoyé à l'employé . . . 122
4.16 Portail LMS : connexion par identifiant employé et QCM . . . 123
4.17 Enregistrement du score dans la collection `FormationRecord` . . . 123
4.18 Déclenchement de la génération d'un module sur un article `AWARENESS` . . . 124
4.19 Diffusion du formation ciblée . . . 124
4.20 Connexion avec ID inconnu . . . 125
4.21 Connexion avec ID non autorisé . . . 125
4.22 Architecture du sprint 6 . . . 129
4.23 Diagramme de séquence « de l'anomalie détectée à l'affichage sur le dashboard » . . . 131
4.24 Diagramme de séquence « décision autonome versus exécution supervisée » 132
4.25 Diagramme d'états d'un incident SOAR . . . 133
4.26 Page `Login` . . . 136
4.27 Page `Rapport` . . . 136
4.28 Vue globale du dashboard SOC . . . 137
4.29 Page `/incidents` et export PDF du rapport . . . 138
4.30 Page `/infrastructure` : état des machines virtuelles (watchdog) . . . 138
4.31 Règle `iptables` appliquée par l'agent d'exécution sur la cible . . . 139
4.32 Attaque simulée, administrateur connecté . . . 140
4.33 Attaque simulée, aucun administrateur connecté (Mode Nuit) . . . 140
4.34 Résultats des tests de validation du sprint 6 . . . 141

4.35 Coupure kafka et reprise 141
4.36 Résultats des tests de validation du sprint 6 142

LISTE DES TABLEAUX

1.1 Objectifs fonctionnels du système NG-IDPS 26
1.2 Objectifs non fonctionnels du système NG-IDPS 28
1.3 Évolution des approches de détection et de réponse 30
1.4 Analyse synthétique des principales approches de sécurité 31
1.5 Comparaison de NG-IDPS avec des plateformes de référence 34
1.6 Limites identifiées et besoins spécifiques de NG-IDPS 38
1.7 Organisation des Sprints du projet NG-IDPS 42
1.8 Rôles Scrum dans le cadre du projet 43

2.1 Acteurs de NG-IDPS 49
2.2 Besoins fonctionnels de l'Administrateur SOC 50
2.3 Besoins fonctionnels de l'Employé 50
2.4 Besoins fonctionnels de l'Agent autonome 51
2.5 Besoins non fonctionnels de NG-IDPS 52
2.6 Backlog du produit 54
2.7 Planification des releases et des sprints 56
2.8 Comparaison des solutions de virtualisation 58
2.9 Matrice de décision pour le moteur de détection réseau 60
2.10 Comparaison des plateformes de gestion des journaux 61
2.11 Matrice de décision pour le bus de messages 62
2.12 Matrice de décision pour le modèle de détection d'anomalies 63
2.13 Comparaison des jeux de données de détection d'intrusions 65
2.14 Comparaison des modèles de langage candidats 65
2.15 Comparaison des approches d'orchestration SOAR 67
2.16 Comparaison des piles serveur 68
2.17 Comparaison des cadres d'interface 69
2.18 Comparaison des solutions de persistance 70

2.19 Outils de gestion de projet, de modélisation et de rédaction 71
2.20 Synthèse de la pile technologique de NG-IDPS 72
2.21 Topics Kafka de NG-IDPS 74

3.1 Backlog détaillé du Sprint 1 80
3.2 Description textuelle du cas d'utilisation d'acquisition Edge 80
3.3 Backlog détaillé du Sprint 2 85
3.4 Description textuelle du cas d'utilisation de Streaming et d'ETL 85
3.5 Backlog détaillé du Sprint 3 90
3.6 Comparaison des deux méthodologies de sélection de caractéristiques . . . 92
3.7 Comparaison des quatre points de fonctionnement du seuil de décision . . . 97
3.8 Étude comparative des trois architectures de détection (mêmes données, même méthodologie) 98
3.9 Validation en conditions réelles : détection croisée Suricata / LSTM-VAE . 101

4.1 Répartition des sprints de la release 2. 108
4.2 Backlog du sprint 4. 109
4.3 Description textuelle du cas d'utilisation « Trier et traiter un article de renseignement ». 110
4.4 Plan de validation du sprint 4. 114
4.5 Backlog du sprint 5. 117
4.6 Description textuelle du cas d'utilisation « Générer et diffuser un parcours de sensibilisation ». 118
4.7 Plan de validation du sprint 5. 123
4.8 Backlog du sprint 6. 126
4.9 Description textuelle du cas d'utilisation « Neutraliser une menace corrélée » 127
4.10 Topics Kafka mobilisés par le sprint 6. 129
4.11 Distinction entre `soar.py` (orchestrateur autonome) et `agent_soar.py` (exécuteur supervisé). 130
4.12 Paliers de traitement du SOAR et mode auto-pilote. 135
4.13 Plan de validation du sprint 6. 139

INTRODUCTION GÉNÉRALE

À l'ère de la transformation numérique, l'interconnexion croissante des systèmes d'information et la dépendance des organisations vis-à-vis de leurs infrastructures réseau marquent une rupture significative avec les pratiques traditionnelles de la sécurité informatique. Le paysage des menaces évolue en parallèle : les cyberattaques deviennent plus variées, plus automatisées et plus difficiles à caractériser à l'avance. La défense moderne ne peut plus reposer uniquement sur des règles prédéfinies ; elle s'appuie désormais sur la complémentarité de plusieurs mécanismes : détection par signatures, analyse comportementale fondée sur l'apprentissage automatique, corrélation de sources multiples, renseignement sur les menaces et automatisation de la réponse. Cette convergence redéfinit les standards de réactivité, de résilience et d'efficacité des centres opérationnels de sécurité (*Security Operations Center*, SOC).

Dans un pays en développement comme la Tunisie, l'adoption de ces approches reste encore limitée. De nombreuses organisations disposent de ressources humaines et financières restreintes et ne peuvent pas assurer une supervision permanente de leurs réseaux. Elles s'appuient souvent sur des mécanismes de protection statiques, les exposant à des délais de détection importants, à des alertes difficiles à exploiter et à des attaques qu'aucune règle connue ne décrit encore. Par ailleurs, les plateformes commerciales intégrées reposent fréquemment sur des licences ou des services dans le nuage qui ne conviennent pas toujours aux organisations soucieuses de conserver la maîtrise de leurs données. Cette situation freine la mise en place d'une stratégie de cyberdéfense proactive et autonome.

C'est dans cette optique que s'inscrit le présent projet de fin d'études, réalisé à l'Institut Supérieur d'Informatique de Mahdia (ISIMa), dans le cadre du Mastère Professionnel en Computer Engineering, spécialité Expert Réseaux. L'objectif principal du projet est de concevoir, de réaliser et de valider un système de détection et de prévention des intrusions réseau de nouvelle génération, désigné **NG-IDPS** (*Next-Generation Intrusion Detection and Prevention System*), capable de fonctionner de manière autonome dans un environnement local et isolé. Ce système vise à combiner plusieurs mécanismes de détec-

tion complémentaires, à analyser et à corréler les événements de sécurité, et à déclencher des actions de réponse maîtrisées, tout en restant accessible à des organisations disposant de moyens limités.

Le présent mémoire retrace les différentes étapes de conception, de développement et de validation de ce projet. Il est structuré en quatre chapitres, suivis d'une conclusion générale :

- **Le premier chapitre** pose le cadre du projet en présentant son contexte, sa problématique et ses objectifs, en étudiant les solutions existantes et en exposant la méthodologie de travail retenue.
- **Le deuxième chapitre** est consacré à la phase préparatoire : l'analyse et la spécification des besoins fonctionnels et non fonctionnels, la planification selon la méthodologie agile Scrum, la justification des choix technologiques et la présentation de l'architecture générale.
- **Le troisième chapitre** traite de la première *Release*, dédiée à la détection : l'acquisition du trafic réseau, l'acheminement des données et la détection comportementale, ainsi que leur validation expérimentale.
- **Le quatrième chapitre** détaille la seconde *Release*, dédiée à la prévention et à la réponse : l'anticipation des menaces par le renseignement, la sensibilisation des utilisateurs, l'orchestration de la réponse automatisée et la supervision par un tableau de bord dédié.
- **La conclusion générale** dresse le bilan du travail réalisé, en discute les limites et propose des perspectives d'évolution.

À travers ce travail, nous avons cherché à apporter une réponse concrète aux besoins de protection des infrastructures réseau des organisations tunisiennes, en proposant une solution rigoureuse, maîtrisable localement et intégrable à des environnements existants.

CHAPITRE 1

PRÉSENTATION DU PROJET ET ÉTUDE DE L'EXISTANT

Sommaire

1.1 Introduction 23

1.2 Champ d'application et Contexte Opérationnel 23

1.3 Présentation du Projet 24

1.3.1 Contexte 24

1.3.2 Problématique 24

1.3.3 Objectifs du Projet 25

1.4 Étude et Analyse de l'Existant 29

1.4.1 Évolution des Systèmes de Détection et de Réponse 29

1.4.2 Analyse des Approches Existantes 31

1.4.3 Analyse Comparative des Solutions Représentatives 33

1.4.4 Limites des Solutions Existantes et Besoins Spécifiques du Projet 38

1.4.5 Positionnement de NG-IDPS 40

1.5 Méthodologie adoptée 41

1.5.1 Méthodologie retenue 41

1.5.2 Méthodologie Scrum 41

1.5.3 Organisation des Sprints 42

1.5.4 Les rôles au sein de Scrum 43

1.5.5 Processus de développement adopté 44

1.6 Conclusion 44

1.1 Introduction

L'évolution rapide des systèmes d'information, l'interconnexion croissante des environnements réseau et la diversification des cybermenaces ont transformé les exigences en matière de cybersécurité. Les mécanismes traditionnels, reposant notamment sur des règles prédéfinies et des mécanismes de détection déterministes, montrent certaines limites face à des attaques de plus en plus variées et automatisées. La protection des actifs numériques appelle désormais des approches complétant la détection classique par l'analyse comportementale, la corrélation multi-source et l'automatisation de certaines réponses. C'est dans cette perspective que s'inscrit le projet **NG-IDPS (Next-Generation Intrusion Detection and Prevention System)**, présenté dans ce chapitre.

Ce chapitre définit le champ d'application du projet, puis son contexte, sa problématique et ses objectifs. Une étude des solutions existantes permet ensuite d'identifier les besoins et les contraintes auxquels NG-IDPS cherche à répondre, avant la présentation des orientations de la solution et de la méthodologie de réalisation retenue. Une synthèse conclut le chapitre et introduit la spécification détaillée développée au chapitre suivant.

1.2 Champ d'application et Contexte Opérationnel

Le présent projet s'inscrit dans le cadre de l'obtention du diplôme de **Mastère Professionnel en Computer Engineering – Expert Réseaux**. Il porte sur l'étude, la conception et l'implémentation d'un système de détection et de prévention des intrusions réseau, désigné **NG-IDPS**, dans le domaine de la cybersécurité des infrastructures réseau. Le projet vise à intégrer plusieurs mécanismes complémentaires de détection, d'analyse, de corrélation et de réponse aux événements de sécurité.

Le système proposé adopte une approche hybride combinant une détection déterministe fondée sur les signatures réseau (**Suricata**), une analyse comportementale reposant sur un modèle **LSTM-VAE**, un triage cognitif assuré par un **modèle de langage local (LLM)** et une orchestration **SOAR (Security Orchestration, Automation and Response)**. Il intègre également un volet de **Threat Intelligence (CTI)** destiné à enrichir l'analyse et à contribuer à l'évolution contrôlée des mécanismes de détection ainsi qu'à la contextualisation des contenus de sensibilisation.

Le contexte opérationnel retenu est celui d'un environnement **on-premise et isolé**, déployé dans un laboratoire virtualisé sur un réseau **VMware Host-Only** (`192.168.56.0/24`). Le périmètre expérimental est limité à ce réseau : les scénarios d'attaque sont générés depuis une machine Kali Linux et les mécanismes de mitigation ciblent exclusivement les ressources du laboratoire. Le champ d'application de NG-IDPS couvre ainsi la chaîne **détection – analyse – corrélation – décision – réponse**, complétée par l'exploitation de renseignements sur les menaces.

1.3 Présentation du Projet

Cette section détaille le contexte d'émergence du projet NG-IDPS, formalise la problématique scientifique et opérationnelle qui le motive, puis énonce les objectifs fonctionnels et non fonctionnels fixés pour y répondre.

1.3.1 Contexte

L'évolution des systèmes d'information, l'interconnexion croissante des infrastructures et la diversification des cybermenaces renforcent les exigences en matière de détection et de prévention des intrusions. Les attaques contemporaines peuvent combiner des techniques connues et des comportements difficiles à identifier par les seuls mécanismes fondés sur des signatures ou des règles prédéfinies. Cette évolution a favorisé l'adoption d'approches complémentaires reposant notamment sur l'analyse comportementale, la corrélation de plusieurs sources d'information et l'automatisation de certaines opérations de sécurité.

Dans ce contexte, les architectures modernes de cybersécurité associent progressivement des mécanismes de détection, d'analyse, d'orchestration et de renseignement sur les menaces. Les solutions existantes proposent différentes combinaisons de ces capacités, selon leurs architectures, leurs modèles de déploiement et leurs possibilités d'intégration. Cette diversité met en évidence l'intérêt d'une approche capable d'articuler plusieurs mécanismes au sein d'une même chaîne opérationnelle.

Le projet NG-IDPS s'inscrit dans ce contexte en étudiant la conception d'une architecture de détection et de prévention adaptée à un environnement **on-premise et isolé**. Une attention particulière est accordée à la maîtrise locale des traitements, à la modularité de l'architecture et à l'intégration de mécanismes complémentaires de détection, d'analyse, de corrélation et de réponse.

1.3.2 Problématique

L'analyse du contexte précédent met en évidence plusieurs défis interdépendants liés à la conception d'un système de détection et de prévention adapté à un environnement local et contraint. Ces défis concernent notamment la rapidité de détection, la maîtrise des faux positifs, la détection de comportements inconnus, l'accessibilité des technologies et la continuité de la réponse.

Défi 1 — La contrainte de latence. La valeur opérationnelle d'une détection dépend en partie de sa capacité à produire une décision dans un délai compatible avec l'évolution de l'incident. L'intégration de plusieurs étapes d'analyse peut toutefois introduire des délais supplémentaires. Comment concevoir un pipeline capable d'effectuer l'analyse comportementale et la prise de décision dans des délais compatibles avec les exigences d'une détection quasi temps réel ?

Défi 2 — La maîtrise des faux positifs. Les mécanismes de détection comportementale peuvent produire des alertes correspondant à des activités légitimes mais inhabituelles. Un volume important de telles alertes peut augmenter la charge d'analyse et compliquer la priorisation des incidents. Comment calibrer le mécanisme de détection afin de maintenir une capacité suffisante à identifier les comportements suspects tout en limitant le nombre de fausses alertes ?

Défi 3 — La détection de comportements inconnus. Les mécanismes fondés sur des signatures reposent sur l'existence de règles correspondant aux comportements ou événements recherchés. Ils ne permettent donc pas, à eux seuls, d'identifier directement certains comportements nouveaux pour lesquels aucune règle de détection appropriée n'est disponible. Comment compléter la détection déterministe par une approche comportementale capable d'identifier des écarts par rapport aux comportements considérés comme normaux ?

Défi 4 — L'accessibilité et la maîtrise de l'infrastructure. Les plateformes de cybersécurité intégrant plusieurs capacités de détection, d'orchestration et d'analyse peuvent être associées à des modèles de licence, d'abonnement ou de déploiement qui ne correspondent pas nécessairement aux contraintes d'organisations disposant de ressources limitées ou nécessitant une maîtrise locale de leurs données. Comment concevoir une architecture reposant principalement sur des composants open-source et, pour ses fonctions critiques de détection, d'analyse et de réponse, sur des modèles auto-hébergés, tout en conservant une chaîne fonctionnelle cohérente et évolutive ?

Défi 5 — La continuité de la réponse. Une supervision humaine continue peut être difficile à assurer dans des environnements disposant de ressources limitées. Certaines situations peuvent ainsi nécessiter l'exécution automatisée d'actions de mitigation selon des conditions préalablement définies. Comment automatiser certaines décisions et actions de réponse tout en conservant des mécanismes de contrôle, de traçabilité, d'audit et de respect du principe de moindre privilège ?

Ces défis définissent ainsi l'espace de conception de ce travail : concevoir un système de détection et de prévention des intrusions combinant des mécanismes déterministes et comportementaux, intégrant l'analyse, la corrélation et la réponse automatisée, tout en restant adapté à un environnement local, isolé et soumis à des contraintes de ressources.

1.3.3 Objectifs du Projet

En réponse à la problématique formulée ci-dessus, les objectifs du projet NG-IDPS sont déclinés en deux catégories : des **objectifs fonctionnels (OF)**, décrivant les capacités que le système doit fournir, et des **objectifs non fonctionnels (ONF)**, définissant les principales contraintes de qualité et de performance auxquelles le système doit répondre.

Objectifs fonctionnels

Les objectifs fonctionnels du système NG-IDPS sont présentés dans le tableau 1.1, couvrant les principales capacités de détection, d'analyse, de réponse, de supervision et de renseignement sur les menaces.

TABLE 1.1 – Objectifs fonctionnels du système NG-IDPS

ID	Objectif	Description
OF1	**Capture et extraction réseau**	Mettre en place une chaîne de capture et d'extraction de caractéristiques réseau à partir des flux et événements générés par le système de détection réseau, afin d'alimenter les mécanismes d'analyse comportementale.
OF2	**Détection comportementale**	Concevoir, entraîner et déployer un modèle de détection d'anomalies comportementales non supervisé fondé sur une architecture LSTM-VAE, capable de fonctionner en inférence continue sur les données réseau.
OF3	**Orchestration et réponse**	Implémenter un moteur SOAR capable de corréler les alertes issues des canaux de détection déterministe et comportemental, d'appliquer des politiques de sécurité définies et de déclencher, lorsque les conditions prévues sont satisfaites, une action automatisée de mitigation réseau par `iptables`.
OF4	**Analyse cognitive**	Intégrer un agent d'analyse et de contextualisation des incidents fondé sur un modèle de langage exécuté localement, produisant un diagnostic structuré comprenant notamment le titre, le résumé exécutif, la sévérité, la technique MITRE ATT&CK identifiée et les recommandations associées.

ID	Objectif	Description
OF5	**Supervision SOC**	Développer un tableau de bord SOC permettant la supervision humaine, la visualisation des incidents et l'intervention manuelle, notamment pour la validation ou l'infirmation d'une action de blocage lorsqu'un opérateur est disponible.
OF6	**Veille et renseignement sur les menaces**	Mettre en œuvre un module de Cyber Threat Intelligence (CTI) capable d'agréger des sources OSINT, d'analyser les informations pertinentes, de contribuer à la génération de règles de détection candidates et de produire, lorsque cela est pertinent, des contenus de sensibilisation ciblés.
OF7	**Sensibiliser les utilisateurs aux menaces à caractère humain.**	Le système doit générer, à partir d'un renseignement CTI classé comme relevant du facteur humain, un parcours de formation ciblé (contenu pédagogique, diagramme explicatif, questionnaire de validation), le diffuser aux utilisateurs concernés et suivre les résultats obtenus. La génération de contenu s'appuie sur une chaîne de repli à trois niveaux – deux services de langage accessibles par API dans le nuage, complétés par un modèle exécuté localement en dernier recours – afin de garantir la continuité du service pédagogique.

Objectifs non fonctionnels

Les objectifs non fonctionnels du système NG-IDPS sont présentés dans le tableau 1.2, en précisant les principales exigences de performance, de fiabilité, de latence, d'autonomie et de maîtrise de l'infrastructure.

TABLE 1.2 – Objectifs non fonctionnels du système NG-IDPS

ID	Critère	Exigence / Cible
ONF1	**Performance de détection**	Assurer, à l'échelle de la chaîne de détection complète (signatures Suricata et analyse comportementale LSTM-VAE combinées), la détection de l'ensemble des classes d'attaques couvertes par le protocole de validation défini au chapitre 3, chaque mécanisme couvrant les angles morts structurels de l'autre. Cet objectif est validé par un test en conditions réelles (rejeu chronologique de trafic de production, chapitre 3), ayant atteint un taux de détection de 100 % sur les six classes d'attaques exécutées (balayage de ports rapide et lent, identification de services, force brute SSH, scan applicatif web, inondation volumétrique).
ONF2	**Fiabilité**	Maintenir, à l'échelle de la chaîne de décision complète (détection, persistance temporelle et filtrage par liste blanche), un taux de déclenchement erroné négligeable sur trafic réel, afin de préserver la pertinence opérationnelle des actions de réponse automatisées. Cet objectif est validé par le même test en conditions réelles, mesurant un taux de 0,05 % de fenêtres bénignes signalées à tort sur plus de trois heures de trafic de production, sans qu'aucune n'ait atteint le seuil de persistance déclenchant une action.
ONF3	**Latence d'inférence**	Atteindre un temps d'inférence du modèle de détection comportementale inférieur à 15 ms par séquence, afin de permettre son intégration dans une chaîne de traitement en flux continu. Cette cible concerne l'inférence du modèle et non la latence de l'ensemble de la chaîne NG-IDPS.

ID	Critère	Exigence / Cible
ONF4	**Autonomie opérationnelle**	Permettre une capacité de réponse automatisée continue, 24 heures sur 24 et 7 jours sur 7, y compris en l'absence d'un opérateur humain connecté au tableau de bord, dans le respect des politiques de sécurité définies et des mécanismes de contrôle prévus.
ONF5	**Maîtrise de l'infrastructure et soutenabilité**	Privilégier des composants open-source et des modèles d'intelligence artificielle auto-hébergés pour les fonctions opérationnelles critiques, notamment la détection, l'analyse et la réponse aux incidents, afin de limiter les coûts de licence et la dépendance aux services cloud propriétaires.

1.4 Étude et Analyse de l'Existant

La justification des choix architecturaux retenus suppose d'analyser les approches de sécurité réseau déployées dans l'industrie et d'identifier leurs principales limites face aux contraintes du très haut débit et aux exigences de confidentialité des données.

1.4.1 Évolution des Systèmes de Détection et de Réponse

Les systèmes de cybersécurité ont progressivement évolué d'une détection principalement fondée sur des signatures vers des approches combinant analyse comportementale, corrélation multi-source, renseignement sur les menaces et automatisation de la réponse, afin de réduire les limites des mécanismes reposant sur une seule source d'information.

Le tableau 1.3 synthétise les principales approches rencontrées dans les solutions de sécurité modernes.

TABLE 1.3 – Évolution des approches de détection et de réponse

Approche	**Principe**	**Apport**	**Limite principale**
NIDS / détection par signatures	Identification d'activités malveillantes à partir de signatures et de règles connues	Détection déterministe des menaces déjà caractérisées	Dépendance à la disponibilité et à la qualité des signatures
Détection comportementale	Identification d'écarts par rapport à un comportement considéré comme normal	Possibilité de détecter des comportements nouveaux ou atypiques	Sensibilité aux variations légitimes du trafic et aux changements de distribution
SIEM / XDR	Centralisation et corrélation d'événements provenant de plusieurs sources	Vision consolidée des événements de sécurité et amélioration de l'investigation	Complexité d'intégration et volume important d'événements à traiter
SOAR	Orchestration des processus d'analyse et d'actions de réponse	Automatisation des opérations répétitives et accélération de la réponse	Nécessité de définir des politiques et des garde-fous pour limiter les actions incorrectes
IA / assistance cognitive	Analyse et contextualisation des informations de sécurité à l'aide de modèles d'apprentissage ou de modèles de langage	Aide à l'interprétation des incidents et à la production de recommandations	Dépendance à la qualité des données et nécessité de contrôler les décisions générées

Synthèse. Ces approches ne sont pas concurrentes mais complémentaires : la détection par signatures fournit une première source d'évidence déterministe, l'analyse comportementale permet d'examiner des écarts non associés à une signature connue, et les méca-

nismes de corrélation et d'orchestration combinent ces informations pour coordonner la réponse. Cette évolution conduit vers des architectures multi-capteurs dans lesquelles détection, analyse, contextualisation et réponse sont réparties entre mécanismes spécialisés.

1.4.2 Analyse des Approches Existantes

L'analyse des principales approches utilisées dans les systèmes de sécurité met en évidence plusieurs mécanismes complémentaires destinés à améliorer la détection, l'investigation et la réponse aux incidents. Ces approches diffèrent notamment par la nature des données exploitées, le niveau d'automatisation et les contraintes liées à leur déploiement. Le tableau 1.4 en présente une synthèse.

TABLE 1.4 – Analyse synthétique des principales approches de sécurité

Approche	**Fonction principale**	**Limites ou contraintes**
Détection par signatures	Identifier des attaques connues à partir de règles et de signatures préétablies.	Ne permet pas, à elle seule, de couvrir efficacement certains comportements nouveaux ou non caractérisés.
Détection comportementale	Identifier des écarts par rapport au comportement attendu d'une activité ou d'un flux réseau.	Peut être sensible aux variations légitimes du trafic et nécessite une modélisation adaptée du comportement normal.
Corrélation multi-source	Regrouper et mettre en relation des événements provenant de plusieurs capteurs ou composants de sécurité.	Augmente la richesse du contexte mais introduit des contraintes de synchronisation, de corrélation et de traitement des événements.
Analyse assistée par IA	Analyser, contextualiser et synthétiser les informations issues des événements de sécurité.	La pertinence des résultats dépend de la qualité du contexte fourni et nécessite un contrôle des décisions ou recommandations générées.

Approche	**Fonction principale**	**Limites ou contraintes**
Réponse automatisée	Exécuter automatiquement des actions de mitigation à partir de règles ou de conditions prédéfinies.	Une automatisation insuffisamment contrôlée peut entraîner des actions inappropriées ou affecter des actifs légitimes.
Threat Intelligence	Enrichir la détection à partir d'informations externes relatives aux menaces, vulnérabilités et techniques d'attaque.	La qualité, la pertinence et l'actualité des sources influencent directement la valeur opérationnelle des informations obtenues.
Déploiement on-premise	Maintenir les composants et les données de sécurité dans une infrastructure contrôlée localement.	Nécessite la disponibilité des ressources matérielles, logicielles et d'administration nécessaires au fonctionnement de la plateforme.

Ces différentes approches couvrent des fonctions complémentaires de la chaîne de défense. La détection par signatures apporte une capacité déterministe, tandis que la détection comportementale permet d'identifier des activités s'écartant des comportements attendus. La corrélation multi-source rapproche ces différentes observations afin d'enrichir le contexte de l'incident, tandis que l'analyse assistée par IA contribue à leur interprétation et à leur synthèse.

La réponse automatisée étend cette chaîne vers l'exécution d'actions de mitigation selon des conditions et des politiques préalablement définies. La Threat Intelligence contribue quant à elle à l'enrichissement du contexte de sécurité et peut participer à l'évolution contrôlée des mécanismes de détection ainsi qu'à la contextualisation des actions de sensibilisation. Enfin, le déploiement on-premise répond à des contraintes de maîtrise locale des données et des traitements, tout en impliquant des exigences propres à l'infrastructure d'hébergement.

Cette complémentarité des fonctions constitue la base de la comparaison des solutions représentatives présentée dans la section suivante.

1.4.3 Analyse Comparative des Solutions Représentatives

Afin de situer NG-IDPS par rapport aux solutions existantes, une comparaison est réalisée à partir de plusieurs critères liés aux objectifs du projet, portant sur les capacités de détection, d'analyse et de réponse ainsi que sur les contraintes de déploiement et de maîtrise des données.

1.4.3.1 Critères retenus.

Les critères considérés sont les suivants :

- **Détection comportementale :** capacité à identifier des comportements ou activités atypiques au-delà des signatures connues.
- **Corrélation multi-source :** capacité à rapprocher des événements provenant de plusieurs sources de sécurité.
- **Analyse assistée par IA :** utilisation de mécanismes d'intelligence artificielle pour l'analyse, la contextualisation ou l'investigation des incidents.
- **Réponse automatisée :** capacité à déclencher ou orchestrer des actions de mitigation à partir des événements détectés.
- **Threat Intelligence :** intégration ou exploitation d'informations relatives aux menaces, vulnérabilités et techniques d'attaque.
- **Déploiement local :** possibilité de déployer les composants de sécurité dans une infrastructure contrôlée localement.
- **Dépendance au cloud :** degré de dépendance à des services cloud pour les fonctions principales de sécurité.
- **Positionnement architectural :** orientation générale de la solution, notamment NIDS, XDR, SIEM, SOAR ou plateforme de sécurité intégrée.

TABLE 1.5 – Comparaison de NG-IDPS avec des plateformes de référence

Critère	**Darktrace**	**CrowdStrike**	**Security Onion**	**Wazuh**	**NG-IDPS**
A. Périmètre et détection					
Périmètre de couverture	Multi-couches (réseau, endpoint, cloud, identité)	Multi-couches (endpoint, cloud, identité)	Réseau + hôte (via Elastic Agent / osquery)	Endpoint + hôte (agent-based)	**Réseau uniquement**
Détection signature déterministe	Non native (approche comportementale)	Non native	Oui (Suricata / Zeek)	Non native	**Oui (Suricata)**
Détection comportementale	Oui (ML propriétaire)	Oui (ML propriétaire)	Oui (Zeek + règles de corrélation)	Oui (FIM, analyse de logs, anomalies)	**Oui (LSTM-VAE, apprentissage profond)**
B. Corrélation et cognition					
Corrélation multi-source automatisée	Oui	Oui	Oui, selon les composants et intégrations utilisés	Oui, selon les composants et intégrations utilisés	**Oui : corrélation via Kafka et moteur SOAR**
Analyse cognitive assistée par LLM	Selon les capacités et intégrations de la plateforme	Selon les capacités et intégrations de la plateforme	Selon les composants et intégrations utilisés	Selon les intégrations disponibles	**Oui : LLM local (Ollama) pour l'analyse et la contextualisation des incidents**

Suite sur la page suivante

Critère	**Darktrace**	**CrowdStrike**	**Security Onion**	**Wazuh**	**NG-IDPS**
Réponse automatisée (SOAR)	Oui, selon les capacités de la plateforme	Oui, selon les capacités de la plateforme	Selon les intégrations et l'architecture retenue	Oui, via Active Response et intégrations	**Oui : politiques de réponse à plusieurs niveaux et mode automatisé**
Threat Intelligence et exploitation opération-nelle	Oui	Oui	Oui, via flux et intégrations CTI	Oui, via intégrations CTI	**Oui : enri-chissement CTI/RAG local, contextua-lisation des menaces et alimenta-tion de la chaîne de prévention**
Génération et adaptation de règles à partir de la CTI	Selon les capacités de la plateforme	Selon les capacités de la plateforme	Oui, via les mécanismes de règles et d'intégration disponibles	Oui, via règles et mécanismes de personna-lisation	**Oui : proposition de règles Suricata et déploie-ment contrôlé après validation**

Suite sur la page suivante

Critère	**Darktrace**	**CrowdStrike**	**Security Onion**	**Wazuh**	**NG-IDPS**
Formation de sécurité contextualisée par incident / CTI	Selon les fonctionnalités et intégrations disponibles	Selon les fonctionnalités et intégrations disponibles	Non native	Non native	**Oui : génération automatique ou proposition conditionnelle selon la pertinence CTI**
Déploiement local / maîtrise des données	Selon l'architecture et l'édition	Selon l'architecture et les composants retenus	Oui	Oui	**Oui : architecture majoritairement locale, avec dépendances externes configurables**
Remplaçabilité technologique	Selon les capacités de la plateforme	Selon les capacités de la plateforme	Oui, via composants et intégrations	Oui, via composants et intégrations	**Élevée, sous respect des contrats d'interface**
Personnalisation du moteur de détection	Selon capacités de la plateforme	Selon capacités de la plateforme	Élevée via règles et intégrations	Élevée via agents et règles	**Élevée : capteur réseau et chaîne de détection adaptables**

Suite sur la page suivante

Critère	**Darktrace**	**CrowdStrike**	**Security Onion**	**Wazuh**	**NG-IDPS**
Personnalisation du moteur ML	Propriétaire	Propriétaire	Selon composants utilisés	Selon composants utilisés	**Oui : modèle d'apprentissage remplaçable**
Personnalisation du moteur LLM	Selon capacités de la plateforme	Selon capacités de la plateforme	Selon édition / intégrations	Selon intégrations	**Oui : runtime et modèle LLM interchangeables**
Séparation physique des fonctions	Selon déploiement	Selon déploiement	Selon architecture	Selon architecture	**Oui : VM-Edge / VM-ML / VM-Response**
Indépendance technologique	Faible à modérée	Faible à modérée	Élevée	Élevée	**Élevée : technologies remplaçables selon les interfaces du pipeline**

1.4.3.2 Interprétation.

L'analyse comparative met en évidence la diversité des capacités proposées par les solutions existantes de détection et de réponse aux incidents — détection par signatures, analyse comportementale, corrélation d'événements, exploitation du renseignement sur les menaces (*Cyber Threat Intelligence – CTI*) et automatisation de certaines actions d'investigation et de réponse — intégrées selon des architectures et des niveaux de couverture différents.

Sur le plan de la détection, les approches traditionnelles conservent un rôle important pour l'identification déterministe des menaces connues, que les plateformes plus récentes complètent par des approches comportementales visant les activités difficilement caractérisables par des signatures statiques. L'intégration du renseignement sur les menaces constitue également une tendance importante, mais sa valeur opérationnelle dépend de la manière dont les informations

CTI sont exploitées dans le cycle de détection et de réponse : enrichissement des événements, contextualisation des alertes, investigation ou adaptation des mécanismes de détection. La comparaison montre par ailleurs que l'automatisation dépasse progressivement le simple déclenchement d'alertes, certaines solutions intégrant des mécanismes d'orchestration, de corrélation et de confinement enchaînant plusieurs actions à partir d'un événement de sécurité.

L'analyse du tableau met ainsi en évidence un enjeu architectural plutôt qu'une simple opposition entre solutions disposant ou non d'une fonctionnalité donnée : ces capacités peuvent exister individuellement dans plusieurs solutions, mais leur combinaison et leur niveau d'intégration au sein d'un même flux opérationnel diffèrent sensiblement. Le positionnement de NG-IDPS s'inscrit dans cette problématique d'intégration : le projet articule détection déterministe, analyse comportementale, CTI, analyse cognitive et réponse orchestrée au sein d'une chaîne cohérente, sans chercher à reproduire isolément une fonctionnalité déjà présente ailleurs. Ces éléments sont détaillés dans la section suivante, consacrée aux limites des solutions existantes et aux besoins spécifiques de NG-IDPS.

1.4.4 Limites des Solutions Existantes et Besoins Spécifiques du Projet

L'analyse comparative précédente montre que les solutions existantes couvrent, selon leur orientation, plusieurs capacités de détection, d'analyse, de renseignement, de corrélation et de réponse, mais que leur intégration, leur mode de déploiement et leur niveau d'automatisation diffèrent sensiblement. Pour NG-IDPS, l'objectif n'est donc pas de considérer ces fonctionnalités comme absentes des solutions existantes, mais d'identifier les contraintes résultant de leur combinaison dans un environnement on-premise et isolé. Le tableau 1.6 synthétise ces constats.

TABLE 1.6 – Limites identifiées et besoins spécifiques de NG-IDPS

Constat	**Limite ou contrainte**	**Besoin identifié pour NG-IDPS**
Détection fondée sur les signatures et les règles	Difficulté à identifier certains comportements nouveaux ou ne correspondant pas à des signatures connues	Compléter la détection déterministe par une analyse comportementale fondée sur l'apprentissage automatique
Multiplication des sources d'observation et d'alerte	Les événements issus de mécanismes de détection différents peuvent être analysés séparément, ce qui complique leur contextualisation et leur priorisation	Corréler plusieurs sources d'évidence au sein d'un mécanisme commun d'analyse et d'orchestration

Constat	**Limite ou contrainte**	**Besoin identifié pour NG-IDPS**
Détection comportementale et apprentissage automatique	Les anomalies produites par un modèle nécessitent une contextualisation et une interprétation avant de pouvoir être exploitées comme éléments de décision	Associer l'inférence comportementale à une étape d'analyse et de triage permettant d'interpréter les observations produites
Utilisation de modèles d'intelligence artificielle	Nécessité de conserver la maîtrise des données traitées, du contexte d'analyse et des résultats produits dans un environnement de sécurité contraint	Privilégier une capacité d'analyse cognitive locale, intégrée au processus opérationnel du système
Threat Intelligence et sources OSINT	L'exploitation opérationnelle des informations de renseignement dépend de leur intégration avec les mécanismes de détection et d'analyse	Intégrer la CTI/OSINT dans une boucle d'analyse permettant l'enrichissement, la contextualisation et, lorsque les conditions sont réunies, l'adaptation contrôlée des mécanismes de détection
Automatisation de la réponse	Une action de réponse fondée sur une observation isolée peut présenter un risque de réponse disproportionnée ou d'affectation d'un actif légitime	Soumettre les actions de réponse à des règles de décision, à la corrélation des éléments d'évidence et à des mécanismes de protection
Fonctionnement opérationnel d'un SOC	Certaines décisions de sécurité nécessitent une interprétation humaine, alors que certaines actions répétitives peuvent être automatisées sous conditions	Permettre une automatisation contrôlée des actions de réponse tout en conservant des mécanismes d'intervention et de validation humaine lorsque cela est nécessaire

Constat	**Limite ou contrainte**	**Besoin identifié pour NG-IDPS**
Environnement on-premise et réseau isolé	Contraintes liées à la maîtrise de l'infrastructure, à la circulation des données et à la dépendance vis-à-vis de services externes	Concevoir une architecture déployable localement et adaptée à un environnement réseau isolé
Évolution et remplacement des composants	L'évolution des mécanismes de détection, d'analyse ou de réponse peut nécessiter le remplacement de composants technologiques	Maintenir une architecture modulaire fondée sur des interfaces et des responsabilités fonctionnelles clairement définies

Ces constats conduisent à retenir, pour NG-IDPS, une architecture fondée sur la complémentarité de plusieurs mécanismes plutôt que sur un détecteur unique : détection déterministe, analyse comportementale, corrélation de plusieurs sources d'évidence, exploitation de la CTI, analyse cognitive et réponse orchestrée soumise à des politiques de sécurité. Cette combinaison doit rester compatible avec les contraintes de déploiement du projet et préserver la modularité du système : les technologies retenues constituent des implémentations des différentes fonctions de l'architecture, organisées autour de responsabilités et d'interfaces fonctionnelles permettant leur évolution ou leur remplacement.

Ces besoins constituent la base du positionnement de NG-IDPS, développé ci-après en relation avec les limites identifiées et les exigences dégagées de l'étude de l'existant.

1.4.5 Positionnement de NG-IDPS

À partir des besoins identifiés, NG-IDPS se positionne comme une architecture intégrée et modulaire de détection, d'analyse et de réponse, associant détection déterministe par signatures, analyse comportementale par apprentissage automatique, analyse cognitive locale des incidents et orchestration. Cette dernière assure notamment la corrélation des alertes et des informations issues des différents mécanismes de détection, l'application des politiques de sécurité et le déclenchement des réponses prévues. L'approche repose sur une séparation claire des responsabilités : la détection fournit les premières observations, l'analyse comportementale apporte une perspective complémentaire, l'orchestration assure la corrélation et la mise en œuvre des politiques de réponse, tandis que l'analyse cognitive contextualise et structure les informations relatives aux incidents sans constituer à elle seule le mécanisme de décision.

NG-IDPS est conçu pour fonctionner dans un environnement on-premise isolé tout en conservant une organisation modulaire. Les technologies retenues constituent des implémentations de référence des différentes fonctions du système et peuvent être remplacées lorsque le composant substitué respecte les interfaces et les contraintes attendues par l'architecture. Le positionnement

du projet repose ainsi moins sur une fonctionnalité technologique isolée que sur l'intégration cohérente de mécanismes complémentaires au sein d'une même chaîne opérationnelle.

1.5 Méthodologie adoptée

Compte tenu de la diversité des fonctionnalités à développer, de l'intégration progressive de plusieurs technologies et de la nécessité de valider régulièrement les différentes composantes du système, une approche agile inspirée du framework Scrum a été retenue pour mener le projet NG-IDPS. Cette approche est adaptée au caractère évolutif du projet et permet d'organiser le développement selon une progression incrémentale.

1.5.1 Méthodologie retenue

L'approche Agile a été choisie pour sa capacité à accompagner un développement progressif et itératif. Contrairement à une approche strictement séquentielle, dans laquelle l'ensemble des besoins et des choix techniques sont définis en amont, elle permet de construire la solution par étapes, de valider les fonctionnalités au fur et à mesure et d'ajuster les priorités en fonction des résultats obtenus.

Cette approche est particulièrement adaptée à NG-IDPS, dont l'architecture repose sur plusieurs composantes interdépendantes, notamment l'acquisition réseau, le traitement des flux, la détection comportementale, l'orchestration SOAR, l'analyse cognitive, la CTI et la supervision. Le développement est ainsi organisé autour de cycles courts alternant analyse, conception, implémentation, intégration et validation. Cette organisation facilite l'identification précoce des difficultés techniques et permet d'ajuster progressivement les choix d'implémentation.

1.5.2 Méthodologie Scrum

Le framework Scrum structure le développement sous forme d'itérations successives appelées *Sprints*, chacune étant associée à un ensemble d'objectifs et de fonctionnalités clairement définis. Dans le cadre de ce projet individuel, les principes de Scrum sont adaptés afin de conserver une organisation itérative, une priorisation explicite du travail et une validation régulière des résultats.

Le développement de NG-IDPS a été organisé autour de deux *Releases*, correspondant aux deux axes fonctionnels principaux du projet : la **détection intelligente**, comprenant l'acquisition des événements réseau, le pipeline de données et la détection comportementale par LSTM-VAE, et la **prévention et la réponse**, comprenant l'orchestration SOAR, l'analyse cognitive par LLM, les mécanismes de contrôle de sécurité, la Threat Intelligence, la formation contextualisée, la supervision et l'automatisation de la réponse.

Cette organisation permet de construire progressivement la plateforme, depuis l'acquisition et l'analyse des événements jusqu'à la mise en œuvre d'actions de prévention et de réponse.

1.5.3 Organisation des Sprints

Sur une durée de projet de six mois, le travail a été réparti en six Sprints mensuels, regroupés en deux Releases comme présenté dans le tableau suivant.

TABLE 1.7 – Organisation des Sprints du projet NG-IDPS

Release	Sprint	Thématique	Principales activités
Release 1 Détection	**Sprint 1**	Acquisition réseau	Mise en place de Suricata, collecte des événements EVE JSON et développement de la chaîne d'acquisition réseau.
	Sprint 2	Pipeline de données	Mise en place du streaming des événements, traitement des flux et intégration de Kafka dans la chaîne de communication.
	Sprint 3	Détection comportementale	Sélection et préparation des caractéristiques, entraînement du LSTM-VAE, calibration du seuil d'anomalie et mise en œuvre de l'inférence en flux continu.
Release 2 Prévention et réponse	**Sprint 4**	Anticipation par le renseignement	Intégration de l'agent CTI et du triage Dual-RAG, génération automatique de règles Suricata candidates et aiguillage du renseignement vers la sensibilisation.

Release	Sprint	Thématique	Principales activités
	Sprint 5	Mitigation du risque humain	Intégration de l'agent Gemini et de la plateforme RAG LMS, génération de parcours de sensibilisation ciblés et suivi de conformité des employés.
	Sprint 6	Orchestration SOAR multi-paliers et supervision	Mise en œuvre de la corrélation des alertes, de l'orchestration SOAR autonome et supervisée, de l'agent d'analyse post-attaque et du tableau de bord SOC.

Cette organisation permet de faire évoluer progressivement le système d'une fonction de détection vers une plateforme intégrant également des capacités de prévention, de réponse et d'exploitation du renseignement sur les menaces. Elle facilite également l'intégration progressive des différents composants en limitant la complexité de chaque étape de développement.

1.5.4 Les rôles au sein de Scrum

Le framework Scrum définit plusieurs responsabilités permettant d'assurer une organisation claire du travail. Dans le cadre d'un projet réalisé individuellement, ces responsabilités sont adaptées au contexte académique du projet tout en conservant leur distinction conceptuelle.

TABLE 1.8 – Rôles Scrum dans le cadre du projet

Rôle	**Responsabilités dans le projet**
Product Owner : Institut Supérieur d'Informatique de Mahdia	Définition et priorisation des fonctionnalités à développer, identification des besoins fonctionnels et techniques, suivi des objectifs de chaque Sprint et validation des fonctionnalités réalisées.

Rôle	**Responsabilités dans le projet**
Scrum Master : Mr. Hamdi Aloulou	Organisation du processus de développement, suivi de l'avancement des Sprints, identification des obstacles techniques et maintien d'une démarche de travail itérative.
Développeur : Aziz Dhifaoui	Analyse, conception, implémentation, intégration, tests et documentation des différents composants de la plateforme NG-IDPS.

Dans le contexte d'un projet réalisé individuellement, ces rôles sont assumés par le même membre du projet, tout en conservant leur séparation conceptuelle afin de maintenir une structure de travail inspirée de Scrum : suivi des priorités, planification des itérations, identification des obstacles et validation progressive des fonctionnalités.

1.5.5 Processus de développement adopté

Chaque Sprint suit un cycle itératif comprenant la définition des objectifs, l'analyse et la conception, l'implémentation, l'intégration, les tests et la validation des fonctionnalités réalisées. À la fin de chaque itération, les résultats sont examinés afin d'identifier les éléments réalisés, les difficultés rencontrées et les ajustements nécessaires pour le cycle suivant. Cette organisation permet de détecter plus tôt les problèmes techniques et de réduire les risques liés à une intégration tardive des composants.

La méthodologie adoptée structure ainsi le développement de NG-IDPS autour d'une progression incrémentale, en cohérence avec les objectifs fonctionnels, les choix architecturaux et les contraintes techniques du projet.

1.6 Conclusion

Ce premier chapitre a situé le projet NG-IDPS dans le contexte actuel de la cybersécurité et mis en évidence les principaux enjeux liés à la détection et à la prévention des intrusions réseau. L'analyse du contexte et de la problématique a montré que l'évolution des menaces justifie de compléter les mécanismes de détection déterministe par des approches capables de

prendre en compte les comportements connus, les anomalies et les informations issues de sources complémentaires. L'étude des solutions existantes a ensuite permis d'identifier les différentes approches adoptées en matière de détection, d'analyse, de Threat Intelligence et de réponse aux incidents, ainsi que les besoins spécifiques liés à leur intégration dans un environnement local et isolé et au contrôle des mécanismes d'automatisation.

De ces constats découlent les besoins retenus pour NG-IDPS : associer détection par signatures et analyse comportementale, corréler plusieurs sources d'évidence, intégrer une analyse cognitive locale, exploiter la Threat Intelligence dans le cycle opérationnel et automatiser certaines actions de réponse sous des politiques de sécurité et des mécanismes de contrôle. Ces capacités sont intégrées au sein d'une architecture modulaire permettant l'évolution ou le remplacement des composants technologiques lorsque les interfaces et les fonctions attendues sont préservées.

NG-IDPS est ainsi positionné comme une architecture intégrée articulant détection déterministe, détection comportementale, orchestration SOAR, analyse cognitive, CTI et mécanismes de prévention, tout en intégrant un volet de sensibilisation contextualisée. L'ensemble établit une continuité entre l'observation du trafic, l'identification des comportements suspects, l'interprétation des incidents et l'application contrôlée des réponses, au sein d'une architecture distribuée fondée sur des interfaces fonctionnelles dédiées. Cette étude constitue ainsi la base conceptuelle de la solution proposée ; le chapitre suivant en définit l'architecture globale, l'organisation des composants et les principaux flux de données.

CHAPITRE 2

PHASE PRÉPARATOIRE ET INGÉNIERIE SYSTÈME

Sommaire

2.1 Introduction 48

2.2 Analyse et spécification des besoins 48

2.2.1 Identification des acteurs 49

2.2.2 Besoins fonctionnels 49

2.2.3 Besoins non fonctionnels 51

2.3 Diagramme des cas d'utilisation globale 53

2.4 Backlog du produit 53

2.5 Planification des sprints 56

2.6 Environnement logiciel 57

2.6.1 Démarche de comparaison 57

2.6.2 Infrastructure de virtualisation et système d'exploitation 58

2.6.3 Détection d'intrusions réseau 59

2.6.4 Collecte, indexation et visualisation des journaux 60

2.6.5 Transport de données en flux 61

2.6.6 Détection d'anomalies par apprentissage profond 63

2.6.7 IA générative, RAG et renseignement sur les menaces 65

2.6.8 Prévention et réponse automatisée 67

2.6.9 Serveur applicatif et communication temps réel 68

2.6.10 Interface de supervision 69

2.6.11 Persistance des données 70

2.6.12 Gestion de projet, modélisation et rédaction 71

2.6.13 Synthèse de la pile technologique 72
2.7 Architecture générale 73
2.8 Conclusion 75

2.1 Introduction

Avant d'aborder l'implémentation du moteur de détection, plusieurs décisions préalables doivent être établies afin de définir précisément le périmètre fonctionnel de NG-IDPS, les contraintes auxquelles le système doit répondre et les choix technologiques permettant sa réalisation. Cette phase préparatoire vise ainsi à transformer les objectifs définis au chapitre **??** en exigences fonctionnelles et non fonctionnelles, puis à les traduire en éléments d'architecture et de planification.

Dans un contexte caractérisé par des ressources matérielles et financières limitées, ces décisions doivent également tenir compte des contraintes réelles de déploiement et d'exploitation du système. Il s'agit notamment de déterminer les acteurs et les interactions avec la plateforme, les fonctionnalités attendues, les exigences de performance et de sécurité, ainsi que les composants logiciels nécessaires à la mise en œuvre de la chaîne de détection, d'analyse, de corrélation et de réponse.

Ce chapitre présente ainsi la démarche d'ingénierie suivie avant la phase de réalisation. La section 2.1 établit l'analyse et la spécification des besoins fonctionnels et non fonctionnels. Ces besoins sont ensuite représentés à travers le diagramme de cas d'utilisation présenté dans la section 2.1, puis structurés sous la forme d'un backlog produit dans la section 2.1. La section 2.1 présente la planification du développement à travers les différents Sprints retenus pour le projet.

La section 2.1 décrit ensuite l'environnement de développement ainsi que les technologies retenues pour les différentes fonctions du système, en tenant compte des contraintes matérielles, logicielles et de déploiement. Enfin, la section 2.1 présente l'architecture générale de NG-IDPS et les principaux flux entre ses composants. Cette architecture constitue le cadre de référence pour les phases d'implémentation et d'intégration développées dans les chapitres suivants.

2.2 Analyse et spécification des besoins

Les besoins du système ont été identifiés à partir de deux sources complémentaires : d'une part, la problématique des centres opérationnels de sécurité (*Security Operations Center*, SOC) présentée au chapitre 1, notamment les enjeux liés à la surcharge des analystes et aux délais de réaction [1], d'autre part, les contraintes propres à l'environnement d'expérimentation retenu pour le projet.

La spécification des besoins s'appuie sur les principes de formulation des exigences définis par la norme ISO/IEC/IEEE 29148 [2]. Chaque exigence est associée à un identifiant unique et formulée de manière à être compréhensible, vérifiable et traçable. La formulation des exigences est également indépendante, autant que possible, des choix d'implémentation, afin de distinguer les fonctionnalités attendues des solutions techniques retenues pour les réaliser.

L'analyse débute par l'identification des acteurs susceptibles d'interagir directement ou indirectement avec NG-IDPS. Les besoins fonctionnels et non fonctionnels sont ensuite dérivés de ces interactions et structurés afin de permettre leur association aux cas d'utilisation, aux éléments du backlog et, ultérieurement, aux composants de l'architecture et aux tests de validation.

2.2.1 Identification des acteurs

Dans le cadre de la modélisation UML, un acteur désigne une entité extérieure au système qui interagit avec celui-ci afin de déclencher une fonctionnalité ou de lui fournir des informations [3]. L'identification des acteurs permet ainsi de délimiter le périmètre fonctionnel de NG-IDPS et de définir les principales interactions représentées dans le diagramme de cas d'utilisation présenté à la section 2.3.

Trois catégories d'acteurs externes sont retenues pour NG-IDPS : l'administrateur SOC, l'employé et les sources de renseignement. Leurs rôles et leurs interactions principales avec le système sont présentés dans le tableau 2.1.

TABLE 2.1 – Acteurs de NG-IDPS

Acteur	**Type**	**Rôle et interaction**
Administrateur SOC	Principal	Analyste ou responsable de la sécurité. Il supervise l'état du système, consulte les alertes et les événements de sécurité, et peut intervenir dans le processus de réponse depuis l'interface de supervision.
Employé	Principal	Collaborateur de l'organisation susceptible d'être concerné par un incident de sécurité. Il peut recevoir des communications ou des actions de sensibilisation générées par le système.
Sources de renseignement	Externe	Sources externes de renseignement sur les menaces, notamment les flux OSINT et les bases de règles publiques, susceptibles d'alimenter les mécanismes de renseignement sur les menaces du système.

2.2.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les services et comportements attendus de NG-IDPS. Ils sont regroupés selon les acteurs qui interagissent avec le système lorsque cette interaction est pertinente. Les besoins internes au fonctionnement autonome de la plateforme sont regroupés séparément sous la responsabilité du système, et non considérés comme des acteurs UML.

Chaque besoin est associé à un identifiant unique. Ces identifiants sont repris dans le backlog présenté à la (section 2.4), afin d'assurer la traçabilité entre les exigences fonctionnelles, les récits utilisateurs, les éléments d'implémentation et les Sprints correspondants.

2.2.2.1 Besoins fonctionnels de l'Administrateur SOC

Les besoins suivants décrivent les principales fonctionnalités mises à disposition de l'Administrateur SOC pour superviser le système et intervenir dans le traitement des incidents.

TABLE 2.2 – Besoins fonctionnels de l'Administrateur SOC

Réf.	**Besoin**
BF-A1	S'authentifier auprès du tableau de bord (jeton JWT) avant tout accès aux données.
BF-A2	Suivre en temps réel les alertes et incidents entrants, avec leur sévérité, l'adresse source et la technique MITRE ATT&CK associée, et être averti par un signal sonore au-delà d'un seuil de gravité.
BF-A3	Consulter le rapport d'incident structuré produit par l'agent d'analyse, puis l'exporter au format PDF.
BF-A4	Piloter la réponse : confirmer, bloquer ou débloquer manuellement une adresse depuis l'interface (événement `send_command`).
BF-A5	Superviser l'état des machines virtuelles et des services (processeur, mémoire, disque, disponibilité).
BF-A6	Consulter les renseignements CTI collectés et les règles de détection générées.
BF-A7	Mener une investigation rétrospective sur les journaux bruts via Kibana.
BF-A8	Régler ses préférences : seuils d'alerte, volume sonore, notifications.

2.2.2.2 Besoins fonctionnels de l'Employé

Les besoins associés à l'Employé concernent principalement les actions de sensibilisation et d'évaluation déclenchées à la suite d'un incident de sécurité.

TABLE 2.3 – Besoins fonctionnels de l'Employé

Réf.	**Besoin**
BF-E1	Recevoir par courriel un parcours de sensibilisation adapté à l'incident qui vient d'être détecté.
BF-E2	Consulter le contenu pédagogique associé : cours, schémas et questionnaire à choix multiples (QCM).
BF-E3	Passer l'évaluation et obtenir une attestation lorsque le score atteint au moins 70.

2.2.2.3 Besoins fonctionnels de l'Agent autonome

Ces besoins décrivent les fonctions exécutées automatiquement par les différents composants internes de NG-IDPS, sans constituer des acteurs UML externes.

TABLE 2.4 – Besoins fonctionnels de l'Agent autonome

Réf.	**Besoin**
BF-S1	Capturer et inspecter le trafic réseau en continu (détection par signatures).
BF-S2	Extraire 16 caractéristiques statistiques du trafic et les publier vers Kafka, en parallèle de l'envoi des journaux bruts vers la pile d'analyse rétrospective.
BF-S3	Détecter les comportements anormaux par reconstruction séquentielle (LSTM-VAE), en filtrant le bruit par persistance, liste blanche et temps de repos.
BF-S4	Se protéger des rafales volumétriques : bouclier anti-DDoS global et limitation de débit par adresse source.
BF-S5	Dédupliquer et corréler les alertes, puis calculer un score de menace composite $S = 0{,}4\,S_{\text{Suricata}} + 0{,}3\,S_{\text{LSTM-VAE}} + 0{,}3\,S_{\text{LLM}}$ pour décider de la réponse.
BF-S6	Bloquer une adresse hostile par règle `iptables` exécutée à distance via SSH, sans délai pour les alertes les plus graves, et lever le blocage à l'expiration.
BF-S7	Notifier les analystes connectés ; si aucun ne l'est, envoyer un courriel puis basculer en mode Auto-Pilote.
BF-S8	Produire un rapport d'incident à sept champs par un LLM local, avec un rapport de repli si le modèle est indisponible.
BF-S9	Enrichir la détection : veille OSINT, génération de règles Suricata, validation par schéma puis essai à blanc avant déploiement.
BF-S10	Vérifier périodiquement (toutes les 15 secondes) la santé des machines virtuelles et persister les incidents en base.

2.2.3 Besoins non fonctionnels

Les besoins non fonctionnels (BNF) décrivent les propriétés de qualité et les contraintes auxquelles NG-IDPS doit satisfaire au-delà des fonctionnalités attendues. Leur formulation s'appuie sur les caractéristiques de qualité définies par la norme ISO/IEC 25010 [4], complétées, lorsque nécessaire, par les contraintes spécifiques de l'environnement expérimental.

Afin de garantir leur vérifiabilité, chaque besoin est associé à un critère d'évaluation mesurable ou à une procédure de vérification permettant de déterminer objectivement son respect.

TABLE 2.5 – Besoins non fonctionnels de NG-IDPS

Réf.	**Caractéristique**	**Exigence**	**Critère de vérification**
BNF-1	Efficacité de performance	Le traitement d'une séquence par le modèle LSTM-VAE doit présenter une latence suffisamment faible pour ne pas constituer un goulot d'étranglement du pipeline de détection.	Latence d'inférence inférieure à 15 ms par séquence, mesurée sur l'environnement matériel expérimental.
BNF-2	Fiabilité	Le système doit continuer à assurer les fonctions essentielles de détection et de supervision lorsqu'un composant non critique devient indisponible.	Vérification par scénarios de défaillance contrôlée et observation du mécanisme de repli, de la surveillance et du redémarrage des composants concernés.
BNF-3	Sécurité	Les accès aux interfaces et aux ressources sensibles doivent être contrôlés et les informations d'authentification ou secrets de configuration ne doivent pas être exposés en clair.	Vérification de l'authentification JWT, de l'utilisation de clés SSH, de l'absence de secrets dans le code source et de la configuration du pare-feu `ufw`.
BNF-4	Compatibilité	Les composants du système doivent pouvoir échanger des données selon des interfaces et des formats définis et stables.	Vérification des événements Socket.io `soar_incident_-incoming`, `soar_incident_updated` et `send_command`, ainsi que du format JSON défini pour les échanges inter-composants.

Réf.	Caractéristique	Exigence	Critère de vérification
BNF-5	Maintenabilité	Les composants doivent être suffisamment découplés pour permettre leur redémarrage individuel sans provoquer l'arrêt des autres services non concernés.	Test de redémarrage individuel des services et vérification de la continuité des échanges inter-composants via les mécanismes de communication prévus.
BNF-6	Utilisabilité	L'interface de supervision doit permettre à un analyste d'identifier rapidement les incidents nécessitant une attention particulière.	Évaluation de la visibilité des alertes critiques au moyen des indicateurs visuels et sonores disponibles dans l'interface de supervision.
BNF-7	Confidentialité	Les données d'alerte destinées à l'analyse cognitive doivent pouvoir être traitées sans transmission obligatoire vers un service LLM externe.	Vérification que l'inférence du modèle de langage est exécutée localement via Ollama et qu'aucun appel à une API LLM externe n'est requis pour le fonctionnement nominal.

2.3 Diagramme des cas d'utilisation globale

La figure 2.1 synthétise les besoins précédents. Nous avons choisi de représenter l'agent autonome comme un acteur à part entière, bien qu'il fasse partie du système : c'est une convention courante pour les systèmes qui agissent seuls, et elle rend visible ce que NG-IDPS fait en l'absence de tout opérateur.

Deux remarques facilitent la lecture. D'abord, les cas UC2 à UC7 supposent une session ouverte : UC1 leur est implicitement inclus, ce que nous n'avons pas dessiné pour ne pas alourdir le schéma. Ensuite, l'attaquant n'est relié qu'à UC10 mais son influence traverse tout le côté droit du diagramme : c'est sa présence qui enclenche, de proche en proche, le blocage, l'analyse et, le cas échéant, la formation des employés concernés.

2.4 Backlog du produit

Le backlog est exprimé sous forme de récits utilisateur (*user stories*), selon le gabarit « En tant que … je veux … afin de … » popularisé par Cohn [5]. La priorité suit la méthode MoSCoW : *Must* pour ce sans quoi le produit n'a pas de sens, *Should* pour ce qui apporte une

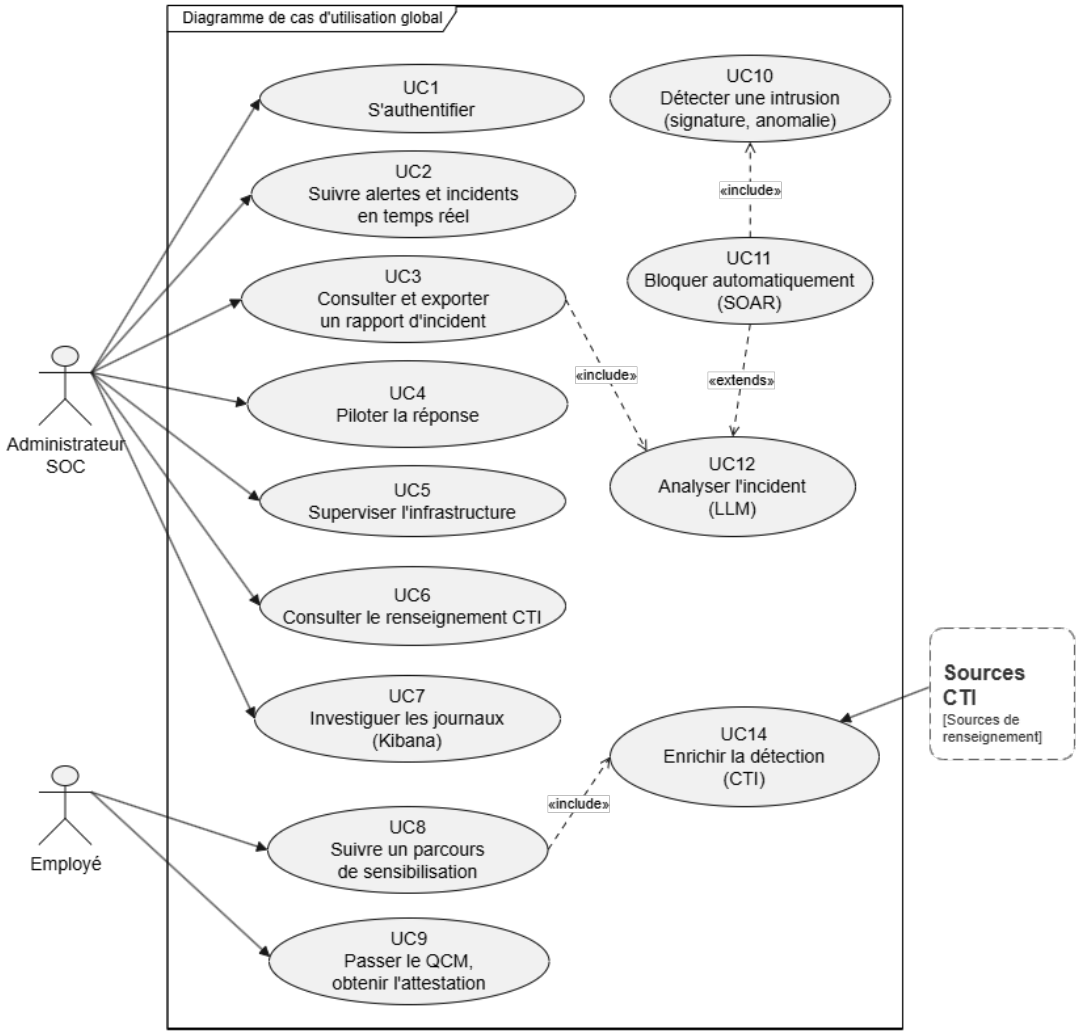


FIGURE 2.1 – Diagramme de cas d'utilisation global de NG-IDPS

forte valeur mais pourrait être différé, *Could* pour le confort. L'ordre de la liste est celui de la valeur décroissante au sein de chaque sprint.

TABLE 2.6 – Backlog du produit

Réf.	**Récit utilisateur**	**Priorité**	**Sprint**	**Besoin**
US-01	En tant qu'administrateur, je veux que le trafic soit inspecté par une sonde de signatures afin de détecter les attaques connues.	Must	S1	BF-S1
US-02	En tant qu'agent autonome, je veux extraire 16 caractéristiques du trafic et les publier sur Kafka afin d'alimenter le moteur d'inférence sans délai.	Must	S1	BF-S2

Réf.	Récit utilisateur	Priorité	Sprint	Besoin
US-03	En tant qu'administrateur, je veux conserver les journaux bruts dans une pile d'analyse afin d'enquêter a posteriori.	Should	S1	BF-A7
US-04	En tant qu'agent autonome, je veux détecter les comportements anormaux en moins de 15 ms afin de repérer les attaques sans signature.	Must	S2	BF-S3
US-05	En tant qu'administrateur, je veux que le bruit soit filtré (persistance, liste blanche, temps de repos) afin de ne traiter que les alertes crédibles.	Must	S2	BF-S3
US-06	En tant qu'agent autonome, je veux me protéger des rafales volumétriques afin de préserver la capacité d'inférence.	Must	S2	BF-S4
US-07	En tant qu'administrateur, je veux que de nouvelles règles soient générées à partir du renseignement OSINT afin d'anticiper les menaces émergentes.	Should	S3	BF-S9
US-08	En tant qu'administrateur, je veux que toute règle générée soit validée (schéma, essai à blanc) afin de ne pas dégrader la détection existante.	Must	S3	BF-S9
US-09	En tant qu'employé, je veux recevoir un parcours de sensibilisation adapté à l'incident afin de réduire le risque humain.	Could	S4	BF-E1, E2
US-10	En tant qu'employé, je veux valider mes acquis et obtenir une attestation afin de prouver ma mise à niveau.	Could	S4	BF-E3
US-11	En tant qu'administrateur, je veux m'authentifier et suivre en temps réel alertes et état des composants afin de piloter la sécurité depuis un point unique.	Must	S5	BF-A1, A2, A5
US-12	En tant qu'administrateur, je veux un rapport d'incident structuré et exportable afin de décider vite et de conserver une trace.	Must	S5	BF-A3, S8

Réf.	Récit utilisateur	Priorité	Sprint	Besoin
US-13	En tant qu'agent autonome, je veux calculer un score de menace composite afin de prioriser la réponse.	Should	S5	BF-S5
US-14	En tant qu'agent autonome, je veux bloquer une adresse hostile via SSH, y compris en l'absence d'analyste, afin de garantir la défense 24 h/24.	Must	S5	BF-S6, S7
US-15	En tant qu'administrateur, je veux commander manuellement un blocage ou un déblocage afin de garder la main sur l'automatisme.	Should	S5	BF-A4
US-16	En tant qu'administrateur, je veux être alerté si une machine virtuelle tombe afin d'agir avant que le pipeline ne s'interrompe.	Should	S5	BF-S10

2.5 Planification des sprints

Le projet suit la méthode Scrum [6], découpée en deux releases. Scrum a été adapté à une équipe d'une seule personne : les cérémonies collectives (mêlée quotidienne, par exemple) n'ont pas de sens ici, mais les artefacts essentiels ont été conservés : un backlog priorisé, des sprints à objectif unique et un incrément fonctionnel à l'issue de chacun.

Le séquencement suit le cheminement de la donnée. La Release 1 construit ce qui *voit* (ingestion et détection) ; la Release 2 construit ce qui *agit* et ce qui *s'explique* (anticipation, formation, supervision et réponse). Le tableau 2.7 résume ce découpage.

TABLE 2.7 – Planification des releases et des sprints

Release	Sprint	Objectif	Récits	Livrable majeur
Release 1	S1	Acquisition réseau et inspection sémantique (sonde Suricata, capture Zero-Copy)	1.1 à 1.3	Sonde opérationnelle, journalisation EVE JSON unifiée
	S2	Pipeline de données à double voie (rapide vers Kafka, froide vers ELK)	2.1 à 2.4	Bus Kafka, pipeline ETL Logstash, tableau de bord Kibana

Release	Sprint	Objectif	Récits	Livrable majeur
	S3	Détection comportementale par moteur d'inférence LSTM-VAE et filtrage du bruit	3.1 à 3.7	Pipeline temps réel sur VM-ML publiant les alertes d'anomalie
Release 2	S4	Anticipation par le renseignement (agent CTI et triage Dual-RAG)	US4.1 à US4.6	Générateur de règles Suricata validées
	S5	Mitigation du risque humain (agent Gemini et RAG LMS)	US5.1 à US5.5	Parcours de sensibilisation ciblés post-incident
	S6	Orchestration SOAR multi-paliers, agent post-attaque et dashboard SOC	US6.1 à US6.9	Application de supervision et boucle de réponse fermée

2.6 Environnement logiciel

La réalisation de NG-IDPS mobilise une chaîne technologique hétérogène : capture réseau bas niveau, courtage de messages distribué, apprentissage profond, inférence de modèles de langage et développement web temps réel. Aucun choix technologique n'est anodin dans ce contexte contraint — une sonde mal dimensionnée perd des paquets, un courtier de messages mal choisi sature sous charge, un modèle de langage mal sélectionné compromet soit la confidentialité, soit la continuité de service. Cette section justifie, catégorie par catégorie, les technologies retenues pour construire l'environnement logiciel du projet.

2.6.1 Démarche de comparaison

Choisir une technologie parce qu'elle est à la mode ne se défend pas devant un jury ; la choisir parce qu'elle l'emporte sur ses concurrentes selon des critères explicites, si. Nous avons donc procédé en deux temps. Pour chaque catégorie, nous identifions les candidats sérieux et les critères qui comptent réellement dans notre contexte : un hôte unique de 24 Go, un budget nul, une latence d'inférence inférieure à 15 ms et des données d'analyse qui ne quittent pas le périmètre.

Pour les trois décisions les plus structurantes (moteur de détection, bus de messages, modèle d'anomalie), nous formalisons le choix par une matrice de décision pondérée [7]. Chaque candidat

c reçoit une note $s_i(c) \in \{1, \ldots, 5\}$ sur chacun des n critères, et son score global vaut

$$S(c) = \sum_{i=1}^{n} w_i \, s_i(c), \qquad \sum_{i=1}^{n} w_i = 1, \tag{2.1}$$

où w_i traduit l'importance du critère i pour NG-IDPS. Lorsque des évaluations publiées existent, elles fondent les notes ; sinon, celles-ci reflètent le jugement de l'auteur après lecture de la documentation officielle. Il faut le dire franchement : cette méthode ne produit pas une vérité mathématique, seulement une décision traçable que chacun peut contester en changeant un poids ou une note. Pour les autres catégories, une comparaison qualitative selon quelques critères discriminants suffit. Les mesures expérimentales (rappel, précision, latence) sont volontairement renvoyées au chapitre 3.

2.6.2 Infrastructure de virtualisation et système d'exploitation

Le projet repose sur plusieurs machines virtuelles reliées par un réseau privé (`192.168.56.0/24`) et sur un hôte Windows dont le GPU sert directement à l'inférence du modèle CTI. La notion de moniteur de machine virtuelle remonte à Popek et Goldberg [8] ; ce qui nous intéresse ici est plus prosaïque : que l'outil tourne sous Windows, gère finement les réseaux virtuels et ne coûte rien. Le tableau 2.8 confronte les trois options réalistes.

TABLE 2.8 – Comparaison des solutions de virtualisation

Critère	**VMware Workstation Pro** [9]	**VirtualBox** [10]	**KVM / Proxmox VE** [11, 12]
Hôte supporté	Windows, Linux	Windows, Linux, macOS	Linux uniquement (Proxmox : machine dédiée)
Réseaux virtuels	Host-only, NAT, pont, éditeur dédié	Host-only, NAT, pont	Ponts Linux, très flexibles
Instantanés	Oui	Oui	Oui (qcow2)
Coût	Gratuit pour l'usage personnel depuis 2024	Gratuit	Gratuit
Compatibilité avec un hôte Windows partagé avec un GPU	Oui	Oui	Non sans dédier la machine
Décision	**Retenu**	Alternative valable	Écarté

KVM est techniquement excellent, mais il suppose un hôte Linux, alors que notre machine de

travail doit rester une station Windows utilisable au quotidien. Proxmox exigerait de la sacrifier entièrement. VirtualBox aurait techniquement fait l'affaire, la préférence pour VMware Workstation relève surtout du confort d'usage (éditeur de réseaux virtuels, gestion des instantanés) et non d'un avantage décisif.

Côté système invité, nous avons retenu Ubuntu Server en version à support long terme (LTS), qui bénéficie de cinq ans de correctifs de sécurité en standard [13]. Debian stable aurait offert une base plus conservatrice mais des paquets plus anciens ; Rocky Linux, compatible RHEL, se justifie surtout en contexte d'entreprise. Ubuntu présente l'avantage décisif de bénéficier d'un dépôt officiel pour Suricata et d'une documentation abondante pour tout le reste de la pile.



FIGURE 2.2 – Logo VMware Workstation Pro

2.6.3 Détection d'intrusions réseau

Trois systèmes ouverts dominent le domaine : Snort, publié par Roesch dès 1999 [14], Bro, rebaptisé Zeek, présenté la même année par Paxson [15], et Suricata, développé par l'*Open Information Security Foundation* (OISF) [16]. Les deux premiers reposent sur des philosophies différentes : Snort applique des signatures, Zeek analyse le comportement des protocoles et produit des journaux riches, tandis que Suricata reprend l'approche à signatures avec un moteur multi-fil et des analyseurs de protocoles natifs.

Les études comparatives sont nuancées. Albin et Rowe, sur le trafic réel d'un réseau fédérateur, concluent que Suricata absorbe des volumes supérieurs à Snort à précision similaire et que ses performances progressent presque linéairement jusqu'à 48 processeurs [17]. Shah et Issac, à 10 Gbit/s, observent moins de paquets perdus avec Suricata mais une consommation de ressources plus élevée, alors que Snort affichait la meilleure exactitude de détection au prix de nombreux faux positifs [18]. Notons que ces travaux portent sur Snort 2.x, monoprocessus ; Snort 3 introduit le multi-fil [19], mais nous n'avons pas trouvé d'évaluation publiée comparable, ce qui nous a incités à la prudence dans la notation.

Le tableau 2.9 applique la matrice pondérée (équation 2.1). Les poids privilégient le débit et l'intégration au pipeline, car la sonde alimente directement Kafka et la pile d'analyse.

TABLE 2.9 – Matrice de décision pour le moteur de détection réseau

Critère	**Poids**	**Suricata**	**Snort 3**	**Zeek** [20]
Débit et parallélisme	0,25	5	3	4
Frugalité en ressources	0,15	3	4	3
Intégration au pipeline (JSON natif, modules)	0,25	5	3	4
Couverture de détection (signatures et protocoles)	0,20	5	4	3
Maturité et communauté	0,15	4	5	3
Score pondéré	1,00	**4,55**	3,65	3,50

Suricata l'emporte principalement grâce à son format de sortie JSON natif (EVE), exploitable directement par Filebeat, et à des paramètres de temporisation des flux ajustables ; leur réglage a permis, comme nous le verrons au chapitre 3, de ramener le délai de détection de bout en bout de plusieurs dizaines de secondes à quelques secondes. Snort reste le plus mature, mais cette maturité pèse moins que l'intégration dans notre pipeline. Zeek, remarquable pour la visibilité, n'est pas conçu comme un moteur de règles.

Un dernier choix mérite d'être expliqué : Suricata fonctionne ici en mode IDS passif, et le blocage est délégué à la couche SOAR. Un mode IPS en ligne aurait rendu la sonde responsable de la disponibilité du réseau ; en cas de défaillance, c'est le trafic légitime qui s'arrêterait. Nous préférons qu'une panne de la détection dégrade la protection plutôt que la connectivité.



FIGURE 2.3 – Logo Suricata

2.6.4 Collecte, indexation et visualisation des journaux

Les journaux bruts empruntent une voie dite « froide », hors du chemin critique de détection : ils servent à l'investigation rétrospective. Quatre plateformes sont candidates (tableau 2.10). La pile Elastic est la référence du domaine [21].

TABLE 2.10 – Comparaison des plateformes de gestion des journaux

Critère	**Elastic Stack 8.x** [22]	**OpenSearch** [23]	**Graylog** [24]	**Splunk**
Licence et coût	Gratuit pour cet usage	Apache 2.0	Édition ouverte gratuite	Commerciale, selon le volume
Ingestion des journaux Suricata	Module Filebeat natif pour EVE JSON	Via Logstash ou Data Prepper	Via Beats ou GELF	Extension dédiée
Visualisation	Kibana	OpenSearch Dashboards	Interface intégrée	Splunk Web
Empreinte mémoire	Élevée (JVM)	Élevée (JVM)	Élevée (moteur + MongoDB)	Élevée
Décision	**Retenu**	Alternative	Écarté	Écarté (coût)

L'argument décisif est l'ingestion : Filebeat sait lire les journaux EVE de Suricata avec un module prêt à l'emploi, ce qui évite d'écrire un analyseur sur mesure. OpenSearch, dérivé d'une ancienne version d'Elasticsearch, serait une option sérieuse, mais son ingestion exige un maillon supplémentaire. Nous acceptons en revanche un inconvénient : l'empreinte mémoire de la JVM est lourde. C'est en partie ce qui justifie que la voie froide reste facultative pour le fonctionnement autonome du système, comme le prévoit la Release 2, qui ne dépend plus d'ELK.



FIGURE 2.4 – Logo Elastic Stack

2.6.5 Transport de données en flux

Le lien entre la sonde, le moteur d'inférence, la couche SOAR et le tableau de bord doit être asynchrone, tolérant aux pannes et capable de *rejouer* un même flux vers plusieurs consommateurs (inférence, agents d'analyse, historique). C'est le profil d'un journal distribué, modèle sur lequel repose Kafka, conçu à l'origine pour le traitement de journaux [25]. Le tableau 2.11 compare cinq options, dont un service géré du nuage (SQS [26]) souvent cité comme point de comparaison.

TABLE 2.11 – Matrice de décision pour le bus de messages

Critère	**Poids**	**Kafka** [27]	**Pulsar** [28]	**RabbitMQ** [29]	**Redis Streams** [30]	**SQS**
Débit et latence	0,25	5	4	3	4	3
Frugalité en ressources	0,15	2	2	4	3	5
Persistance et rejeu	0,25	5	5	3	3	2
Intégration (clients, connecteurs)	0,20	5	3	4	3	3
Déploiement local	0,15	5	5	5	5	1
Score pondéré	1,00	**4,55**	3,90	3,65	3,55	2,75

Le travail de Dobbelaere et Esmaili donne le contexte le plus utile : Kafka atteint les débits les plus élevés grâce à son modèle de journal partitionné, tandis que RabbitMQ conserve un avantage de latence pour les volumes modestes [31]. Or notre besoin premier n'est pas la latence d'un message isolé mais la possibilité, pour plusieurs consommateurs indépendants, de lire le même topic à leur rythme, d'où la note maximale de Kafka en persistance et rejeu. RabbitMQ retire un message une fois acquitté, ce qui convient aux files de tâches mais pas à ce scénario, même si ses flux (*streams*) réduisent l'écart. Pulsar, sérieux challenger, sépare serveurs et stockage : cette architecture apporte de la souplesse à grande échelle mais exige davantage de composants, ce que notre hôte ne peut se permettre. SQS, enfin, est éliminé par l'exigence de déploiement local. Le contrepoint honnête est le coût de Kafka en mémoire ; l'abandon de ZooKeeper au profit du mode KRaft, que nous utilisons, en atténue la charge.



FIGURE 2.5 – Logo Kafka

2.6.6 Détection d'anomalies par apprentissage profond

Choix du modèle. La détection par signatures ne voit que ce qu'elle connaît. Compléter Suricata par un détecteur d'anomalies, qui apprend ce qu'est un trafic normal pour repérer ce qui s'en écarte, répond à ce point aveugle [32, 33]. Cinq familles sont candidates : la forêt d'isolation [34], le SVM à une classe [35], le *Deep SVDD* [36], l'autoencodeur dense et l'autoencodeur variationnel récurrent (LSTM-VAE). Ce dernier associe un réseau à mémoire longue [37], qui capte les dépendances temporelles, à un espace latent probabiliste régularisé [38]. Park et al. l'ont introduit pour détecter des anomalies dans des séries multimodales [39] ; An et Cho ont montré l'intérêt d'un score fondé sur la probabilité de reconstruction plutôt que sur la seule erreur [40].

Le critère le plus lourd est la modélisation temporelle : une attaque réseau se manifeste rarement par un paquet isolé mais par une *séquence*, comme dans un balayage lent de ports ou une attaque par force brute. Les méthodes qui évaluent chaque vecteur indépendamment ne peuvent pas la voir.

TABLE 2.12 – Matrice de décision pour le modèle de détection d'anomalies

Critère	**Poids**	**Forêt d'isol.**	**SVM 1 classe**	**Deep SVDD**	**AE dense**	**LSTM-VAE**
Modélisation temporelle	0,30	2	2	2	2	5
Non-linéarité, haute dimension	0,20	3	4	5	4	5
Latence d'inférence	0,20	5	3	4	5	4
Score exploitable et calibrable	0,15	3	3	2	3	4
Maturité de l'outillage	0,15	5	5	3	5	4
Score pondéré	1,00	3,40	3,20	3,15	3,60	**4,50**

Le LSTM-VAE domine surtout grâce au premier critère. La forêt d'isolation est imbattable en rapidité et en simplicité, mais elle ignore l'ordre des événements. Le SVM à une classe se heurte au coût d'entraînement et d'inférence quand les données grossissent. Le *Deep SVDD* apprend une hypersphère autour des données normales sans distribution explicite, et l'on connaît le risque d'effondrement de cette hypersphère [36]. L'autoencodeur dense est un solide second, mais son espace latent déterministe se prête moins à un seuil bien calibré. Ce choix n'est pas gratuit : le LSTM-VAE est plus difficile à expliquer qu'une forêt d'isolation. Le vote par caractéristique que nous décrirons au chapitre 3 compense en partie ce défaut en indiquant *quelles* variables ont déclenché l'alerte.



FIGURE 2.6 – Modèle Deep LSTM-VAE

Cadre logiciel. Deux bibliothèques se partagent l'apprentissage profond : TensorFlow, avec son API de haut niveau Keras [41], et PyTorch [42]. PyTorch, avec son exécution impérative, domine la recherche ; TensorFlow reste avantageux pour la mise en production, avec des graphes compilables et des formats d'export adaptés au déploiement. Notre modèle est entraîné puis chargé sous forme d'un fichier `.keras`, et la compilation du chemin d'inférence en graphe (`tf.function`) réduit la latence d'un facteur supérieur à dix par rapport à l'appel naïf, comme nous le mesurerons au chapitre 3. Les étapes classiques de prétraitement et de sélection de caractéristiques s'appuient sur scikit-learn [43].

Jeu de données. Le tableau 2.13 compare trois jeux publics courants.

TABLE 2.13 – Comparaison des jeux de données de détection d'intrusions

Critère	**NSL-KDD** [44]	**UNSW-NB15** [45]	**CIC-IDS2017** [46]
Année	2009 (dérivé de KDD'99)	2015	2017
Nature du trafic	Simulé, très ancien	Synthétique (générateur IXIA), 49 caractéristiques	Capture de cinq jours, plus de 80 caractéristiques de flux
Attaques	Familles obsolètes	9 familles	Force brute, DoS/DDoS, web, botnet, infiltration, balayage de ports
Exploitable en séquences (horodatage)	Non	Oui	Oui
Décision	Écarté (obsolète)	Alternative	**Retenu**

Deux atouts ont pesé : l'existence d'une journée de trafic entièrement bénin, indispensable pour entraîner un modèle non supervisé sur le seul comportement normal, et la diversité des attaques modernes. Le jeu fournit à la fois les captures brutes et des fichiers de flux [47], ce qui laisse le choix de la chaîne d'extraction des caractéristiques. Il n'est toutefois pas sans défaut : Engelen et al. ont montré que plus de 20 % des traces du jeu d'origine contenaient des erreurs de construction des flux ou d'étiquetage [48], constat prolongé par Lanvin et al. [49]. Nous en tenons compte en complétant l'évaluation par des attaques rejouées depuis notre propre machine Kali, afin de ne pas dépendre d'un seul corpus.

2.6.7 IA générative, RAG et renseignement sur les menaces

Modèle de langage. NG-IDPS emploie un LLM pour deux tâches : transformer une alerte brute en rapport d'incident lisible, et raisonner sur les renseignements de menace. Trois options se présentent (tableau 2.14).

TABLE 2.14 – Comparaison des modèles de langage candidats

Critère	**GPT-4** [50]	**Llama 3 / 3.2** [51]	**Mistral 7B** [52]
Déploiement	API dans le nuage	Local	Local
Données	Envoyées au fournisseur	Restent dans le périmètre	Restent dans le périmètre

Critère	**GPT-4** [50]	**Llama 3 / 3.2** [51]	**Mistral 7B** [52]
Coût	À l'usage	Nul (matériel existant)	Nul (matériel existant)
Licence	Propriétaire	Licence communautaire Meta (ouverte, non OSI)	Apache 2.0
Adéquation à une VM sans GPU	Sans objet	Version 3B légère, exécutable sur CPU	Plus lourd sur CPU
Décision	Écarté	**Retenu**	Alternative

Le raisonnement tient en trois points. Les rapports d'incident contiennent des adresses, des charges utiles et des éléments de topologie : les envoyer à un tiers contredit le besoin BNF-9. Le coût à l'usage d'une API, ensuite, s'accorde mal avec un système qui tourne en continu. Enfin, la famille Llama offre un modèle de 3 milliards de paramètres qui reste utilisable sur CPU dans une machine virtuelle sans GPU (Llama 3.2 3B, pour le rapport d'incident), et un modèle plus grand qui profite du GPU de l'hôte (Llama 3, pour le CTI). Mistral 7B, sous licence Apache 2.0, est une alternative défendable, mais son empreinte plus forte ralentit l'inférence sur CPU. Précisons que le module de formation du Sprint 4 fait exception : il s'appuie sur l'API Gemini pour générer les contenus pédagogiques.

Moteur d'inférence. Le modèle doit être servi par un runtime. **Ollama** [53] expose une interface REST locale, gère le téléchargement et la quantification des modèles et permet d'imposer une sortie JSON, ce dont dépend le rapport à sept champs. **vLLM**, fondé sur la gestion paginée de la mémoire d'attention [54], excelle en débit pour de nombreux utilisateurs concurrents mais suppose un GPU. **llama.cpp** [55] est un moteur C/C++ performant sur CPU, sur lequel Ollama s'est historiquement appuyé, mais sans la couche de service. Ollama réunit donc ce dont nous avons besoin : API REST, sortie contrainte, exécution sur CPU comme sur GPU.

Génération augmentée par récupération. Un LLM seul peut inventer des règles ou des identifiants de menace. La génération augmentée par récupération (RAG) [56] l'ancre sur des documents récupérés au préalable. Pour l'index vectoriel, FAISS [57] est une bibliothèque exécutée dans le processus de l'agent, sans serveur à administrer ; Milvus [58] vise les déploiements distribués à très grande échelle, et Chroma [59] privilégie la simplicité de prise en main pour les applications LLM. Avec quelques dizaines de milliers de règles Suricata à indexer, FAISS suffit largement et n'ajoute aucun service à l'hôte. Les textes sont vectorisés avec `nomic-embed-text` [60], disponible localement via Ollama. Enfin, la sortie du LLM lors de la génération de règles est validée par des schémas Pydantic [61] : ce garde-fou empêche une réponse malformée d'atteindre les fichiers de règles.



FIGURE 2.7 – Logo Llama 3.2

2.6.8 Prévention et réponse automatisée

Pare-feu. Le blocage repose sur le sous-système Netfilter du noyau Linux [62], accessible par `iptables` ou son successeur `nftables`. Ce dernier offre des ensembles pour des recherches plus efficaces quand la liste de blocage grossit ; sur les distributions récentes, la commande `iptables` n'est d'ailleurs souvent qu'une couche de compatibilité au-dessus de lui. Nous avons gardé `iptables` pour sa simplicité et sa présence universelle, en veillant à l'idempotence des règles (pas de doublons). `ufw`, une simple surcouche, sert au durcissement statique des machines mais pas au blocage dynamique par adresse. `fail2ban` réagit à l'analyse de fichiers de journaux : il ne sait pas consommer un score produit par un modèle. La migration vers les ensembles `nftables` est une perspective d'évolution naturelle.

Exécution distante. Le blocage doit s'exécuter sur la machine qui voit le trafic, tandis que la décision se prend ailleurs. Nous utilisons SSH [63] avec authentification par clés, piloté depuis Python par Paramiko [64]. Ansible [65], sans agent et idempotent, serait le choix naturel pour de la configuration, mais le démarrage d'un *playbook* se compte en secondes : incompatible avec un blocage quasi instantané.

Orchestration. Le tableau 2.15 compare notre orchestrateur développé sur mesure à des plateformes existantes.

TABLE 2.15 – Comparaison des approches d'orchestration SOAR

Critère	**Moteur Python sur mesure**	**Shuffle** [66]	**TheHive + Cortex** [67]	**Splunk SOAR**
Intégration à Kafka	Native (consommateur)	Par webhook ou connecteur	Par connecteur	Par connecteur
Empreinte	Minimale	Conteneurs, plus lourde	JVM + base, lourde	Lourde
Paliers déterministes maîtrisés	Oui, en code	Oui, via flux de travail	Orientée gestion de cas	Oui

Critère	**Moteur Python sur mesure**	**Shuffle** [66]	**TheHive + Cortex** [67]	**Splunk SOAR**
Coût	Nul	Gratuit (édition libre)	Édition gratuite limitée, sinon commerciale	Commercial
Généralité et interface	Faibles	Élevées	Élevées	Élevées
Décision	**Retenu**	Alternative	Écarté	Écarté (coût)

Nous avons donc écrit notre propre moteur SOAR. Le compromis est assumé : nous perdons l'interface et la généralité d'une plateforme éprouvée, et nous héritons de leur maintenance ; en échange, l'escalade en trois paliers s'exprime en quelques dizaines de lignes lisibles, se branche directement sur Kafka et ne consomme presque rien. Pour un système dont la défense académique repose précisément sur cette logique de décision, la maîtriser de bout en bout nous a paru préférable.



FIGURE 2.8 – Logo Python

2.6.9 Serveur applicatif et communication temps réel

Le serveur applicatif consomme les événements Kafka, sert l'API, persiste les incidents et diffuse les alertes aux navigateurs. Le tableau 2.16 compare trois piles.

TABLE 2.16 – Comparaison des piles serveur

Critère	**Node.js + Express** [68]	**Django** [69]	**FastAPI** [70]
Modèle d'exécution	Événementiel, non bloquant	Synchrone, asynchrone partiel	Asynchrone (ASGI)
Temps réel bidirectionnel	Socket.io natif	Nécessite Channels	WebSocket, sans reconnexion
Client Kafka	KafkaJS	Bibliothèques Python	Bibliothèques Python

Critère	**Node.js + Express** [68]	**Django** [69]	**FastAPI** [70]
Cohérence avec l'interface	Même langage (JS/TS)	Langage différent	Langage différent
Décision	**Retenu**	Écarté	Alternative

Tilkov et Vinoski ont montré que le modèle événementiel de Node.js convient aux applications à nombreuses connexions simultanées, ce qui décrit un tableau de bord SOC à plusieurs analystes [68]. Django serait plus à l'aise pour un site classique que pour un flux d'événements permanent. FastAPI est un candidat sérieux, mais partager un langage avec l'interface (TypeScript) réduit la charge cognitive d'un projet mené seul.

Pour l'échange temps réel lui-même, trois mécanismes existent. Les évènements envoyés par le serveur (SSE) sont unidirectionnels : ils ne peuvent pas transporter la commande `send_command` de l'administrateur vers le serveur. Le protocole WebSocket standard [71] autorise l'échange dans les deux sens, mais laisse à l'application la gestion des reconnexions et des salons. Socket.io [72] ajoute ces mécanismes, plus un repli automatique en cas de proxy hostile. Il est retenu, et ses trois évènements (`soar_incident_incoming`, `soar_incident_updated`, `send_command`) forment un contrat stable entre serveur et interface.

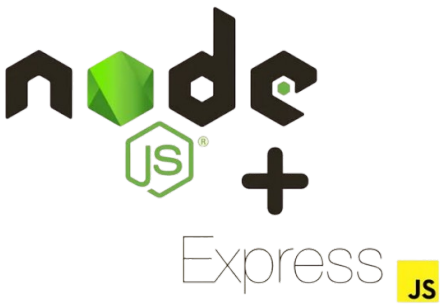


FIGURE 2.9 – Logo NodeJS - Express

2.6.10 Interface de supervision

L'interface doit rester fluide malgré l'arrivée continue d'alertes, et son apparence compte : un analyste qui surveille un écran de nuit a besoin de contraste et de hiérarchie visuelle.

TABLE 2.17 – Comparaison des cadres d'interface

Critère	**Next.js (React)** [73]	**Vue / Nuxt** [74]	**Angular** [75]
Rendu	Serveur (SSR) et client	Serveur et client	Client, serveur en option

Critère	**Next.js (React)** [73]	**Vue / Nuxt** [74]	**Angular** [75]
Routage	Par fichiers	Par fichiers (Nuxt)	Configuration explicite
Typage	TypeScript courant	TypeScript pris en charge	TypeScript imposé
Écosystème de composants	Très vaste	Vaste	Complet, plus rigide
Courbe d'apprentissage	Modérée	Douce	Raide
Décision	**Retenu**	Alternative	Écarté

Next.js s'impose par son routage par fichiers, son typage TypeScript et un écosystème de composants qui nous permet d'assembler un tableau de bord sans réinventer chaque élément. Angular impose un cadre plus rigide, disproportionné pour l'application visée. Pour le style, Tailwind CSS [76] convient à une identité visuelle sur mesure (thème sombre, effet de verre dépoli) là où Bootstrap ou une bibliothèque de composants imposeraient leur propre esthétique.

NEXT.JS

FIGURE 2.10 – Logo NextJS

2.6.11 Persistance des données

Deux besoins de stockage cohabitent. Les incidents sont des documents JSON semi-structurés, dont le contenu varie selon l'attaque et dont le schéma évolue au fil des sprints ; les indicateurs de compromission de l'agent CTI sont des enregistrements simples à recherche exacte. Cattell [77] montre que le choix entre stockages relationnels et NoSQL dépend de la forme des données et de leur évolution, plus que d'une supériorité absolue.

TABLE 2.18 – Comparaison des solutions de persistance

Critère	**MongoDB** [78]	**PostgreSQL** [79]	**SQLite** [80]
Modèle	Documents JSON	Relationnel (JSON possible)	Relationnel embarqué
Évolution du schéma	Souple	Migrations nécessaires	Migrations nécessaires
Déploiement	Service local	Service local	Fichier, sans serveur

Critère	**MongoDB** [78]	**PostgreSQL** [79]	**SQLite** [80]
Rôle dans NG-IDPS	Incidents, enregistrements de formation	Non retenu	Mémoire locale des indicateurs CTI

MongoDB est retenu pour les incidents : le rapport à sept champs est déjà un document, et le pilote Node.js le manipule sans transformation. PostgreSQL et son type JSONB auraient fonctionné ; nous ne prétendons pas qu'il y ait ici une supériorité technique franche. La souplesse du schéma et la continuité avec la pile Node.js ont fait pencher la balance, et le déploiement en local restreint la surface d'attaque. SQLite, lui, ne sert qu'à l'agent CTI : sans serveur ni administration, il répond aux recherches exactes d'indicateurs et complète l'index sémantique FAISS.



FIGURE 2.11 – Logo Mongo BD

2.6.12 Gestion de projet, modélisation et rédaction

Le tableau 2.19 regroupe les outils transverses, dont le choix a moins d'enjeu technique mais pèse sur la conduite du projet.

TABLE 2.19 – Outils de gestion de projet, de modélisation et de rédaction

Usage	**Retenu**	**Alternatives**	**Raison**
Suivi du backlog	GitHub Projects [81]	Jira, Trello	Lié aux dépôts et aux tickets, gratuit
Modélisation	PlantUML, Mermaid	Outils graphiques	Diagrammes sous forme de texte, versionnables
Rédaction	LaTeX (Overleaf) et BibTeX	Traitement de texte	Équations, références et numérotation automatisées



FIGURE 2.12 – Logo Github

2.6.13 Synthèse de la pile technologique

Le tableau 2.20 récapitule la pile retenue et rappelle sur quel nœud chaque brique est déployée.

TABLE 2.20 – Synthèse de la pile technologique de NG-IDPS

Couche	**Technologie retenue**	**Version**	**Nœud**
Virtualisation	VMware Workstation, Ubuntu Server LTS	—	Hôte
Détection par signatures	Suricata	8.0.6	VM-Edge
Journaux bruts	Filebeat, Logstash, Elasticsearch, Kibana	8.x	VM-Edge / VM-ML
Transport	Apache Kafka (mode KRaft)	4.3.1	VM-ML
Détection d'anomalies	LSTM-VAE (TensorFlow, Keras, scikit-learn)	TF 2.21.0, Keras 3.15.0	VM-ML
LLM et RAG	Ollama, Llama 3.2 3B, Llama 3, FAISS, nomic-embed-text, Pydantic	—	VM-Response / Hôte
Réponse	Moteur SOAR Python, SSH (Paramiko), iptables	Python 3.12	VM-ML / VM-Response
Serveur	Node.js, Express, Socket.io, KafkaJS, Nodemailer	—	Hôte

Suite à la page suivante...

Couche	Technologie retenue	Version	Nœud
Interface	Next.js, TypeScript, Tailwind CSS	—	Hôte
Persistance	MongoDB, SQLite	—	Hôte / VM-Response

2.7 Architecture générale

NG-IDPS est un système distribué, événementiel et faiblement couplé : ses composants ne s'appellent jamais directement, ils échangent des messages sur des topics Kafka. La figure 2.13 en donne la vue de déploiement, avec quatre machines virtuelles et l'hôte physique.

- **VM-Edge** (`192.168.56.128`) héberge la sonde Suricata, Filebeat et le collecteur `edge_collector.py` qui extrait les 16 caractéristiques. C'est aussi la cible du blocage.
- **VM-ML** (`192.168.56.130`) porte le broker Kafka, la pile ELK, l'inférence LSTM-VAE (`realtime_interface.py`) et le moteur de corrélation et de décision `soar.py`.
- **VM-Response** (`192.168.56.140`) exécute les agents : analyse post-attaque (`agent_ia.py`), renseignement (`agent_cti.py`) et exécution du blocage par SSH (`agent_soar.py`). Elle a été séparée de VM-ML parce que Ollama saturait la mémoire de cette dernière.
- **VM-Kali** (`192.168.56.150`) simule l'attaquant.
- **L'hôte Windows** fait tourner le serveur `server.js`, MongoDB, le tableau de bord Next.js et Ollama avec le modèle Llama 3 accéléré par GPU.

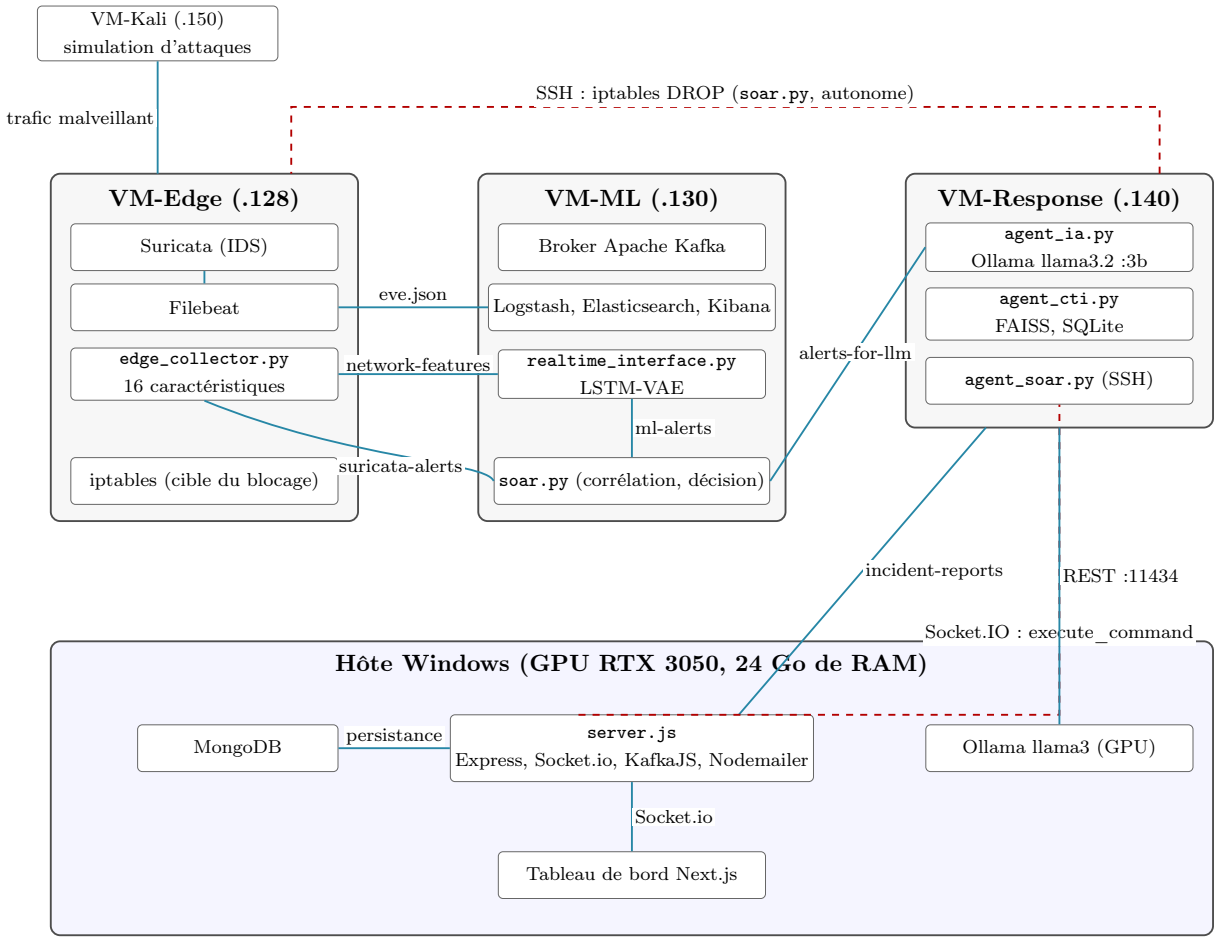


FIGURE 2.13 – Architecture de déploiement de NG-IDPS

Les échanges reposent sur cinq topics Kafka présenté dans le tableau 4.10.

TABLE 2.21 – Topics Kafka de NG-IDPS

Topic	Producteur	Consommateur	Contenu
network-features	edge_collector.py	realtime_-interface.py	16 caractéristiques
ml-alerts	realtime_-interface.py	soar.py	Anomalie ML confirmée
suricata-alerts	edge_collector.py (relai des alertes Suricata)	soar.py	Alerte par signature
alerts-for-llm	soar.py	agent_ia.py, server.js	Alerte corrélée à analyser / à afficher
incident-reports	agent_ia.py, soar.py (bypass Zero Trust)	server.js	Rapport structuré ou alerte de contournement

Le parcours d'un incident, de la sonde à l'écran, se déroule en sept étapes.

1. Suricata inspecte le trafic ; ses journaux partent vers la voie froide (ELK) pendant que `edge_collector.py` extrait les caractéristiques (publiées sur `network-features`) et relaie les alertes par signature sur `suricata-alerts`.
2. `realtime_interface.py` construit des fenêtres de 10 pas de temps, calcule l'erreur de reconstruction du LSTM-VAE, applique les filtres (persistance, liste blanche, bouclier anti-DDoS) et publie une anomalie confirmée sur `ml-alerts`.
3. `soar.py` consomme en parallèle `ml-alerts` et `suricata-alerts`, déduplique et corrèle les alertes sur une fenêtre de 30 s, puis décide en autonomie complète : en cas de corrélation forte ou de signature critique, il déclenche aussitôt le blocage en appliquant lui-même, par SSH, la règle `iptables` sur VM-Edge.
4. L'alerte est relayée sur `alerts-for-llm` ; `agent_ia.py` interroge Ollama, obtient le rapport à sept champs (ou un rapport de repli) et le publie sur `incident-reports`.
5. `server.js` persiste l'incident dans MongoDB et le diffuse au tableau de bord par Socket.io ; si aucun analyste n'est connecté, un courriel est envoyé.
6. Un administrateur connecté peut confirmer ou annuler manuellement une action via l'événement Socket.IO `execute_command`, relayé à `agent_soar.py` sur VM-Response, qui exécute la commande par SSH et renvoie un accusé `command_result`.
7. En l'absence prolongée d'analyste, le mode Auto-Pilote emprunte ce même canal Socket.IO vers `agent_soar.py` pour appliquer le blocage sans attendre une validation humaine.

L'escalade en trois paliers qui structure les étapes 3, 6 et 7 (blocage autonome immédiat par `soar.py`, confirmation manuelle, mode Auto-Pilote) sera détaillée au chapitre 4, avec ses seuils. Une distinction architecturale importante y est développée : la décision autonome (`soar.py`, déclenchée par Kafka) est séparée de l'exécution supervisée par un humain (`agent_soar.py`, déclenchée par Socket.IO), ces deux composants ne communiquant jamais directement entre eux.

2.8 Conclusion

Ce chapitre a posé les fondations du projet. L'analyse des besoins a identifié cinq acteurs, vingt et un besoins fonctionnels et neuf besoins non fonctionnels mesurables, traduits en seize récits utilisateur répartis sur six sprints. La comparaison des technologies a permis de justifier la pile retenue, en appliquant une matrice pondérée aux trois décisions les plus structurantes (Suricata, Kafka, LSTM-VAE) et une analyse qualitative aux autres ; elle a aussi mis en lumière des contreparties assumées, comme l'empreinte mémoire de Kafka et d'ELK ou l'obligation de

maintenir notre propre moteur SOAR. Enfin, l'architecture de déploiement et le parcours d'un incident donnent le cadre dans lequel s'inscrivent les deux chapitres suivants. Le chapitre 3 aborde la Release 1 : la mise en place de l'ingestion parallèle et du moteur de détection.

CHAPITRE 3

DÉTECTION - INGESTION PARALLÈLE ET MOTEUR STOCHASTIQUE (RELEASE 1)

Sommaire

3.1 Introduction 79

3.2 Déploiement et Optimisation du Moteur d'Inspection Réseau (Sprint 1) 79

3.2.1 Backlog du Sprint 1 80

3.2.2 Analyse Fonctionnelle 80

3.2.3 Conception 81

3.2.4 Réalisation 82

3.2.5 Validation 82

3.2.6 Conclusion du Sprint 1 83

3.3 Intégration du Pipeline de Streaming Double Chemin (Sprint 2) 84

3.3.1 Justification Stratégique et Choix Architecturaux 84

3.3.2 Backlog du Sprint 2 84

3.3.3 Analyse Fonctionnelle : Routage et Enrichissement 85

3.3.4 Conception 86

3.3.5 Réalisation 87

3.3.6 Validation 87

3.3.7 Conclusion du Sprint 2 89

3.4 Implémentation du Modèle d'Apprentissage Profond LSTM-VAE (Sprint 3) 89

3.4.1 Backlog du Sprint 3 89
3.4.2 Analyse : Justification Stratégique et Choix Algorithmique . . . 90
3.4.3 Conception : Ingénierie des Données (Data Preprocessing & Feature Engineering) 91
3.4.4 Conception 93
3.4.5 Réalisation : Entraînement et choix des hyperparamètres 95
3.4.6 Réalisation : Calibration Rigoureuse du Seuil de Décision . . . 95
3.4.7 Validation : Étude comparative des architectures de détection . 98
3.4.8 Validation : Tests sur données synthétiques 98
3.4.9 Réalisation : Mécanismes de cyber-résilience et inférence temps-réel (Edge-to-ml) 99
3.4.10 Validation : Diagnostic et durcissement sécuritaire en conditions réelles 100
3.4.11 Validation : Convergence, Latence et Faux Positifs en Conditions Opérationnelles 102
3.4.12 Limites Identifiées et Perspectives 104
3.4.13 Conclusion du Sprint 3 104
3.5 Conclusion 104

3.1 Introduction

La validation théorique de la modélisation architecturale abordée au chapitre précédent marque la fin de la phase préparatoire et le début de l'implémentation opérationnelle de notre plateforme de cyberdéfense. Ce troisième chapitre est consacré à l'ingénierie détaillée de la *Release 1*, qui matérialise le **Plan de Données (*Data Plane*)** de notre architecture distribuée.

Dans le contexte des réseaux à très haut débit et des menaces polymorphes, la robustesse de toute couche d'Intelligence Artificielle dépend fondamentalement de la fidélité, de la vélocité et de l'intégrité des informations qu'elle ingère. La littérature scientifique démontre qu'une IA, aussi optimisée soit-elle, échouera face à une architecture d'acquisition souffrant de perte de paquets (*Packet Drop*) ou de congestion de flux [82]. Cette première itération est donc d'une criticité absolue : elle doit garantir une captation exhaustive en périphérie (*Edge Computing*), et un flux de transmission asynchrone hautement résilient, avant d'alimenter nos algorithmes d'apprentissage profond.

Conformément à notre cadre méthodologique Agile Scrum et aux bonnes pratiques d'ingénierie de la donnée (*Data Engineering*), nous formalisons ici l'exécution technique des trois premiers Sprints :

- Le **Sprint 1** couvre le déploiement et l'optimisation bas-niveau de la sonde nids *Suricata*, en exploitant les capacités de *Zero-Copy* du noyau Linux pour éliminer la latence d'acquisition.
- Le **Sprint 2** est dédié à l'intégration du pipeline de streaming Big Data selon le paradigme de l'Architecture Lambda, exploitant *Apache Kafka* en mode KRaft pour l'inférence temps réel, et la *Stack ELK* pour l'investigation (*Forensic*).
- Le **Sprint 3** détaille l'ingénierie mathématique et logicielle de notre modèle stochastique *LSTM-VAE* sous *Keras 3*. Ce sprint présente également l'intégralité de la démarche expérimentale menée pour valider ce choix : ingénierie des caractéristiques comparée (sélection globale vs. sélection équilibrée par type d'attaque), calibration rigoureuse du seuil de décision (comparaison de quatre points de fonctionnement), étude comparative contre deux architectures alternatives (if et ae déterministe), tests de généralisation sur données synthétiques, et validation en conditions réelles contre trois classes d'attaques.

3.2 Déploiement et Optimisation du Moteur d'Inspection Réseau (Sprint 1)

L'objectif de ce premier Sprint est d'établir l'observabilité réseau primaire à la périphérie de l'infrastructure (*Edge Node*). Suricata agit comme le capteur sensoriel de notre architecture, traduisant les flux binaires bruts transitant sur le réseau en événements structurés, sémantiques et interrogeables. Pour une architecture *DevSecOps* à haute vélocité, cette phase d'ingénierie nécessite de garantir l'ingestion asynchrone du trafic sans introduire de goulots d'étranglement matériels (latence ou *Packet Drop*).

3.2.1 Backlog du Sprint 1

La planification opérationnelle de ce cycle repose sur la décomposition de l'Épique d'acquisition en récits systèmes (*System Stories*). Afin de structurer le déroulement de cette première phase d'ingénierie, le tableau 3.1 détaille les tâches d'infrastructure nécessaires pour garantir une ingestion du trafic sans friction.

TABLE 3.1 – Backlog détaillé du Sprint 1

ID	Sprint	User Story (System Story)
1.1	Sprint 1 : Déploiement NIDS	En tant que Système, je dois capturer le trafic réseau via l'interface en mode *promiscuous* sans altérer la latence de production.
1.2		En tant qu'Architecte, je veux configurer Suricata pour activer le multithreading noyau (`AF_PACKET`) et éviter le *packet drop*.
1.3		En tant que Système, je dois sérialiser les flux décodés au format unifié *EVE JSON* pour faciliter l'ingestion asynchrone.

3.2.2 Analyse Fonctionnelle

Le cas d'utilisation principal est la *Capture et Inspection Sémantique des Flux*, où le système autonome intercepte le trafic entrant et sortant pour y déceler les premières anomalies connues par l'évaluation des signatures heuristiques. Pour formaliser cette cinématique fonctionnelle, la description textuelle du cas d'utilisation nominal est spécifiée en détail par le tableau 3.2.

TABLE 3.2 – Description textuelle du cas d'utilisation d'acquisition Edge

Cas d'utilisation	Capture et Inspection Sémantique des Flux
Acteur	Suricata (Système Autonome déployé sur la VM Edge)
Résumé	Le moteur intercepte le trafic sur l'interface d'écoute, décode la couche applicative (HTTP, DNS, TLS), la compare aux règles locales et génère un journal unifié.
Pré-conditions	L'interface réseau virtuelle est basculée en mode *promiscuous*. Les règles de détection personnalisées sont strictement isolées.
Scénario nominal	1. Le paquet Ethernet arrive sur l'interface d'écoute. 2. Suricata acquiert le paquet en exploitant un tampon circulaire (*Ring Buffer*) *Zero-Copy*. 3. Le décodeur interne identifie le protocole applicatif. 4. Le moteur de règles évalue le flux via un arbre de décision optimisé pour générer d'éventuelles alertes. 5. Le module de sortie (*Output*) sérialise l'événement complet au sein du fichier `eve.json`.

Cas d'utilisation	**Capture et Inspection Sémantique des Flux**
Post-conditions	Un log JSON structuré est persisté sur le disque local, prêt à être ingéré en continu par les pipelines de streaming.

3.2.3 Conception

La conception de la couche d'acquisition nécessite une compréhension pointue de l'orchestration des tâches asynchrones au niveau du système d'exploitation. La méthode de capture des paquets dicte la performance limite du nids. Trois implémentations ont été évaluées :

1. *PCAP (Packet Capture) :* Interface historique mais obsolète pour les hauts débits. Chaque paquet engendre un basculement de contexte (*Context Switch*) entre l'espace noyau (*Kernel Space*) et l'espace utilisateur (*User Space*), entraînant une saturation CPU.

2. *DPDK (Data Plane Development Kit) :* Bypass total du noyau Linux. Bien qu'offrant des performances maximales, elle requiert des cartes réseau (NIC) physiques spécifiques et n'est pas viable dans notre infrastructure virtualisée VMware.

3. **AF_PACKET (Solution Retenue) :** API native du noyau Linux (version 3.X+) permettant d'accéder aux paquets avec une approche **Zero-Copy**, réduisant drastiquement les interruptions processeur (*Context Switches*). Le noyau écrit directement les paquets dans un tampon circulaire mémoire (*Memory-mapped Ring Buffer*), lu simultanément par l'application en espace utilisateur.

Afin de justifier notre choix pour le multithreading, le séquençage bas-niveau et l'orchestration interne des threads d'exécution au sein de la sonde réseau sont illustrés par le diagramme de la figure 3.1.

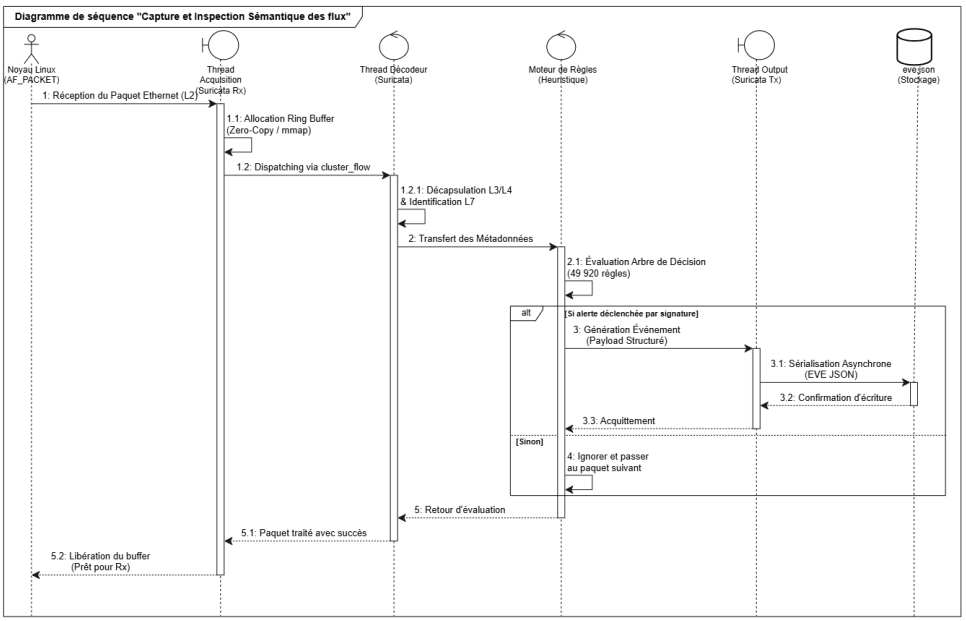


FIGURE 3.1 – Diagramme de séquence « Capture et Inspection Sémantique des Flux »

3.2.4 Réalisation

La réalisation technique s'est traduite par l'installation du démon Suricata et l'édition experte du fichier de configuration `suricata.yaml`. La méthode de capture a été configurée sur **AF_PACKET**.

Pour maximiser les performances, le mode d'exécution des *Threads* a été paramétré sur **Workers** (plutôt que *AutoFP*). Dans ce modèle, le même thread (CPU core) est responsable de l'acquisition, du décodage et de la détection d'un même paquet, supprimant les verrous de synchronisation (*Locks*) entre les cœurs.

De plus, le mode d'équilibrage de charge *cluster_flow* a été activé pour répartir l'inspection du trafic équitablement sur tous les cœurs CPU disponibles. Ce mode utilise un hachage symétrique garantissant que tous les paquets appartenant à une même session TCP (même quintuplet : IP source/dest, Port source/dest, Protocole) soient traités par le même thread, condition *sine qua non* pour permettre le réassemblage d'état TCP (TCP Stream Reassembly) et détecter les charges utiles fragmentées.

En matière de bonne gouvernance système, nous avons établi une contrainte stricte sur la VM Edge : l'exécution de tâches lourdes d'Entrée/Sortie (I/O) ou de mises à jour système est proscrite durant les phases de capture pour prévenir toute famine CPU (*CPU starvation*) induisant des pertes de paquets (*Packet Drops*).

3.2.5 Validation

L'intégrité de la sérialisation des métadonnées applicatives, ainsi que la levée immédiate des alertes déterministes, sont validées par l'inspection du journal de sortie. L'objectif principal de l'activation d'*EVE JSON* est de générer un format de données pivot, auto-descriptif et nativement ingérable par Apache Kafka, contournant les limitations syntaxiques des anciens fichiers de log plat (`fast.log`).

L'extrait de log présenté par la figure 3.2 confirme la conformité du format JSON requis par les composants aval de notre architecture.



```
{
  "timestamp": "2026-10-03T12:05:31.980502+0000",
  "flow_id": 1951207078738284,
  "in_iface": "ens34",
  "event_type": "flow",
  "src_ip": "192.168.56.128",
  "src_port": 51650,
  "dest_ip": "192.168.56.130",
  "dest_port": 5044,
  "ip_v": 4,
  "proto": "TCP",
  "flow": {
    "pkts_toserver": 7,
    "pkts_toclient": 2,
    "bytes_toserver": 8484,
    "bytes_toclient": 138,
    "start": "2026-10-03T12:05:26.847516+0000",
    "end": "2026-10-03T12:05:26.850488+0000",
    "age": 0,
    "state": "new",
    "reason": "timeout",
    "alerted": false,
    "exception_policy": [
      {
        "target": "stream_midstream",
        "policy": "ignore"
      }
    ]
  },
  "tcp": {
    "tcp_flags": "00",
    "tcp_flags_ts": "00",
    "tcp_flags_tc": "00"
  }
}
```


FIGURE 3.2 – Validation de la génération des logs de détection

Un point de vigilance identifié lors de cette phase de validation, et qui s'est révélé décisif ultérieurement (cf. §3.4.10), concerne la structure exacte du sous-objet `tcp` de l'événement `flow` : celui-ci expose des champs hexadécimaux cumulés (`tcp_flags`, `tcp_flags_ts`, `tcp_flags_tc`) plutôt que des indicateurs booléens nommés, une particularité de version qui a dû être prise en compte lors de l'extraction des caractéristiques pour le modèle de détection comportementale (Sprint 3).

3.2.6 Conclusion du Sprint 1

Ce premier Sprint a permis d'établir une couche d'acquisition réseau fiable, performante et exhaustive à la périphérie de l'infrastructure. Le choix de Suricata en mode `AF_PACKET`/`Workers`, validé par l'inspection de la télémétrie *EVE JSON*, garantit une captation du trafic sans perte de paquets et un format de sortie directement exploitable par les composants aval. Les particularités du format `EVE JSON` identifiées ici (structure du sous-objet `tcp`, disponibilité des règles locales) constituent des prérequis techniques directement réinvestis dans l'ingénierie des caractéristiques du Sprint 3.

3.3 Intégration du Pipeline de Streaming Double Chemin (Sprint 2)

La télémétrie brute sérialisée au format JSON sur la VM Edge (Sprint 1) est structurellement inutilisable en l'état pour notre nœud d'apprentissage automatique distant. Une transmission synchrone (ex : requêtes API directes) rendrait l'infrastructure monolithique et fragile : une coupure réseau ou une surcharge du modèle IA bloquerait la sonde réseau.

Ce Sprint 2 a pour objectif de construire un pipeline Big Data **à double chemin** hautement résilient : un chemin temps réel (*Fast Data*) via *Apache Kafka* pour alimenter le modèle ML en continu, et un chemin de stockage différé (*Cold Data*) via la *Stack ELK* (Elasticsearch, Logstash, Kibana) pour l'audit légal (Forensic) et la supervision par le SOC.

3.3.1 Justification Stratégique et Choix Architecturaux

L'acheminement de la donnée sans goulot d'étranglement a nécessité des choix d'ingénierie stricts, basés sur la comparaison des standards de l'industrie :

- **Courtage de messages : Apache Kafka vs RabbitMQ.** RabbitMQ (orienté file d'attente / *Queue-based*) supprime les messages une fois consommés et sature rapidement en mémoire vive lors d'un pic de charge (DDoS). **Apache Kafka** (orienté journal / *Log-based*) a été retenu car il persiste les événements sur disque, offrant un tampon d'absorption massif (*Shock Absorber*) et permettant à de multiples consommateurs (SOAR, ML) de relire les flux de manière asynchrone [25].

- **Consensus Kafka : KRaft vs Zookeeper.** Historiquement, Kafka dépendait de Zookeeper pour l'élection des contrôleurs, ce qui doublait l'empreinte mémoire (*JVM footprint*). Nous avons déployé Kafka dans son architecture moderne **KRaft** (Kafka Raft), implémentant directement le protocole de consensus mathématique *Raft* [83], réduisant drastiquement la consommation RAM, un critère vital pour notre infrastructure virtualisée.

- **Extraction et Transport : Filebeat vs Agent Lourd.** Déployer un ETL complexe directement sur la passerelle réseau (*Edge*) aurait provoqué une famine CPU (*CPU Starvation*) préjudiciable à Suricata. La solution retenue déporte la charge : un agent ultra-léger (**Filebeat**) expédie les logs bruts, et le traitement coûteux (Parsing, Typage) est réalisé de manière asynchrone par **Logstash** sur le nœud d'analyse distant.

3.3.2 Backlog du Sprint 2

Pour planifier l'acheminement de la donnée et l'enrichissement sémantique, les récits techniques nécessaires à ce pipeline de streaming sont détaillés par le tableau 3.3.

TABLE 3.3 – Backlog détaillé du Sprint 2

ID	Sprint	User Story (System Story)
2.1	Sprint 2 : Pipeline Double Chemin	En tant que Système, je dois expédier les logs vers Kafka (temps réel) ET vers Logstash (Filebeat) via une architecture découplée.
2.2		En tant qu'Architecte Data, je dois parser, typer et enrichir (GeoIP) les données dans Logstash pour contextualiser la menace.
2.3		En tant qu'Architecte, je veux indexer ces données dans Elasticsearch en limitant le *heap JVM* pour prévenir les erreurs *Out Of Memory*.
2.4		En tant qu'Analyste SOC, je veux un tableau de bord global (*Kibana*) pour auditer visuellement la volumétrie et les alertes.

3.3.3 Analyse Fonctionnelle : Routage et Enrichissement

Le cas d'utilisation central de ce Sprint est le *Routage Résilient et l'Enrichissement de la Donnée*. Ce processus asynchrone doit garantir qu'un événement généré à l'instant T soit streamé instantanément vers Kafka pour le modèle prédictif, tout en étant enrichi et indexé pour la supervision humaine. La description de ce courtage de messages est spécifiée par le tableau 3.4.

TABLE 3.4 – Description textuelle du cas d'utilisation de Streaming et d'ETL

Cas d'utilisation	Routage Résilient et Enrichissement
Acteurs	Collecteur Python, Agent Filebeat, Broker Kafka, Pipeline ELK
Résumé	Acheminement des logs bruts en temps réel via Kafka d'une part, et extraction, enrichissement sémantique et écriture en base de données distribuée de l'autre.
Pré-conditions	Les réseaux virtuels sont routés (communication inter-VM), et le broker Kafka est initialisé en mode KRaft.

Cas d'utilisation	**Routage Résilient et Enrichissement**
Scénario nominal	1. L'agent Python `edge_collector.py` ET le démon *Filebeat* détectent une nouvelle ligne JSON via un mécanisme de suivi d'inode (*Tailing*). 2. **Chemin IA (Temps Réel) :** Le collecteur Python expédie l'événement vers le *Topic* Kafka `network-features`. 3. **Chemin Forensic (Audit) :** *Filebeat* expédie l'événement vers le port d'écoute de *Logstash*. 4. *Logstash* parse le JSON, applique des mutations (typage explicite) et enrichit l'IP via une base GeoIP. 5. *Elasticsearch* indexe l'événement structuré dans ses *shards*.
Post-conditions	La donnée est disponible avec une latence quasi-nulle pour l'IA, et centralisée et requêtable dans Elasticsearch pour la supervision SOC.

3.3.4 Conception

La conception s'appuie sur le paradigme de l'architecture *Lambda*, garantissant un découplage strict entre la périphérie d'acquisition (*Edge*) et les clusters analytiques distants. Ce découplage protège le système en cas de partition réseau : si la connexion avec le nœud ML est rompue, Kafka met en cache les événements, et Filebeat gère une file d'attente à rétroaction (*Backpressure*). Les données sont transmises automatiquement dès le rétablissement de la connectivité sans aucune perte de paquets.

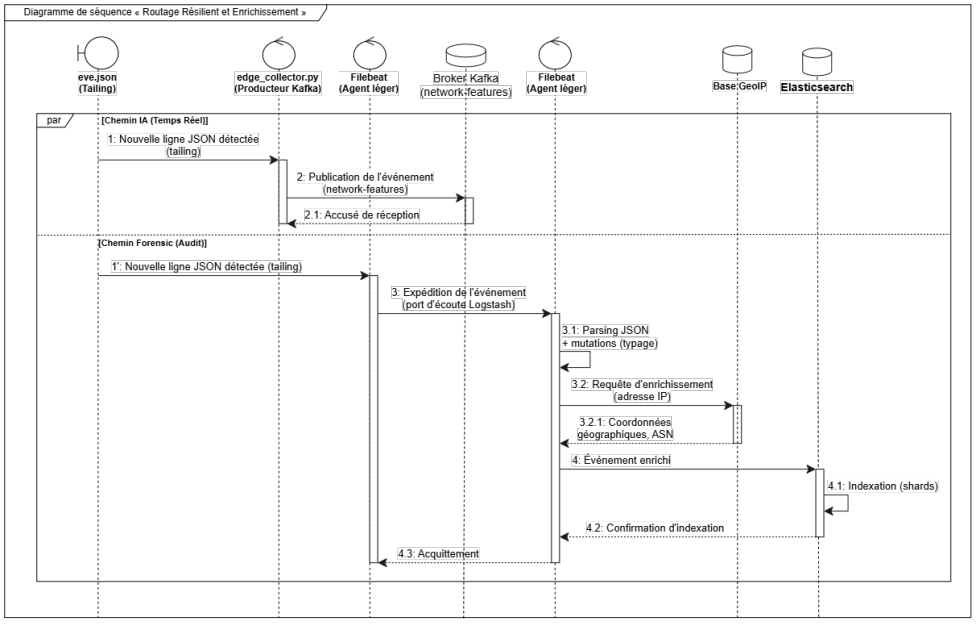


FIGURE 3.3 – Diagramme de séquence et d'architecture « Routage Résilient et Enrichissement »

3.3.5 Réalisation

Sur la VM Edge, le script `edge_collector.py` (agissant en tant que *Kafka Producer*) lit le descripteur du fichier `eve.json` en continu et expédie, pour chaque événement de type `flow`, un vecteur de caractéristiques normalisé vers le *Topic* `network-features`.

Un enseignement opérationnel important, établi ultérieurement lors de la campagne de diagnostic du Sprint 3 (§3.4.10), mérite d'être signalé ici : le paramètre de regroupement des envois (`linger_ms`) du producteur Kafka a été délibérément maintenu à sa valeur minimale (`linger_ms=0`, envoi immédiat par message) plutôt que configuré en *micro-batching*. Un essai de regroupement des messages a en effet été testé comme piste de réduction du bruit de détection généré par le trafic interne du pipeline lui-même ; il s'est révélé inefficace (les caractéristiques de type indicateur de drapeau TCP sont invariantes à la fréquence d'envoi) et contre-productif (l'augmentation de la taille de charge utile par envoi a dégradé d'autres caractéristiques). Cette configuration a donc été conservée à l'identique.

En parallèle, l'ETL (*Extract, Transform, Load*) s'opère sur **Logstash**. Dans le bloc de configuration *filter*, deux opérations critiques ont été codées :

1. **Type Coercion (Mutations) :** En JSON, les entiers peuvent être mal interprétés comme des chaînes de caractères (Strings). Une mutation force le typage strict des variables volumétriques (`bytes_in`, `pkts_out`), condition absolue pour que l'IA et Elasticsearch puissent exécuter des opérations mathématiques (MSE, Agrégations).

2. **Enrichissement CTI (GeoIP) :** Le filtre interroge une base de données de Cyber Threat Intelligence pour associer des coordonnées spatiales (Latitude/Longitude, ASN) aux adresses IP publiques attaquantes, fournissant un contexte géopolitique à l'analyste SOC.

Enfin, le moteur d'indexation **Elasticsearch** a fait l'objet d'un *Tuning* système. La taille de sa machine virtuelle Java (JVM) a été hardcodée (`-Xms1g -Xmx1g`) pour limiter la consommation de RAM. Sans cette limite, Elasticsearch tenterait d'absorber toute la mémoire disponible, risquant d'invoquer le processus *OOM Killer* de Linux, ce qui ferait crasher l'entraînement de notre modèle IA sur la même machine.

3.3.6 Validation

La robustesse théorique du pipeline décrite en conception ne vaut que si elle se vérifie en pratique. Cette section apporte trois preuves complémentaires de bon fonctionnement : la tolérance aux pannes du courtage Kafka, l'exactitude de l'enrichissement sémantique opéré par Logstash, et la capacité du tableau de bord à absorber le volume réel du trafic ingéré.

3.3.6.1 Preuve de Tolérance aux Pannes (Kafka)

Afin de valider l'intégrité de notre pipeline, nous avons vérifié le cœur de l'architecture asynchrone. La figure 3.4 démontre la réussite du courtage de messages. Un script consommateur (*Consumer*), instancié sur le nœud d'Intelligence Artificielle, s'abonne avec succès au *Topic*

`network-features` et dépile en temps réel la structure JSON émise par la passerelle Edge, prouvant le fonctionnement du mode KRaft sans Zookeeper.



FIGURE 3.4 – Validation de la communication asynchrone inter-nœuds via Apache Kafka

3.3.6.2 Preuve d'Enrichissement Sémantique (Logstash GeoIP)

L'opération d'ETL a été validée en injectant une adresse IP publique de test (localisée en Russie) dans le flux d'attaque. L'inspection de l'index dans Kibana (Figure 3.5) certifie que Logstash a correctement intercepté le flux, interrogé sa base de données, et ajouté la structure spatiale avant l'insertion dans Elasticsearch.

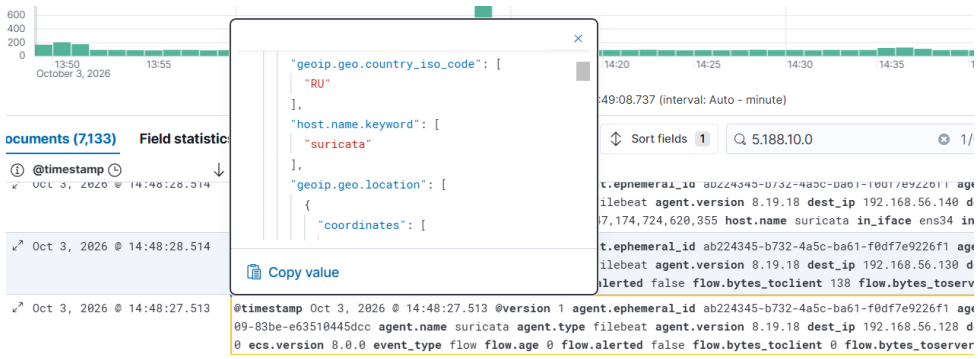


FIGURE 3.5 – Validation de l'enrichissement GeoIP par Logstash

3.3.6.3 Preuve de Scalabilité du Dashboard (Kibana)

Pour répondre aux exigences de supervision opérationnelle d'un SOC, les métriques sérialisées ont été agrégées dans un tableau de bord global de *Data Visualization*. La figure 3.6 démontre la capacité de l'architecture de streaming à supporter le volume massif du *Big Data* : près de 16,5 million d'événements ont été ingérés, catégorisés par criticité, et rendus interrogeables en quelques millisecondes.

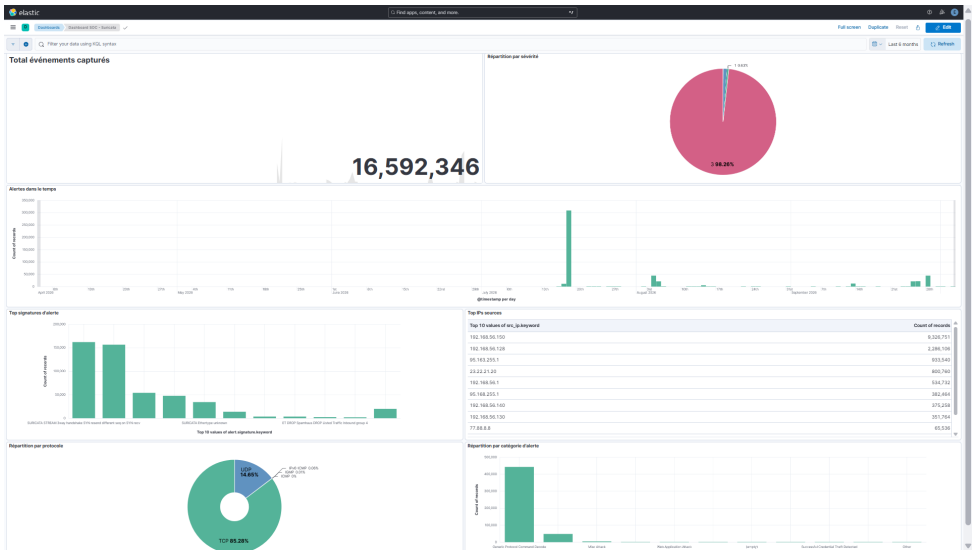


FIGURE 3.6 – Tableau de bord SOC global

3.3.7 Conclusion du Sprint 2

Ce Sprint a doté l'architecture d'un pipeline de streaming à double chemin, découplant strictement l'alimentation temps réel du modèle prédictif (Kafka) de la supervision forensique différée (ELK). La validation de la tolérance aux pannes, de l'enrichissement sémantique et de la scalabilité du tableau de bord confirme la robustesse de ce plan de données. Le canal Kafka ainsi constitué (`Topic network-features`) constitue le point d'entrée exclusif du moteur d'inférence comportemental développé au Sprint 3.

3.4 Implémentation du Modèle d'Apprentissage Profond LSTM-VAE (Sprint 3)

La détection des menaces furtives, telles que les fuites de données lentes (*Slow Read/Write*) ou les mouvements latéraux sophistiqués, dépasse par nature les capacités des systèmes de détection d'intrusion (ids) déterministes basés sur des signatures. Tout en s'inscrivant dans la continuité de notre gouvernance de projet **Agile Scrum** (Sprint 3), l'ingénierie de ce modèle d'apprentissage profond requiert l'application d'une méthodologie stricte issue du **MLOps** (*Machine Learning Operations*). Ce cadre d'ingénierie logicielle garantit la reproductibilité de l'ingénierie des caractéristiques (*Feature Engineering*), l'entraînement stochastique, la calibration empirique du seuil de décision, l'étude comparative de plusieurs familles de modèles, et le déploiement résilient de l'inférence en temps réel face aux cyberattaques visant l'infrastructure d'analyse elle-même.

3.4.1 Backlog du Sprint 3

Le tableau 3.5 détaille les récits systèmes couvrant, au-delà de l'implémentation initiale, l'intégralité de la démarche de validation scientifique du modèle retenue pour ce Sprint.

TABLE 3.5 – Backlog détaillé du Sprint 3

ID	Sprint	User Story (System Story)
3.1	Sprint 3 : Modèle LSTM-VAE	En tant que Data Scientist, je dois sélectionner un sous-ensemble de caractéristiques garantissant une couverture équilibrée entre types d'attaque, pas uniquement les plus fréquents.
3.2		En tant qu'Architecte IA, je dois concevoir et entraîner un modèle vae récurrent capable de modéliser la normalité du trafic sans supervision.
3.3		En tant qu'Ingénieur MLOps, je dois calibrer rigoureusement le seuil de décision par une méthodologie indépendante du jeu de test final.
3.4		En tant que Chercheur, je dois comparer objectivement le modèle retenu à des architectures alternatives (non supervisée classique, autoencodeur déterministe).
3.5		En tant que Data Scientist, je dois valider la capacité de généralisation du modèle sur des données synthétiques indépendantes du jeu d'entraînement.
3.6		En tant qu'Ingénieur Sécurité, je dois valider la détection en conditions réelles face à des attaques effectivement exécutées (scan, brute-force).
3.7		En tant qu'Architecte Sécurité, je dois diagnostiquer et corriger les faux positifs générés par le trafic légitime de l'infrastructure elle-même.

3.4.2 Analyse : Justification Stratégique et Choix Algorithmique

Dans le cadre de la cybersécurité réseau, la sélection de l'algorithme d'apprentissage non supervisé est la pierre angulaire de l'architecture. Trois familles d'approches ont été implémentées, comparées et évaluées de façon empirique (méthodologie et résultats chiffrés détaillés en §3.4.7) :

- **Isolation Forest (if) :** Cet algorithme évalue la non-conformité spatiale d'une observation en isolant les points nécessitant peu de partitionnements aléatoires de l'espace des caractéristiques [34]. Il ignore par construction toute notion de séquentialité temporelle entre observations successives.
- **LSTM-Autoencodeur déterministe (ae) :** Architecture identique au modèle retenu (mêmes couches récurrentes, mêmes caractéristiques), mais sans composante variationnelle – goulot d'étranglement Dense classique et perte de reconstruction pure, sans divergence de Kullback-Leibler.

- **Deep LSTM-VAE (Solution Retenue) :** Le trafic réseau est intrinsèquement une série temporelle. Les couches **LSTM** (*Long Short-Term Memory*) capturent la dynamique séquentielle du comportement réseau à travers leur système de portes logiques (Forget, Input, Output gates) [84]. L'auto-encodeur variationnel (**VAE**) modélise la distribution probabiliste continue du trafic sain [85], la régularisation bayésienne de son espace latent contraignant le modèle à apprendre une frontière de normalité plus généraliste qu'un simple autoencodeur.

Cette justification qualitative initiale a fait l'objet d'une validation empirique complète et chiffrée, présentée en §3.4.7, qui confirme l'apport spécifique de chacun de ces choix architecturaux.

3.4.3 Conception : Ingénierie des Données (Data Preprocessing & Feature Engineering)

Afin de transformer les trames réseau brutes du dataset de référence *CIC-IDS-2017* [86] en tenseurs exploitables par le réseau de neurones, une phase critique de *Feature Engineering* a été orchestrée via la bibliothèque `pandas` et le module `scikit-learn` [87].

3.4.3.1 Prétraitement et Nettoyage

Chaque fichier source (huit journées de capture) est traité par blocs (*chunks* de 100 000 lignes) pour maîtriser l'empreinte mémoire. Les valeurs infinies et manquantes, fréquentes sur les métriques de débit (division par une durée de flux nulle), sont systématiquement éliminées (`replace([np.inf, -np.inf], np.nan).dropna()`). Le jeu d'entraînement est constitué **exclusivement** de trafic bénin (80 % des flux légitimes, tirage aléatoire à graine fixe pour garantir la reproductibilité), condition nécessaire à l'apprentissage non supervisé de la normalité.

3.4.3.2 Sélection des Caractéristiques : de la Sélection Globale à la Sélection Équilibrée

L'espace vectoriel initial du dataset comprenait plus de 78 caractéristiques. Une première stratégie de sélection statistique par `SelectKBest` (score anova F) a été appliquée sur l'ensemble du trafic malveillant regroupé en une seule classe binaire (« attaque » vs. « bénin »), retenant 16 caractéristiques (temporelles et volumétriques : écarts-types d'inter-arrivée, temps d'inactivité, statistiques de taille de paquet).

Cette première sélection s'est révélée **structurellement biaisée** : le score anova agrégé est dominé par les types d'attaque les plus nombreux dans l'échantillon (DDoS, DoS Hulk, représentant plusieurs centaines de milliers de flux), noyant le signal discriminant des types d'attaque statistiquement minoritaires mais opérationnellement critiques (attaques par force brute FTP/SSH, comportement de Bot). La validation empirique de ce modèle (§3.4.7) a confirmé un rappel proche de 0 % sur ces catégories.

Une seconde méthodologie de sélection, dite **équilibrée**, a donc été développée : le score anova est calculé **séparément pour chaque type d'attaque contre le trafic bénin**, et la

sélection finale combine, pour les catégories identifiées comme faibles, leurs caractéristiques les plus discriminantes propres, complétées jusqu'à 16 par les caractéristiques les plus fortes au sens du score global. Le tableau 3.6 synthétise les deux approches.

TABLE 3.6 – Comparaison des deux méthodologies de sélection de caractéristiques

Critère	**Sélection globale**	**Sélection équilibrée**
Nature des variables retenues	Timings/tailles agrégés (IAT, Idle)	Drapeaux TCP (SYN/PSH/ACK/URG) + tailles
Recall – FTP-Patator	≈ 0 %	98,01 %
Recall – SSH-Patator	≈ 0 %	6,23 %
Recall – Bot	≈ 1 %	12,15 %
Recall – DDoS / DoS Hulk	70 – 84 %	70 – 84 % (stable)
Recall – PortScan	≈ 0 %	≈ 0 % (inchangé, cf. §3.4.8)

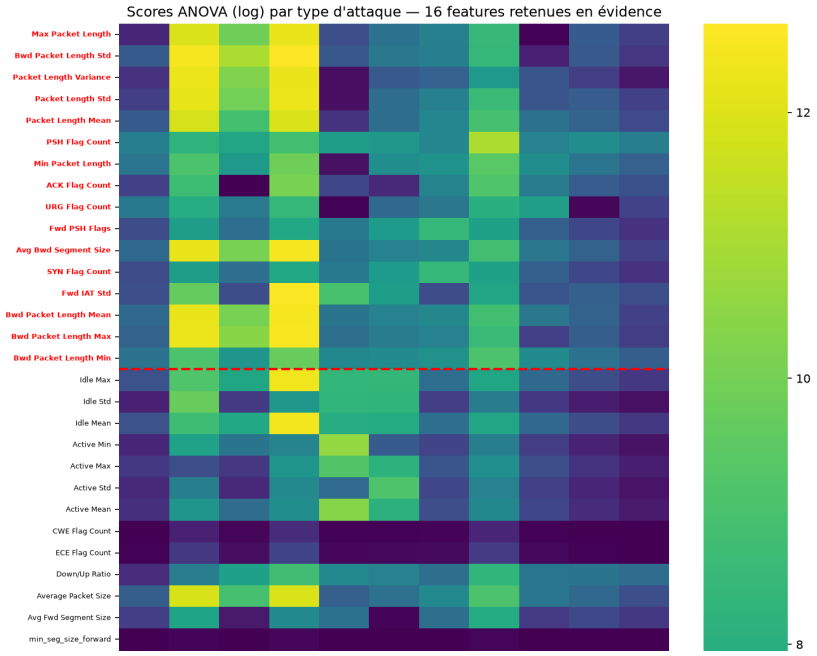


FIGURE 3.7 – Matrice des scores ANOVA par type d'attaque ayant motivé la sélection équilibrée

3.4.3.3 Mise à l'Échelle et Normalisation (Scaling)

Pour éviter la saturation des gradients (*Vanishing/Exploding Gradients*) et la divergence de la fonction de perte durant la rétropropagation, les caractéristiques du dataset ont nécessité une mise à l'échelle stricte. L'utilisation d'un `StandardScaler` a été écartée au profit d'un

`MinMaxScaler` (bornage strict sur l'intervalle $[0, 1]$), ajusté **uniquement** sur le sous-ensemble d'entraînement bénin pour prévenir toute fuite d'information (*data leakage*) entre les distributions d'entraînement et de test. Les cellules LSTM utilisant des fonctions d'activation non linéaires telles que la Sigmoïde ($\sigma(x) \in [0, 1]$), l'injection de tenseurs strictement normalisés sur les mêmes asymptotes mathématiques accélère considérablement la convergence des poids synaptiques du réseau.

3.4.4 Conception

L'architecture développée au cœur de notre module d'intelligence artificielle fusionne la mémorisation récurrente avec l'inférence variationnelle. L'encodeur LSTM traite la fenêtre temporelle de trafic ($T = 10$ pas de temps, séquences construites par IP source) et projette la séquence vers un espace latent Z de dimension $D = 16$ – retenue à l'identique du nombre de caractéristiques d'entrée, un choix pragmatique de départ documenté ici comme tel plutôt que le fruit d'une recherche exhaustive d'hyperparamètre.

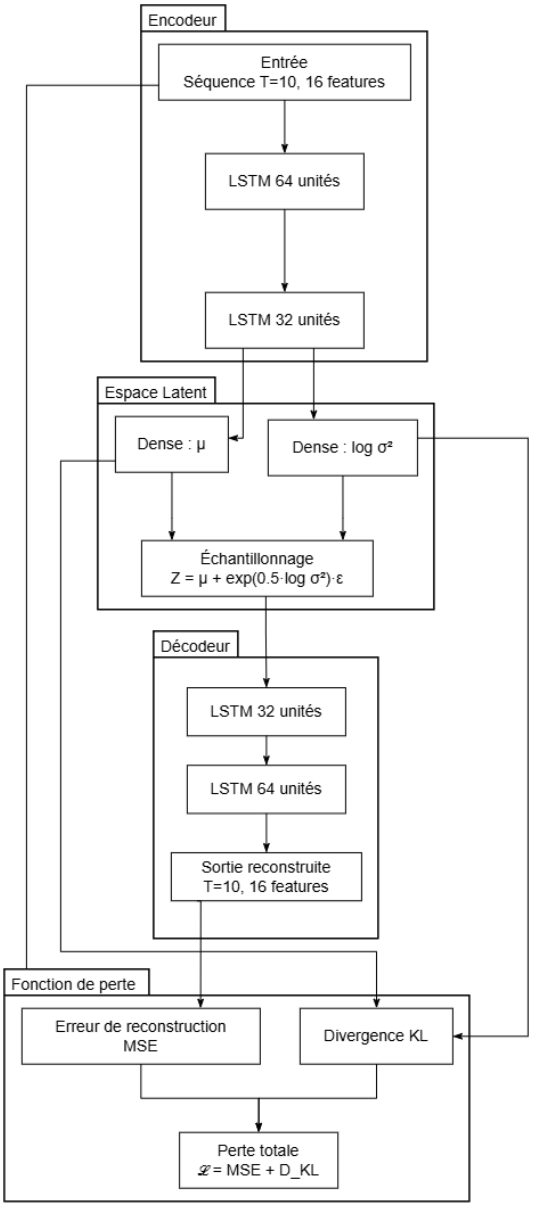


FIGURE 3.8 – Architecture et flux d'inférence probabiliste du modèle LSTM-VAE

L'apprentissage non supervisé de ce modèle s'appuie sur la maximisation de la borne inférieure de l'évidence, ou *Evidence Lower Bound* (elbo). Formulée en tant que fonction de perte (Loss) à minimiser, l'équation globale $\mathcal{L}(\theta, \phi; \mathbf{x})$ est définie par :

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{\mathrm{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z})) \tag{3.1}$$

Cette équation fondamentale force le réseau de neurones à accomplir un compromis mathématique strict :

1. **La reconstruction (Erreur Quadratique Moyenne – MSE) :** Le premier terme (l'espérance) évalue la capacité du décodeur p_θ à reconstituer la séquence originale $\mathbf{x}$ après sa compression. Un comportement anormal engendrera une erreur MSE élevée car le décodeur, entraîné exclusivement sur du trafic bénin, échoue à le reconstruire fidèlement.

2. **La régularisation (Divergence de Kullback-Leibler) :** Le terme D_{KL} contraint l'encodeur q_ϕ à mapper les séquences saines sur une distribution normale centrée réduite $\mathcal{N}(0, I)$ dans l'espace latent.

L'échantillonnage stochastique dans l'espace latent bloquant par définition le calcul des dérivées pour la rétropropagation du gradient, nous avons implémenté l'**Astuce de Reparamétrisation** (*Reparameterization Trick*). Cette solution mathématique a été codée via une couche Keras sur mesure (`CoucheEchantillonnage`) définie par :

$$\mathbf{z} = \boldsymbol{\mu} + \exp\left(\frac{1}{2}\mathbf{log}\,\boldsymbol{\sigma}^{\mathbf{2}}\right) \odot \boldsymbol{\epsilon} \quad \text{avec} \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I) \tag{3.2}$$

3.4.5 Réalisation : Entraînement et choix des hyperparamètres

L'entraînement a été conduit avec l'optimiseur **Adam**, un `batch_size` de 128, et le rappel Keras `EarlyStopping` (patience de 10 époques sur la perte de validation), évitant de fixer arbitrairement un nombre d'époques : l'entraînement s'interrompt organiquement lorsque `val_loss` cesse de s'améliorer, et seuls les poids synaptiques optimaux sont restaurés et sérialisés. La taille de fenêtre temporelle ($T = 10$) résulte d'un compromis pragmatique entre richesse contextuelle et latence d'inférence, plutôt que d'un balayage systématique – un point explicitement identifié comme axe d'amélioration méthodologique (cf. Limites, §3.4.8).

Le modèle déployé (architecture 64-32-16-32-64, seize caractéristiques, fenêtre de dix pas) a été entraîné en 4 480 secondes, soit environ 1 h 25. Pour documenter le coût de la procédure, un réentraînement complet a été réalisé pour ce rapport : 11 367 pas par époque, 16 à 17 millisecondes par pas, soit environ trois minutes par époque sur processeur, sans accélération matérielle. Prévu pour 50 époques au maximum, il s'est arrêté de lui-même à la dix-septième, la meilleure perte de validation ayant été atteinte à la septième. Ses seuils $\mu + 1\sigma = 58,44$ et $\mu + 3\sigma = 98,09$ sont à moins de 0,5 % de ceux du modèle déployé (58,28 et 97,62), ce qui confirme la reproductibilité de la procédure.

À l'issue de l'entraînement, l'erreur de reconstruction (MSE) sur le jeu de validation bénin a pour moyenne $\mu = 38,62$ et pour écart-type $\sigma = 19,67$. Le seuil de décision déployé en production est $\tau = \mu + 1,5\sigma = 68,12$, retenu après comparaison de quatre points de fonctionnement : l'optimum F_1 (37,77), $1,5\sigma$ (68,12), 2σ (77,95) et 3σ (97,62) (§3.4.6, tableau 3.7). Ce seuil global est complété, pour la règle de vote, par seize seuils individuels fixés au 98e percentile de l'erreur de chaque caractéristique sur le jeu de validation bénin.

3.4.6 Réalisation : Calibration Rigoureuse du Seuil de Décision

La performance d'un détecteur d'anomalies par reconstruction repose presque entièrement sur un seul paramètre scalaire : le seuil de décision au-delà duquel une erreur de reconstruction est jugée anormale. Cette sous-section documente la démarche suivie pour fixer ce seuil de façon rigoureuse plutôt qu'arbitraire – de la découverte d'un défaut de calibration silencieux jusqu'à la comparaison chiffrée de plusieurs points de fonctionnement candidats.

3.4.6.1 Diagnostic d'un Défaut de Recalibration

Une première anomalie méthodologique a été identifiée en cours de projet : le seuil opérationnel ($\mu+1\sigma$ de l'erreur de reconstruction sur le jeu de validation bénin) n'était **pas recalculé à chaque réentraînement** du modèle – le script d'entraînement initial ne persistait que le seuil à 3σ dans les métadonnées, tandis que le pipeline d'inférence et d'évaluation consommait silencieusement une valeur de repli codée en dur, potentiellement héritée d'un entraînement antérieur. Ce défaut a été corrigé en modifiant le script d'entraînement pour qu'il recalcule et persiste systématiquement μ, σ, le seuil à 1σ et les 16 seuils individuels par caractéristique (percentile 98) à chaque exécution.

3.4.6.2 Méthodologie de Calibration Rigoureuse

Après correction, une méthodologie de calibration indépendante du jeu de test final a été mise en œuvre : le jeu de test est scindé en un sous-ensemble de **calibration** (50 %, stratifié) utilisé exclusivement pour choisir le seuil, et un sous-ensemble de **test final** (50 %) sur lequel les métriques sont rapportées – évitant tout biais optimiste de calibration sur les mêmes données que celles utilisées pour le reporting.

Sur le sous-ensemble de calibration, la courbe roc (*Receiver Operating Characteristic*) et la courbe Précision-Rappel ont été tracées (figure 3.9), produisant une aire sous la courbe roc de AUC-ROC $= 0,79$ et une aire sous la courbe Précision-Rappel de AUC-PR $= 0,84$ – mesures indépendantes du choix d'un seuil particulier [88]. Ces courbes décrivent le comportement du détecteur pour l'ensemble des seuils possibles, et non pour un seuil unique : la première met en regard le taux de vrais positifs (part des attaques détectées) et le taux de faux positifs (part des séquences bénignes signalées à tort) lorsque le seuil sur l'erreur de reconstruction varie ; la seconde met en regard la précision (part des alertes qui correspondent à une vraie attaque) et le rappel.

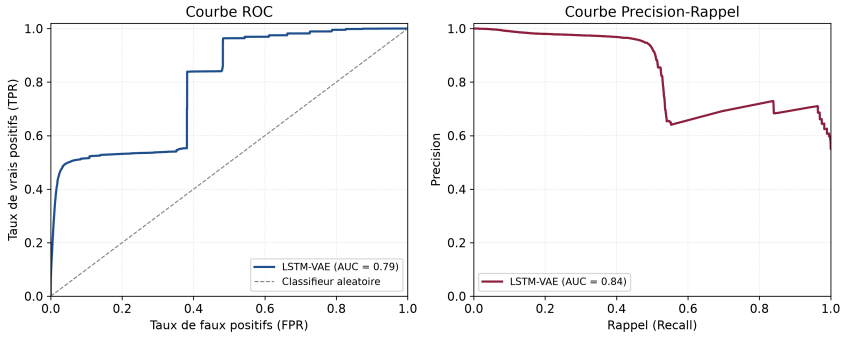


FIGURE 3.9 – Courbes ROC et Précision-Rappel sur le jeu de calibration

Ces courbes révèlent un comportement très contrasté, que la seule valeur d'AUC-ROC $= 0,79$ résume mal. La courbe roc s'élève très vite : environ la moitié des attaques est détectée pour moins de 5 % de séquences bénignes signalées à tort. Elle se stabilise ensuite sur un long plateau (rappel d'environ 0,52 à 0,55 pour des taux de faux positifs de 10 à 38 %), où abaisser le seuil n'apporte presque aucun gain. Deux marches verticales apparaissent enfin, à des taux de faux positifs

d'environ 38 % (le rappel passe de 0,55 à 0,84) puis 48 % (de 0,84 à 0,96). Une marche verticale traduit un groupe important de séquences d'attaque dont les erreurs de reconstruction sont presque identiques et se situent dans la partie haute de la distribution des séquences bénignes : ces attaques ne se distinguent du trafic normal qu'au prix d'un taux de fausses alertes inacceptable en exploitation. Le détecteur sépare donc très bien environ la moitié des attaques, et à peine l'autre moitié.

La courbe Précision-Rappel confirme cette lecture. La précision reste supérieure à 0,95 jusqu'à un rappel d'environ 0,48, puis chute brutalement (de 0,95 à 0,65 pour un rappel de 0,48 à 0,55) dès que le seuil atteint la zone où les séquences bénignes deviennent nombreuses ; elle remonte ensuite partiellement lorsque les deux groupes d'attaques sont captés. En fin de courbe, la précision tend vers environ 0,55 : à rappel total, elle égale la proportion d'attaques du jeu de calibration, qui constitue la ligne de base d'un classifieur sans pouvoir discriminant. Le AUC-PR $= 0,84$ se lit donc par rapport à cette ligne de base et non à zéro.

Deux enseignements en découlent pour le choix du seuil. D'une part, l'optimum du score F_1 se situe au sommet de la seconde marche, à un taux de faux positifs d'environ 48 % : près de la moitié du trafic bénin serait signalée, ce qui le rend inutilisable en production. D'autre part, le point de fonctionnement finalement retenu ($1,5\sigma$, taux de faux positifs de 9,23 %, rappel de 51,89 %) se situe à l'entrée du plateau : il capte la quasi-totalité des attaques bien séparées pour moins de 10 % de fausses alertes, ce qui correspond, avec la proportion d'attaques du jeu, à une précision d'environ 0,87. Aucun critère statistique unique ne suffit donc à trancher ; plusieurs critères de sélection ont ensuite été comparés : indice de Youden ($\max(\text{TPR} - \text{FPR})$), optimum du score F_1, optimum du score F_2 (pondérant le rappel), un critère sensible au coût (pondération $10\times$ pour un faux négatif), et des points de fonctionnement à taux de faux positifs fixé (1 %, 5 %, 10 %).

3.4.6.3 Comparaison des Points de Fonctionnement Retenus

Quatre seuils candidats, tous évalués sur le jeu de test final (jamais vu pendant la calibration) selon la règle de décision hybride complète (vote ≥ 4 caractéristiques au-delà de leur seuil individuel **OU** erreur globale au-delà du seuil), ont été comparés par type d'attaque. Le tableau 3.7 synthétise ce résultat central.

TABLE 3.7 – Comparaison des quatre points de fonctionnement du seuil de décision

Seuil	FPR global	Recall global	FTP-Patator	Infiltration	DDoS
F_1-optimal (37,77)	48,32 %	96,31 %	100,00 %	96,83 %	100,00 %
$1,5\sigma$ (68,12) – retenu	**9,23 %**	**51,89 %**	**98,01 %**	**55,56 %**	**83,98 %**
2σ (77,95)	5,81 %	51,89 %	91,59 %	42,86 %	83,98 %
3σ (97,62)	3,12 %	50,90 %	29,24 %	22,22 %	83,98 %

Le seuil $1,5\sigma$ ($\tau = 68,12$) a été retenu comme point de fonctionnement opérationnel : il ramène le taux de faux positifs de 48,32 % (optimum F_1, inexploitable en production) à 9,23 %, tout en préservant le rappel sur les catégories les plus dégradées aux seuils supérieurs (FTP-Patator, Infiltration). Cette calibration est documentée et justifiée directement dans les métadonnées du modèle produites en fin d'entraînement, garantissant la traçabilité du choix.

3.4.7 Validation : Étude comparative des architectures de détection

Afin de valider objectivement le choix du LSTM-VAE face aux alternatives présentées en §3.4.2, les trois architectures ont été entraînées et évaluées dans des conditions **strictement identiques** : mêmes 16 caractéristiques équilibrées, même jeu d'entraînement bénin, même jeu de test, même seuil recalibré pour chaque modèle selon la même méthodologie. L'Isolation Forest a été entraînée en mode non supervisé strict (`contamination='auto'`), sans aucune fuite de label. Le tableau 3.8 synthétise les résultats.

TABLE 3.8 – Étude comparative des trois architectures de détection (mêmes données, même méthodologie)

Métrique	**LSTM-VAE (retenu)**	**Isolation Forest**	**LSTM-AE (sans VAE)**
Recall global	51,89 %	50,95 %	50,19 %
FPR global	**9,23 %**	57,36 %	1,75 %
Precision	87,41 %	52,17 %	**97,24 %**
Recall FTP-Patator	**98,01 %**	0,23 %	0,00 %
Recall SSH-Patator	**6,23 %**	0,29 %	0,00 %
Recall DDoS	83,98 %	84,31 %	83,79 %

Ce résultat, bien que le rappel global agrégé soit quasi identique entre les trois modèles ($\approx$ 50–52 %), révèle une divergence radicale par type d'attaque : **sans la composante variationnelle**, le LSTM-Autoencodeur perd intégralement la capacité de détection sur les attaques par force brute (0,00 % contre 98,01 %), tandis qu'**Isolation Forest**, dénué de toute modélisation temporelle, présente un taux de faux positifs prohibitif (57,36 %) en configuration non supervisée honnête. Ce résultat valide empiriquement, avec preuve chiffrée, les deux choix architecturaux justifiés qualitativement en §3.4.2 : la modélisation séquentielle (LSTM) est décisive pour les attaques répétitives, et la régularisation variationnelle (VAE) est décisive pour la sensibilité fine du détecteur.

3.4.8 Validation : Tests sur données synthétiques

Au-delà de l'évaluation statistique sur données réelles, deux familles de tests complémentaires ont été menées pour éprouver la robustesse du modèle dans des conditions contrôlées : des

séquences synthétiques reproduisant fidèlement le profil statistique de chaque type d'attaque, puis des profils entièrement indépendants du jeu de données d'entraînement, afin de distinguer ce que le modèle a réellement appris de ce qu'il a simplement mémorisé.

3.4.8.1 Test Synthétique par Profil Statistique Réel

Pour chaque type d'attaque, des séquences synthétiques ont été construites à partir de la moyenne et de l'écart-type *réels* du type considéré (deux variantes : *prototype*, variance nulle, et *réaliste*, avec variance mesurée), à vérité terrain garantie. Ce test confirme la hiérarchie de détection observée sur données réelles et révèle un mécanisme intéressant sur les attaques par force brute FTP : le décalage moyen seul (configuration *prototype*, 0 %) ne suffit pas à la détection, c'est la **variance inter-tentatives** (configuration *réaliste*, 36,5 %) qui porte le signal discriminant – une nuance mécaniste que seul un test à variance contrôlée pouvait révéler.

3.4.8.2 Test d'Isolation du Cas PortScan

Un test synthétique dédié, construit à partir du profil statistique réel de 158 804 flux *PortScan*, démontre que même le signal le plus pur possible (moyenne exacte, variance nulle, répétée sur les 10 pas de temps de la fenêtre) reste très largement sous le seuil de décision (MSE = $41,74$ pour un seuil = $68,12$). Ce résultat élimine définitivement les hypothèses de sous-calibration du seuil ou de dilution par variance : le signal distinctif d'un scan de ports (diversité des ports de destination contactés par une même source) est structurellement absent de l'espace des 16 caractéristiques de niveau flux, quel que soit le modèle ou le seuil retenu.

3.4.8.3 Test de Généralisation Hors-Distribution

Un jeu de données artificiel, construit sans lecture d'aucune statistique CICIDS2017 (profils raisonnés à partir de connaissances réseau générales, exprimés en fractions de la plage normalisée apprise par le `MinMaxScaler`), a été soumis au modèle final. Après correction de deux artefacts méthodologiques identifiés en cours de test (encodage des drapeaux TCP en valeurs binaires plutôt que continues, variance temporelle artificiellement nulle), un profil de trafic bénin générique reste néanmoins classé comme anormal par le modèle. Ce résultat, honnête et reproductible sur trois tentatives indépendantes, illustre une propriété connue des détecteurs d'anomalies par reconstruction : la frontière de normalité apprise reflète une variété de corrélations conjointes spécifique à la distribution d'entraînement, non une région définie par des bornes indépendantes par variable. Cette limite est documentée comme telle en §3.4.12 et justifie *a posteriori* le choix architectural d'un déploiement on-premise calibré localement plutôt qu'un modèle générique transférable tel quel.

3.4.9 Réalisation : Mécanismes de cyber-résilience et inférence temps-réel (Edge-to-ml)

Le démon d'orchestration Kafka (`realtime_interface.py`) exécute le modèle en production. Au-delà de l'algorithmique IA, une ingénierie logicielle avancée a été codée pour garantir la survie

de l'infrastructure de défense face aux attaques par épuisement de ressources (*State Exhaustion*).

3.4.9.1 Tampon Glissant Isolé (Stateful Windowing)

Le maintien d'un historique temporel indépendant pour chaque équipement du réseau a été implémenté en exploitant la structure *thread-safe* `collections.deque`. Le démon alloue dynamiquement une fenêtre glissante de taille $T = 10$ par adresse IP source, isolant les contextes et prévenant le mélange des flux lors de l'inférence.

3.4.9.2 Pré-filtre Volumétrique Global (Bouclier Anti-DDoS Spoofing)

Justification de l'implémentation : Lors d'une attaque DDoS distribuée par usurpation d'adresse IP (*IP Spoofing*), l'attaquant génère une adresse aléatoire par paquet. Puisque le tampon du LSTM-VAE requiert 10 paquets pour évaluer une IP, l'attaque esquive le modèle par sa conception (*By Design*).

Ingénierie de la Solution : Un compteur macroscopique intercepte la métrologie du trafic total (indépendamment des IP) *avant* son ingestion par le LSTM-VAE. Si la vélocité globale excède le seuil physique de 300 paquets / 2 secondes, l'algorithme court-circuite le réseau de neurones et émet le signal critique `ENABLE_DDOS_SHIELD`.

3.4.9.3 Persistance Temporelle et Liste Blanche (Whitelist)

Afin de prévenir les tempêtes d'alertes (*Alert Storm*), une décision de blocage n'est déclenchée que si une même IP source cumule au moins **3 évaluations anormales sur ses 5 dernières séquences** (`SEUIL_PERSISTANCE=3`), absorbant les faux positifs isolés avant toute action automatique. Les actifs critiques déclarés (`actifs_critiques.json`, gérable depuis le tableau de bord SOC) sont exemptés de l'action de blocage automatique – l'anomalie reste néanmoins calculée, journalisée et visible par l'administrateur (`*** SÉCURITÉ WHITELIST ***`), garantissant qu'aucune usurpation d'identité whitelistée ne reste totalement invisible.

3.4.10 Validation : Diagnostic et durcissement sécuritaire en conditions réelles

3.4.10.1 Validation Face à des Attaques Réellement Exécutées

Trois classes d'attaques ont été menées en conditions réelles depuis une machine Kali Linux contre l'infrastructure de laboratoire, avec observation simultanée des journaux Suricata et du moteur d'inférence. Le tableau 3.9 synthétise ces trois campagnes.

TABLE 3.9 – Validation en conditions réelles : détection croisée Suricata / LSTM-VAE

Attaque	**Suricata**	**LSTM-VAE**	**Observation**
Nikto (scan web)	Détecté	Détecté (5 votes, MSE ≈ 200)	Détection croisée, blocage en < 3s
Nmap -sS (scan ports)	Détecté	Détecté (MSE ≈ 104)	Détection croisée, blocage en < 5s
Hydra (brute-force SSH)	Non détecté	Détecté (7 votes, MSE ≈ 225, blocage en 2 évaluations)	Aucune règle ET comportementale de brute-force SSH applicable en réseau privé

La figure 3.10 illustre une détection à large spectre. Côté Suricata, des alertes multiples sont levées dès 15 :58 :15 (inondation HTTP, sonde Spring Boot, anomalie de protocole, balayage SYN, accès administrateur ColdFusion). Côté modèle, sept caractéristiques sont en violation, dont des tailles de paquet, pour une erreur de reconstruction d'environ 200 face à un seuil de 68,12 ; la persistance (trois évaluations anormales sur les cinq dernières) est atteinte et la décision d'isolation émise à 15 :58 :23,341.

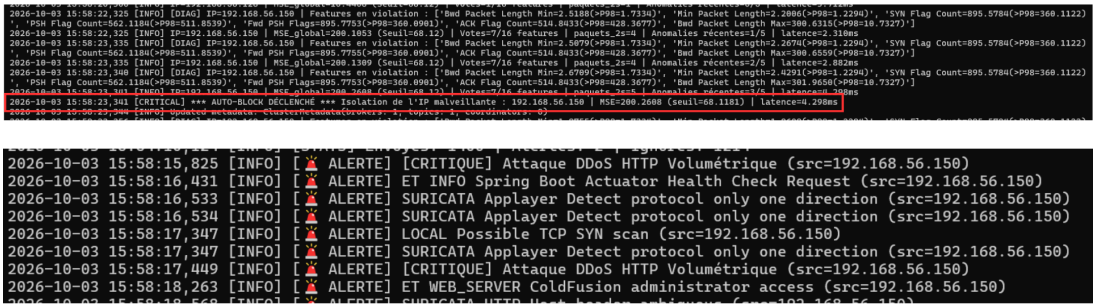


FIGURE 3.10 – Logs de détection croisée Suricata/LSTM-VAE lors de l'attaque Nikto

Le cas Nmap (figure 3.11), l'attaque a été détectée par les deux couches : alerte Suricata à 15 :38 :36, puis décision d'isolation du modèle à 15 :38 :39,635, quelque millisecondes après sa première évaluation anormale. Deux caractéristiques seulement sont en violation (`SYN Flag Count` et `ACK Flag Count`), aucune taille de paquet n'étant perturbée, contrairement aux cas Nikto et Hydra ; l'erreur globale de 104,4 dépasse néanmoins le seuil.

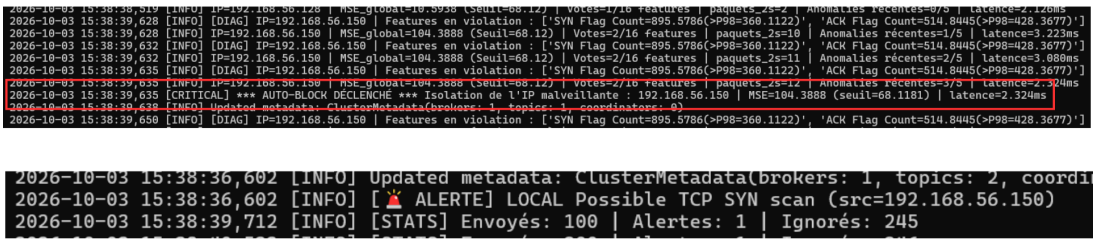


FIGURE 3.11 – Logs de détection croisée Suricata/LSTM-VAE lors de l'attaque Port Scan

Le cas Hydra (figure 3.12) produit le signal le plus intense de la série : sept caractéristiques en violation, dont des tailles de paquet (`Bwd Packet Length Max`, `Min Packet Length`), une erreur de reconstruction de 224,6 et une isolation à la troisième évaluation anormale. Il constitue la démonstration la plus directe de la complémentarité architecturale du système : l'inspection du jeu de règles actif confirme l'**absence de toute règle comportementale** de détection du brute-force SSH par comptage de tentatives échouées – la seule couverture théorique repose sur des listes de réputation d'IP publiques, structurellement inapplicables à un réseau privé de laboratoire. Le LSTM-VAE, lui, détecte et bloque cette attaque sans dépendre d'aucune signature préexistante.

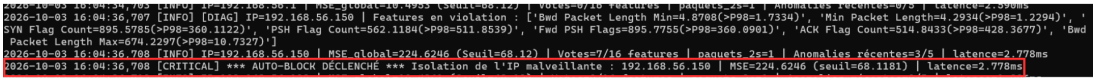


FIGURE 3.12 – Logs de détection et d'auto-blocage lors de l'attaque Hydra (brute-force SSH)

3.4.10.2 Durcissement contre l'Usurpation d'Adresse : Protection ARP

Ce mécanisme de confiance, comme tout mécanisme fondé sur l'adresse IP source, demeure théoriquement vulnérable à l'usurpation. Une contre-mesure au niveau de la couche liaison a donc été implémentée et **testée offensivement** : sur le nœud d'inférence, une règle `arptables` rejette toute trame ARP dont l'adresse IP source est celle de `vm-edge` et dont l'adresse MAC source diffère de la sienne, rendant caduque l'usurpation de cette adresse depuis un segment réseau tiers. La règle est rechargée à chaque démarrage par un service `systemd` dédié, et sa présence a été vérifiée après redémarrage de la machine virtuelle.

Cette protection a été validée par une attaque réelle d'empoisonnement ARP (`arpspoof`) menée depuis Kali Linux contre le nœud d'inférence, en se faisant passer pour `vm-edge`. En haut, la machine d'attaque émet en boucle des réponses ARP associant l'adresse de `vm-edge` à sa propre adresse MAC. En bas, la table de voisinage du nœud protégé, relevée pendant l'attaque, affiche toujours l'adresse MAC légitime de `vm-edge`, et le compteur de la règle progresse sur trois relevés successifs : 20, 22 puis 39 paquets rejetés (920, 1 012 puis 1 794 octets), tandis que la politique par défaut continue d'accepter le reste du trafic ARP (99 puis 114 paquets). Les trames falsifiées sont ainsi interceptées avant d'atteindre la table de résolution ARP du système.

3.4.11 Validation : Convergence, Latence et Faux Positifs en Conditions Opérationnelles

L'entraînement du modèle s'est stabilisé grâce à l'`EarlyStopping`, validant le calibrage du *Learning Rate* de l'optimiseur Adam (Figure 3.13).

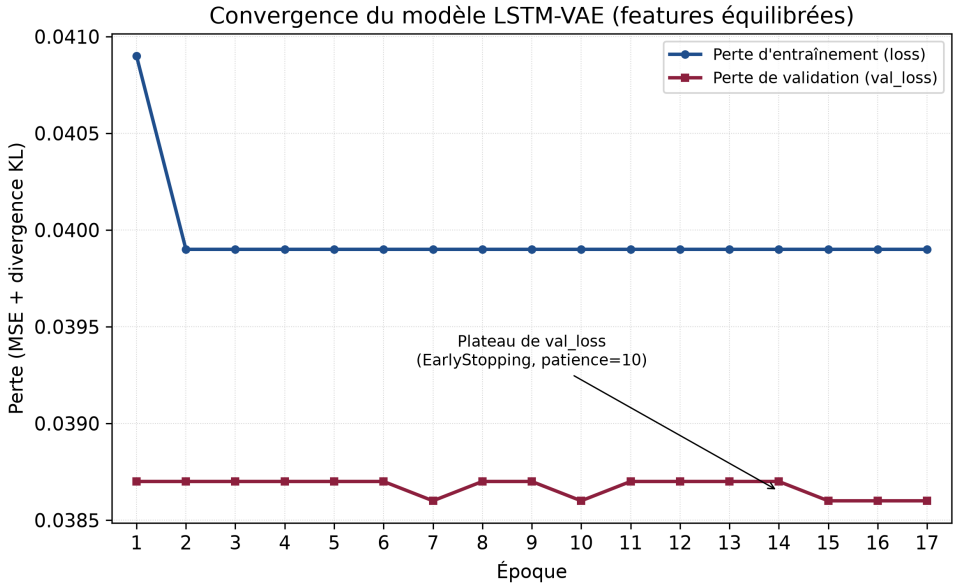


FIGURE 3.13 – Courbes de convergence algorithmique du modèle LSTM-VAE avec Early Stopping

Les journaux d'inférence en production certifient une latence computationnelle unitaire de l'ordre de **2 à 4 millisecondes** par séquence, cohérente avec la pré-compilation des graphes Tensor via le décorateur Keras `@tf.function(reduce_retracing=True)` et le *warm-up* du modèle au démarrage du service.

Chaque évaluation produit une ligne de trace unique qui rassemble les signaux de la règle de décision : l'erreur de reconstruction globale, confrontée au seuil de 68,12 ; le nombre de caractéristiques individuelles au-delà de leur seuil (votes, sur seize) ; le nombre d'évaluations anormales parmi les cinq dernières (persistance) ; et la latence de l'inférence. En régime nominal, sur les hôtes de la liste blanche, l'erreur reste stable autour de 10, très en deçà du seuil, avec zéro ou un vote et aucune évaluation anormale récente. Lors d'une attaque, les mêmes traces montrent une rupture nette : erreurs de 104 (balayage de ports) à 225 (force brute SSH), de deux à sept votes, et un compteur de persistance qui passe de 1 à 3 sur trois évaluations consécutives, la décision d'isolation n'étant émise qu'à la troisième (figure 3.11). C'est cet écart d'un ordre de grandeur entre régime nominal et régime d'attaque, combiné à l'exigence de trois évaluations anormales sur cinq, qui étouffe le bruit résiduel : une évaluation isolée au-delà du seuil ne déclenche aucune action.

Après application du mécanisme de neutralisation sélective et de la protection ARP (§3.4.10), le système opère avec un taux de faux positifs global de **9,23 %** au niveau séquence (seuil $1,5\sigma$), absorbé en pratique par la persistance temporelle avant toute action de blocage, et un rappel variant structurellement de $< 1\,\%$ (PortScan, attaques applicatives web) à 98 % (force brute FTP) selon le type d'attaque – une hétérogénéité expliquée mécaniquement en §3.4.8 et §3.4.12, non un défaut de calibration.

3.4.12 Limites Identifiées et Perspectives

La démarche de validation exhaustive menée dans ce Sprint met en évidence des limites structurelles, documentées ici en toute transparence plutôt que dissimulées :

- **Angle mort structurel sur le balayage de ports (PortScan) et les attaques applicatives web** : la signature de ces attaques repose respectivement sur la diversité de destinations à travers plusieurs flux (nécessitant une agrégation par IP source non disponible dans le format de capture actuel) et sur le contenu du payload applicatif (hors du périmètre d'une analyse de flux réseau). Ces angles morts sont structurellement comblés par la couche de détection par signatures (Suricata), justifiant *a posteriori* l'architecture hybride du système global.
- **Limite de généralisation hors distribution** (§3.4.8), documentée comme axe d'amélioration nécessitant une recalibration périodique sur trafic local.
- **Vulnérabilité résiduelle à l'usurpation IP**, significativement réduite par la protection `arptables` (validée expérimentalement) mais non éliminée dans l'absolu ; une authentification applicative (SASL/SSL) du canal Kafka constitue la perspective de durcissement complémentaire la plus robuste, documentée mais non implémentée dans le périmètre de ce Sprint.
- **Hyperparamètres non exhaustivement balayés** (dimension latente, taille de fenêtre temporelle) : les valeurs retenues sont pragmatiques et documentées comme telles plutôt que présentées comme un optimum démontré.

3.4.13 Conclusion du Sprint 3

Ce Sprint a livré, au-delà de l'implémentation du modèle LSTM-VAE, une démarche de validation scientifique complète et reproductible : une méthodologie de sélection de caractéristiques corrigeant un biais structurel identifié empiriquement, une calibration de seuil indépendante du jeu de test rapporté, une étude comparative chiffrée contre deux architectures alternatives confirmant la pertinence des choix effectués, des tests de généralisation sur données synthétiques et hors-distribution documentant honnêtement les limites du modèle, et une validation en conditions réelles ayant permis de détecter et corriger deux défauts opérationnels (configuration Suricata, faux positifs d'infrastructure) tout en démontrant, preuve à l'appui, la complémentarité effective entre détection par signatures et détection comportementale.

3.5 Conclusion

Ce troisième chapitre a scellé le succès technique et métrologique de notre *Release 1*. En validant conjointement les Sprints 1, 2 et 3, nous avons érigé un système d'acquisition et d'analyse robuste, dont chaque composante a été soumise à une démarche de preuve empirique plutôt qu'à une simple implémentation déclarative.

La sonde Suricata, optimisée via l'API *AF_PACKET* au niveau du noyau Linux, extrait la sémantique des flux à l'Edge sans perte de paquets. Le pipeline de streaming à double chemin, bâti sur le paradigme de l'Architecture Lambda, garantit la supervision SOC (*Forensic*) via la Stack ELK, et l'alimentation temps réel via Apache Kafka en mode KRaft.

Enfin, le moteur prédictif *LSTM-VAE*, développé sous Keras 3, a fait l'objet de la démarche de validation la plus approfondie du projet : sélection de caractéristiques corrigée par diagnostic empirique, calibration de seuil indépendante en quatre points de fonctionnement comparés, étude comparative contre *Isolation Forest* et un autoencodeur déterministe confirmant chiffres à l'appui la pertinence de l'architecture retenue, tests sur données synthétiques et hors-distribution documentant honnêtement ses limites, et validation en conditions réelles contre trois classes d'attaques ayant permis d'identifier et corriger deux défauts opérationnels concrets. Le système opère avec une latence d'inférence de 2 à 4 millisecondes et un taux de faux positifs maîtrisé de 9,23 % au niveau séquence, absorbé par la persistance temporelle avant toute action automatique.

L'ensemble de ce **Plan de Données (*Data Plane*)** étant pleinement opérationnel, résilient et rigoureusement validé, nous disposons de la matière première indispensable pour basculer de l'observation vers la cyberdéfense active. Le chapitre suivant (Chapitre 4) abordera la *Release 2*, dédiée à la composante exécutive du projet : le développement de l'orchestrateur de réponse autonome (*SOAR*) et la mise en œuvre de la gouvernance *Zero Trust*.

CHAPITRE 4

PRÉVENTION, SOAR, AGENTS IA ET DASHBOARD SOC (RELEASE 2)

Sommaire

4.1 Introduction 108

4.2 Sprint 4 : Anticipation par le renseignement (agent CTI et triage Dual-RAG) 108

4.2.1 Backlog du sprint 4 108

4.2.2 Analyse Fonctionnelle 109

4.2.3 Conception 111

4.2.4 Réalisation 112

4.2.5 Validation 114

4.2.6 Conclusion du sprint 4 116

4.3 Sprint 5 : Mitigation du risque humain (agent Gemini et RAG LMS) 117

4.3.1 Backlog du sprint 5 117

4.3.2 Analyse Fonctionnelle 118

4.3.3 Conception 119

4.3.4 Réalisation 120

4.3.5 Validation 123

4.3.6 Conclusion du sprint 5 125

4.4 Sprint 6 : Orchestration SOAR multi-paliers, agent post-attaque et dashboard SOC 126

4.4.1 Backlog du sprint 6 126

4.4.2 Analyse Fonctionnelle 127
4.4.3 Conception 128
4.4.4 Réalisation 133
4.4.5 Validation 139
4.4.6 Conclusion du sprint 6 142
Conclusion 142

4.1 Introduction

La première release a livré la couche d'interception et de détection : la sonde IDS Suricata, le bus Kafka et le moteur d'inférence LSTMVAE (chapitre 3). Ce chapitre présente la seconde release, qui transforme cette capacité de détection en une capacité de *prévention* et de *réponse autonome* : anticipation des menaces par renseignement, mitigation du risque humain, et pilotage centralisé du SOC par une orchestration SOAR multi-paliers.

Il convient de préciser d'emblée le périmètre du SOAR dans NG-IDPS : il ne s'agit pas d'un composant de détection, mais d'un moteur de *corrélation, décision et remédiation* qui consomme les verdicts produits par la release 1 (Suricata, LSTMVAE) pour décider d'une action (journalisation, notification, blocage) et l'exécuter physiquement sur l'infrastructure.

La release 2 est organisée en trois sprints, numérotés dans la continuité de la release 1 et résumés dans le tableau 4.1. Chaque sprint est présenté selon la même démarche : backlog, analyse, conception, réalisation et validation.

TABLE 4.1 – Répartition des sprints de la release 2.

Sprint	**Intitulé**	**Livrable majeur**
Sprint 4	Anticipation par le renseignement (agent CTI et triage *Dual-RAG*)	Génération automatique de règles Suricata et aiguillage vers la sensibilisation
Sprint 5	Mitigation du risque humain (agent Gemini et RAG LMS)	Plateforme de sensibilisation ciblée post-incident
Sprint 6	Orchestration SOAR multi-paliers, agent post-attaque et dashboard SOC	Boucle de réponse fermée et supervision temps réel

4.2 Sprint 4 : Anticipation par le renseignement (agent CTI et triage Dual-RAG)

Ce premier sprint de la release 2 vise à automatiser la veille OSINT et la production de règles de détection préventives, afin de réduire la fenêtre d'exposition entre la publication d'une menace et sa détection par Suricata. L'agent réalisé assure en outre un *triage* de chaque article ingéré, aiguillant les menaces techniques vers la génération de règles et les contenus de sensibilisation vers le pipeline de formation du sprint 5.

4.2.1 Backlog du sprint 4

Le backlog ci-dessous regroupe les user stories retenues pour ce sprint, priorisées selon leur contribution à la chaîne de triage et de génération de règles (tableau 4.2).

TABLE 4.2 – Backlog du sprint 4.

ID	User story	Critère d'acceptation	Priorité
US4.1	En tant qu'analyste SOC, je veux que chaque article OSINT ingéré soit classé automatiquement en *menace technique*, *sensibilisation* ou *bruit*, afin de router le contenu vers le bon pipeline.	Chaque article reçoit un `type_article` parmi `TECHNICAL_THREAT`, `AWARENESS`, `NOISE`.	Haute
US4.2	En tant qu'analyste, je veux que les menaces soient vectorisées (`nomic-embed-text`) afin de les comparer à l'historique et aux règles existantes.	Chaque article est converti en vecteur et comparé par recherche sémantique.	Haute
US4.3	En tant qu'analyste, je veux une recherche de similitude sur l'index FAISS des règles Suricata de référence afin d'éviter les doublons et de contextualiser la menace.	Les k règles de référence les plus proches sont injectées dans le contexte du LLM.	Haute
US4.4	En tant qu'analyste, je veux qu'une règle Suricata candidate soit générée localement par `llama3.2:3b` pour toute menace technique non couverte.	Une règle candidate est produite et associée à l'article.	Haute
US4.5	En tant que responsable sécurité, je veux que la sortie du LLM soit validée par un schéma Pydantic v2 (score de fiabilité, IoCs, règle) afin de garantir sa conformité avant toute publication.	Toute sortie non conforme (*ex.* contenu publicitaire classé `NOISE`) est rejetée et journalisée.	Haute
US4.6	En tant que responsable sécurité, je veux qu'une règle validée soit persistée avec un statut `PENDING` afin qu'un administrateur puisse l'approuver avant déploiement.	La règle est visible sur le dashboard avec son statut.	Moyenne

4.2.2 Analyse Fonctionnelle

Le cas d'utilisation principal de ce Sprint est le *Triage et Traitement d'un Article de Renseignement*, où l'agent CTI ingère chaque article OSINT, le classe automatiquement entre menace

technique, sensibilisation ou bruit, puis déclenche l'action correspondante (génération d'une règle Suricata candidate ou aiguillage vers le pipeline de sensibilisation du sprint 5). Pour formaliser cette cinématique fonctionnelle, la description textuelle du cas d'utilisation nominal est spécifiée en détail par le tableau 4.3.

TABLE 4.3 – Description textuelle du cas d'utilisation « Trier et traiter un article de renseignement ».

Cas d'utilisation	**Trier et traiter un article de renseignement**
Acteurs	Agent CTI (principal), analyste SOC (secondaire), sonde Suricata (système externe)
Pré-conditions	Ollama actif ; index FAISS et bases SQLite chargés ; au moins une source OSINT active.
Scénario nominal	1. L'agent interroge le backend pour récupérer les sources OSINT actives. 2. Pour chaque source, il ingère les articles non encore traités (déduplication par lien/identifiant). 3. Il vectorise chaque article (`nomic-embed-text`) et interroge la mémoire épisodique et l'index FAISS. 4. Il demande à `llama3.2:3b` un triage strict au format JSON (type, titre, résumé, IoCs, règle, score de fiabilité). 5. La sortie est validée par le schéma Pydantic v2. 6. Selon le `type_article` : une règle candidate (`TECHNICAL_THREAT`) ou un contenu de sensibilisation (`AWARENESS`) est persisté en base avec le statut initial correspondant. 7. L'article est mémorisé dans la base vectorielle épisodique pour enrichir les triages futurs.
Scénarios alternatifs	(A1) Article déjà traité (lien connu) : ignoré sans appel au LLM. (A2) Sortie classée `NOISE` (contenu publicitaire/hors-périmètre) : rejet par le validateur Pydantic, aucune persistance. (A3) Sortie non conforme au schéma : rejet et journalisation (`ValidationError`). (A4) Source inaccessible ou type non reconnu : source ignorée pour le cycle courant.
Post-conditions	Un article est persisté en base avec un statut `PENDING` (menace technique) ou `PENDING_FORMATION` (sensibilisation), ou rejeté sans effet de bord.

4.2.3 Conception

Cette section détaille l'architecture du sprint, la dynamique du pipeline, puis son modèle de données.

4.2.3.1 Architecture du sprint

L'agent CTI (`agent_cti.py`) s'exécute sur `vm-response` et s'appuie exclusivement sur l'inférence locale Ollama (`llama3.2:3b` pour la génération, `nomic-embed-text` pour les embeddings), ce qui préserve la confidentialité des données de renseignement en évitant tout appel à un fournisseur LLM externe pour ce sprint. La figure 4.1 illustre ses interactions avec les bases vectorielles, le LLM local, l'API du backend et la sonde Suricata.

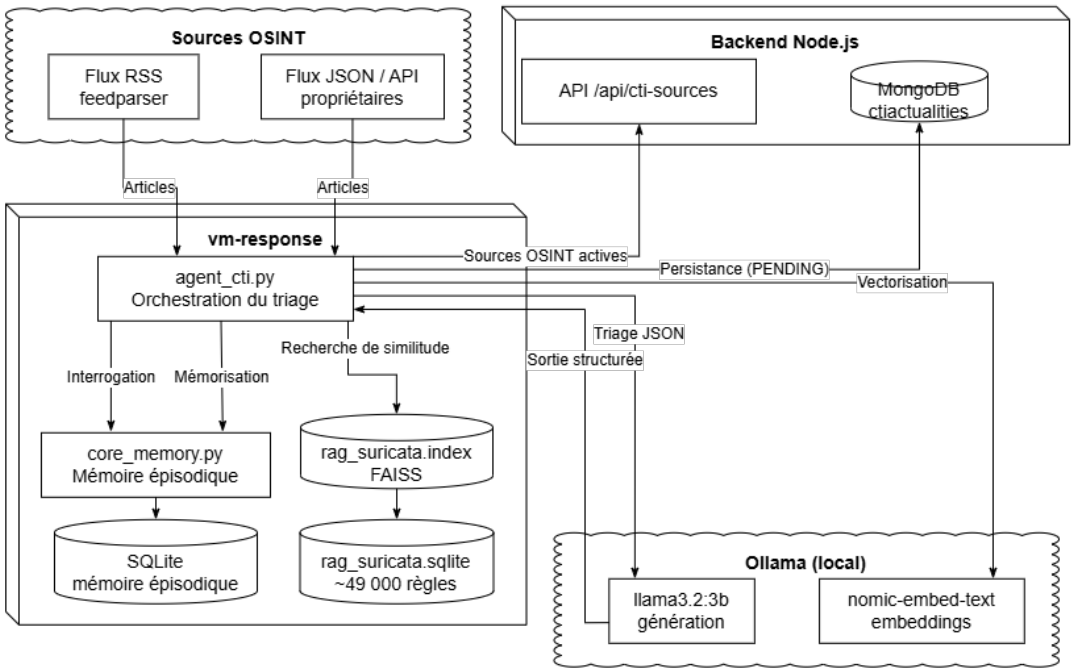


FIGURE 4.1 – Architecture du sprint 4

Le pipeline *Dual-RAG* repose sur deux volets de récupération complémentaires et indépendants : la mémoire épisodique des menaces déjà traitées (`core_memory.py`, recherche sémantique sur les incidents passés) et l'index des règles Suricata de référence (FAISS + SQLite, `rag_suricata.index`/`rag_suricata.sqlite`), tous deux interrogés avant la génération [56]. Les sources OSINT elles-mêmes sont hétérogènes : flux RSS classiques (`feedparser`) et flux JSON/API propriétaires, toutes deux administrables dynamiquement depuis le dashboard.

4.2.3.2 Diagramme de séquence

La figure 4.2 décrit l'enchaînement des appels, de la collecte OSINT jusqu'à la persistance de l'article trié.

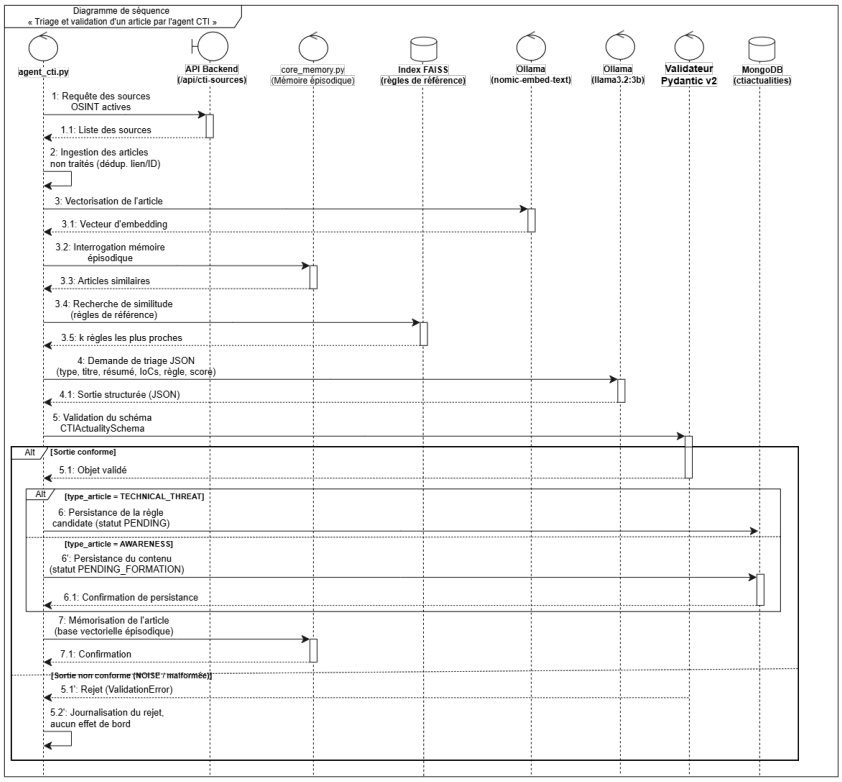


FIGURE 4.2 – Diagramme de séquence "triage et validation d'un article par l'agent CTI"

4.2.4 Réalisation

Cette section présente l'implémentation des composants et les interfaces obtenues.

4.2.4.1 Environnement et composants

Les composants du sprint sont regroupés dans `vm-response/` : `agent_cti.py` (orchestration du triage), `core_memory.py` (mémoire épisodique SQLite/FAISS), ainsi que l'index de référence `rag_suricata.index` et sa base `rag_suricata.sqlite` (environ 49 000 règles Suricata pré-indexées, utilisées en lecture seule).

L'agent publie ses résultats via l'API du backend Node.js (`/api/cti-sources` pour la lecture des sources actives, persistance dans la collection MongoDB `ctiactualities`), ce qui découple le processus d'inférence de la couche de présentation.

4.2.4.2 Interfaces et résultats

Les captures suivantes illustrent l'exécution de l'agent et le résultat produit.

La figure 4.3 montre la fenêtre de journal du tableau de bord à l'issue d'une exécution de l'agent CTI, lancée à la demande et terminée en 9 secondes. L'en-tête rappelle l'état de l'exécution (« Terminé »), son heure de lancement et sa durée. Le corps du journal, dont les lignes de débogage sont masquées, déroule le cycle complet dans l'ordre chronologique. L'agent charge d'abord ses ressources : le jeu de 49 655 règles de référence qui sert de modèle syntaxique, la base locale `threat_intel` et l'index vectoriel (50 empreintes en mémoire). Il parcourt ensuite les sources

actives une à une. Pour la première, Hacker News, un article nouveau est repéré puis analysé par le modèle, et le triage le classe `TECHNICAL_THREAT` ; le bulletin est alors inséré dans MongoDB au statut `PENDING`, en attente de validation par l'analyste, et la ligne `[GUARDRAIL]` confirme que sa sortie a satisfait le contrôle de structure. Cette analyse occupe environ 5,5 secondes de l'exécution. Les deux sources suivantes, CERT-FR et le flux de démonstration, n'appellent aucune analyse : aucune ligne de traitement n'apparaît pour elles. Cette séquence illustre le scénario T4.1 du tableau 4.4 (classification et persistance d'un article technique) ; la règle de détection proposée pour un tel article est présentée à la figure 4.4.



FIGURE 4.3 – Journal d'exécution de l'agent CTI : ingestion et triage d'un article

La figure 4.4 illustre le cycle complet d'une règle candidate : la carte générée par l'agent CTI pour la faille Citrix NetScaler (en bas), puis son déploiement réussi sur `vm-edge` par l'agent de déploiement (en haut), qui valide la syntaxe par un test à blanc avant l'injection et le rechargement à chaud de Suricata, et dont le statut passe à « Active en production ».

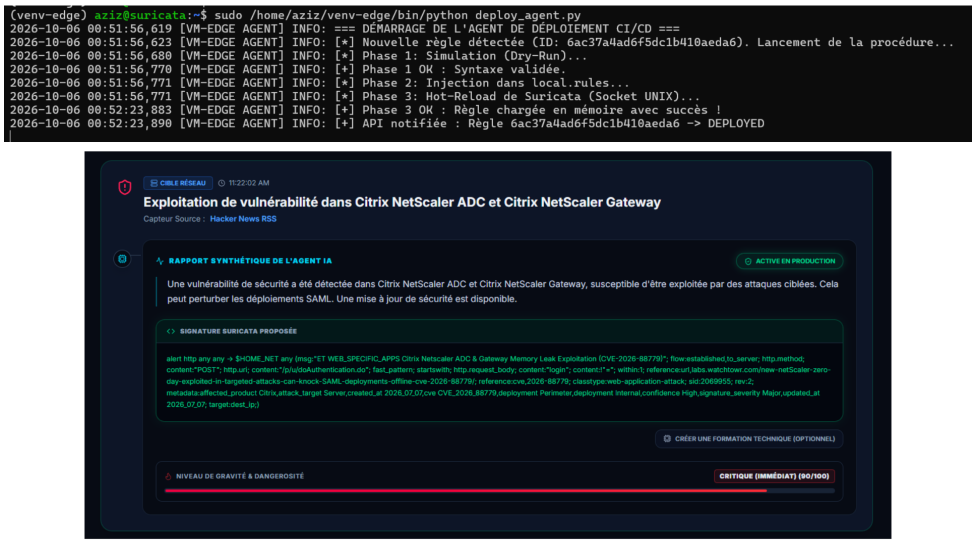


FIGURE 4.4 – Génération et validation d'une règle Suricata candidate

La page Veille CTI (figure 4.5) centralise l'activité de l'agent : les compteurs du haut distinguent les bulletins totaux, ceux en attente de validation côté Suricata, les règles déjà actives en production et les cibles orientées vers la sensibilisation, tandis que le fil d'actualités liste chaque article analysé avec son statut, sa catégorie et le niveau de gravité attribué par l'agent.

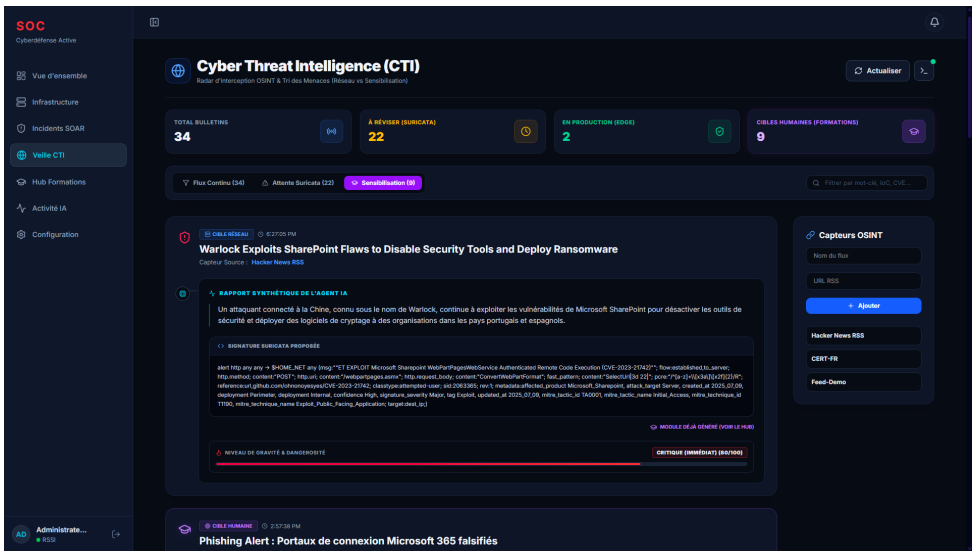


FIGURE 4.5 – Fil d'actualités CTI affiché sur le dashboard, statut `PENDING`

4.2.5 Validation

La validation vérifie chaque critère d'acceptation du backlog par un scénario de test dédié (tableau 4.4).

TABLE 4.4 – Plan de validation du sprint 4.

Test	Scénario	Résultat attendu	Résultat observé
T4.1	Article technique nouveau soumis à l'agent	Classé `TECHNICAL_THREAT`, règle candidate générée	Figure 4.6
T4.2	Article de sensibilisation	Classé `AWARENESS`, statut `PENDING_FORMATION`	Figure 4.7
T4.3	Contenu publicitaire/hors-périmètre	Rejeté (`NOISE`), non persisté	Figure 4.8
T4.4	Article déjà présent en base (même lien)	Ignoré sans appel au LLM	Figure 4.9

La figure 4.6 montre un article classé `TECHNICAL_THREAT` en attente de validation : résumé exécutif, règle Suricata candidate et niveau de gravité (ici neutralisé à 50/100, le modèle ayant initialement renvoyé un score non exploitable), avant la décision de l'analyste (« Approuver & Injecter »).

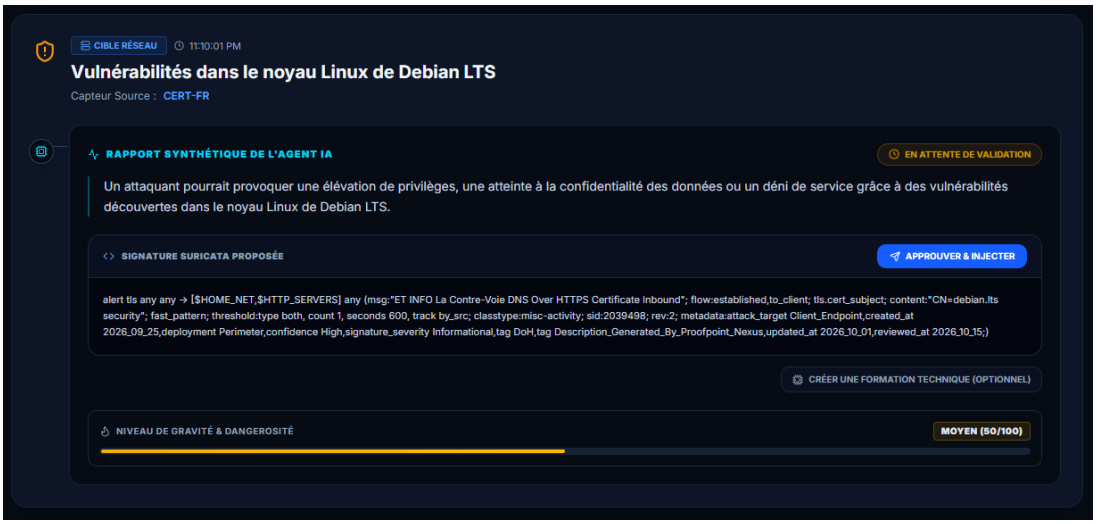


FIGURE 4.6 – Article technique

La figure 4.7 montre un article classé `AWARENESS` : la cible est humaine plutôt que réseau, aucune règle Suricata n'est proposée, et l'analyste dispose d'un bouton pour générer directement le module de formation correspondant.



FIGURE 4.7 – Article de sensibilisation

La figure 4.8 illustre le pré-filtre promotionnel ajouté au garde-fou : l'article « Black Friday Sale », bien qu'étiqueté `AWARENESS` par la source, est reconnu comme publicitaire avant toute analyse et écarté sans appel au modèle, comme le confirme le bilan de l'exécution (« filtrés (NOISE)=1 »).

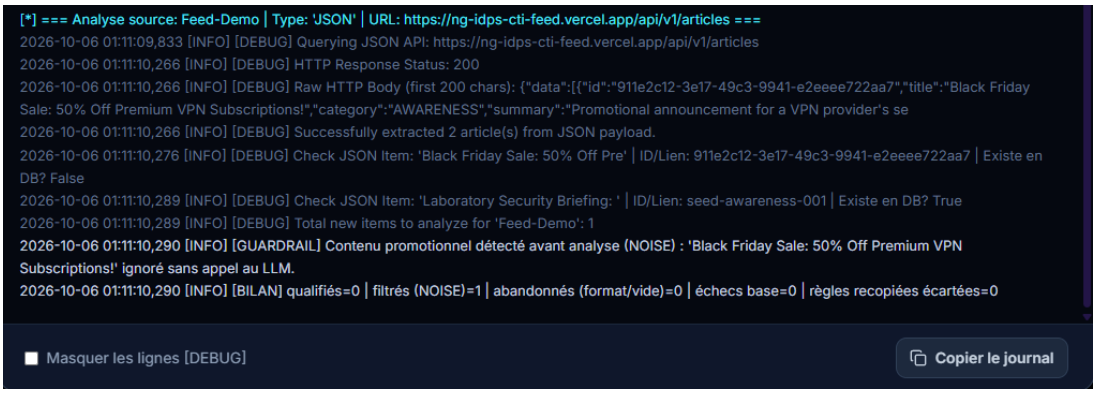


FIGURE 4.8 – Contenu publicitaire

La figure 4.9 illustre la déduplication des articles : chaque entrée du flux RSS est comparée à la base avant analyse, et les deux articles déjà présents (`Existe en DB? True`) sont ignorés sans appel au modèle de langage.



FIGURE 4.9 – Article déjà présent en base

4.2.6 Conclusion du sprint 4

Ce sprint a livré un agent CTI autonome et local, capable de trier automatiquement le renseignement OSINT entre menace technique et contenu de sensibilisation, tout en neutralisant les risques d'injection de contenu de faible qualité. Le sprint suivant exploite précisément la branche « sensibilisation » de ce triage pour s'attaquer à un autre maillon de la chaîne de sécurité : le facteur humain.

4.3 Sprint 5 : Mitigation du risque humain (agent Gemini et RAG LMS)

Le facteur humain demeure une cause majeure de compromission : le rapport DBIR de Verizon attribue à l'élément humain non malveillant environ 68 % des violations analysées. Ce sprint génère donc des parcours de sensibilisation personnalisés à partir des contenus classés `AWARENESS` par l'agent CTI du sprint 4.

4.3.1 Backlog du sprint 5

Le tableau 4.5 liste les user stories du sprint.

TABLE 4.5 – Backlog du sprint 5.

ID	User story	Critère d'acceptation	Priorité
US5.1	En tant que responsable sécurité, je veux qu'un module de cours complet (titre, diagramme Mermaid, 3 chapitres, QCM de 5 à 7 questions) soit généré via l'API `gemini-3.6-flash` à partir d'un article `AWARENESS`.	Module conforme au format JSON attendu, enregistré dans la collection `ctiactualities` (statut `FORMATION_GENERATED`).	Haute
US5.2	En tant que responsable sécurité, je veux diffuser la formation par e-mail (Nodemailer) à un département ciblé ou à l'ensemble des employés, listés dans `employes.json`.	E-mail personnalisé reçu, contenant l'identifiant employé et le lien vers le portail.	Haute
US5.3	En tant qu'employé, je veux m'authentifier sur le portail LMS par mon identifiant employé et passer le QCM de validation des acquis.	Connexion réussie par `id_employe` ; note calculée et affichée.	Moyenne
US5.4	En tant qu'auditeur, je veux que chaque tentative soit enregistrée dans la collection MongoDB `FormationRecord` (dernier score conservé).	Un enregistrement par couple (employé, cours), mis à jour à chaque nouvelle tentative.	Haute

ID	User story	Critère d'acceptation	Priorité
US5.5	En tant qu'administrateur, je veux consulter la liste des résultats de formation, avec un accès protégé par authentification.	Route protégée par jeton JWT et rôle RSSI.	Moyenne

4.3.2 Analyse Fonctionnelle

Le cas d'utilisation principal de ce Sprint est *Générer et Diffuser un Parcours de Sensibilisation*, où un article `AWARENESS` déclenche, sur décision de l'administrateur, la génération d'un module de formation par l'API Gemini, sa diffusion ciblée aux employés et le suivi de leur conformité via le QCM de validation. Le tableau 4.6 en spécifie la description textuelle détaillée.

TABLE 4.6 – Description textuelle du cas d'utilisation « Générer et diffuser un parcours de sensibilisation ».

Cas d'utilisation	Générer et diffuser un parcours de sensibilisation
Acteurs	Administrateur (principal), service Gemini, employé
Pré-conditions	Article `AWARENESS` présent en base ; clé API Gemini configurée ; annuaire `employes.json` disponible.
Scénario nominal	1. L'administrateur déclenche la génération depuis le dashboard pour un article donné. 2. Le backend construit un prompt contraint (titre, résumé, contraintes de format JSON strict) et appelle `gemini-3.6-flash`. 3. Le module généré (cours, diagramme Mermaid, QCM) est validé puis enregistré ; le statut de l'article passe à `FORMATION_GENERATED`. 4. L'administrateur choisit la cible de diffusion (un département ou l'ensemble des employés). 5. Un e-mail personnalisé est envoyé à chaque destinataire, contenant son identifiant employé et un lien vers le portail LMS. 6. L'employé s'authentifie par son identifiant, suit le module et passe le QCM. 7. Le score est soumis et enregistré dans `FormationRecord` (mise à jour si nouvelle tentative).
Scénarios alternatifs	(A1) Réponse Gemini non conforme au JSON attendu : échec renvoyé à l'administrateur, aucune persistance. (A2) Identifiant employé inconnu au portail : accès refusé (401). (A3) Aucun destinataire trouvé pour la cible choisie : diffusion annulée.

Suite à la page suivante...

Post-conditions	Un enregistrement `FormationRecord` est créé ou mis à jour pour chaque tentative de l'employé.

4.3.3 Conception

Cette section présente l'architecture, la dynamique et le modèle de données du sprint.

4.3.3.1 Architecture du sprint

La figure 4.10 montre comment un article `AWARENESS` déclenche, sur décision de l'administrateur, la génération d'un parcours de formation par l'API Gemini, sa diffusion ciblée et son suivi de conformité.

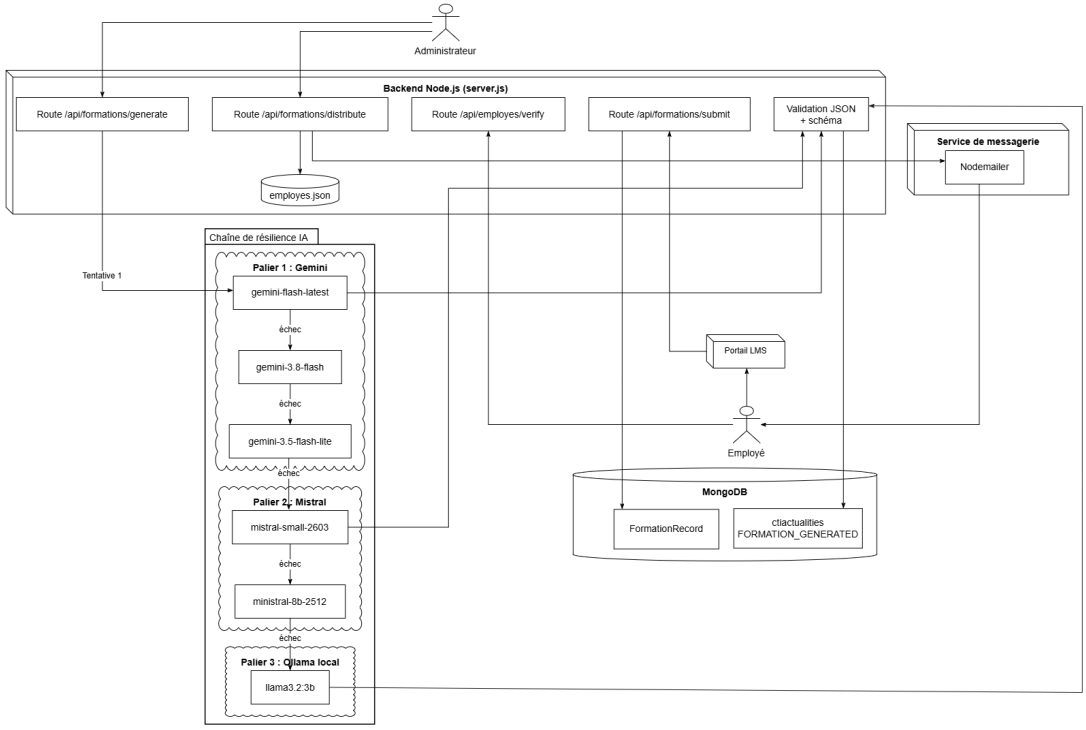


FIGURE 4.10 – Architecture du sprint 5

Contrairement à l'agent CTI du sprint 4, ce composant s'exécute côté backend Node.js et fait appel à un modèle LLM externe hébergé dans le nuage [89] ; seuls le titre et le résumé de la menace, déjà vulgarisés par le triage local, lui sont transmis, sans détail d'infrastructure sensible.

4.3.3.2 Diagramme de séquence

Le diagramme de la figure 4.11 détaille les échanges entre le backend, l'API Gemini, le service de messagerie et MongoDB.

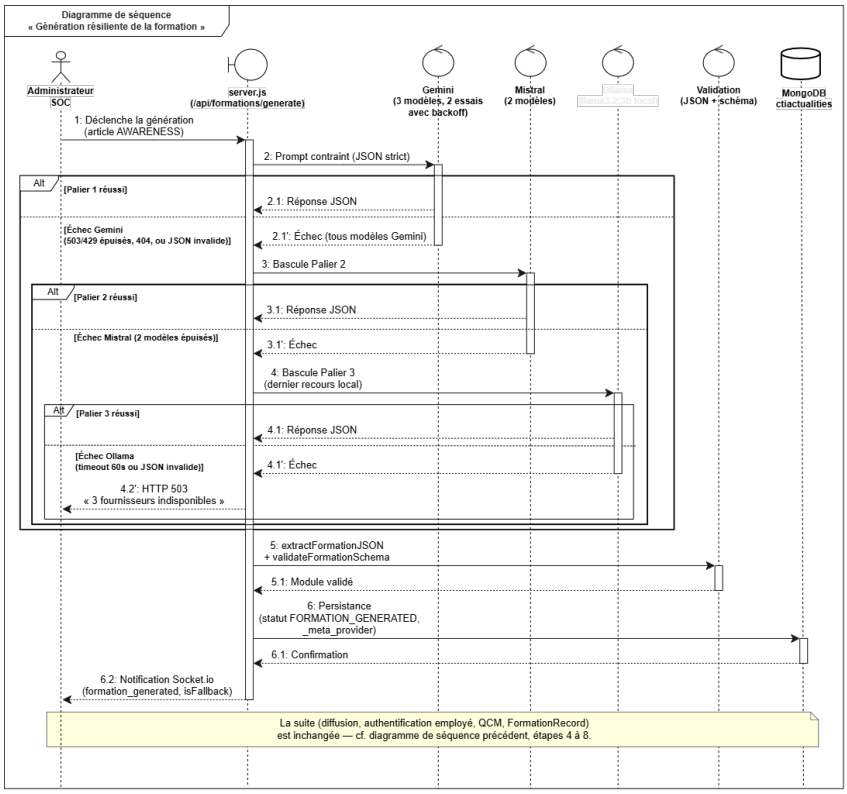


FIGURE 4.11 – Diagramme de séquence « Génération résiliente de la formation »

4.3.4 Réalisation

Cette section présente l'implémentation et les interfaces du LMS.

4.3.4.1 Composants

La génération du module est implémentée côté backend (`server.js`, route /api/formations/generate) via la librairie unifiée `@google/genai`. La diffusion (`/api/formations/distribute`) est protégée par le middleware d'authentification JWT et lit l'annuaire structuré par département (`employes.json`). L'authentification employé (`/api/employes/verify`) et la soumission de score (`/api/formations/submit`) sont exposées sans jeton, l'identifiant employé faisant office de clé d'accès au portail.

4.3.4.2 Interfaces et résultats

Les captures suivantes illustrent la chaîne complète.

La figure 4.12 illustre le régime nominal de la cascade : Gemini (« Cloud – Primaire ») répond directement, sans bandeau de repli, et le module généré reste protégé par le même mécanisme d'accès restreint par identifiant personnel.

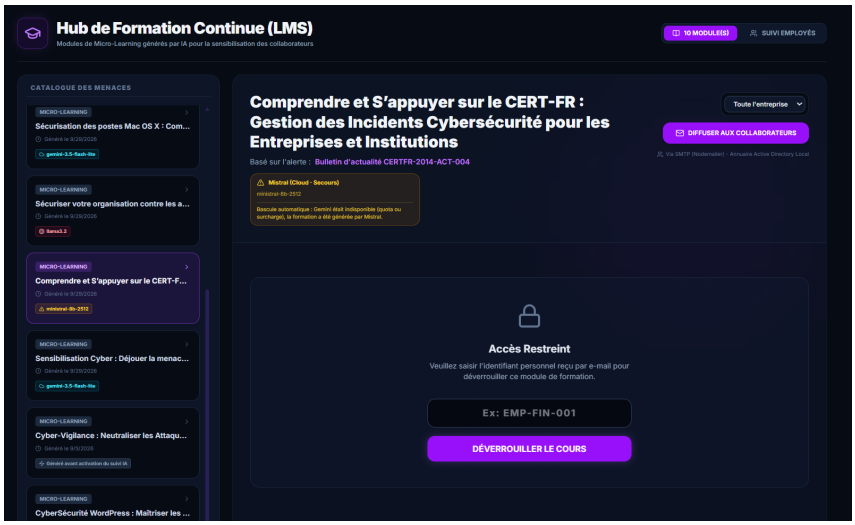


FIGURE 4.12 – Module de cours généré par Gemini

La figure 4.13 illustre le Hub de Formation Continue et sa cascade de résilience : Gemini étant indisponible au moment de la génération, la bascule automatique vers Mistral est tracée dans un bandeau dédié, et le module reste accessible aux collaborateurs par un identifiant personnel transmis par e-mail.

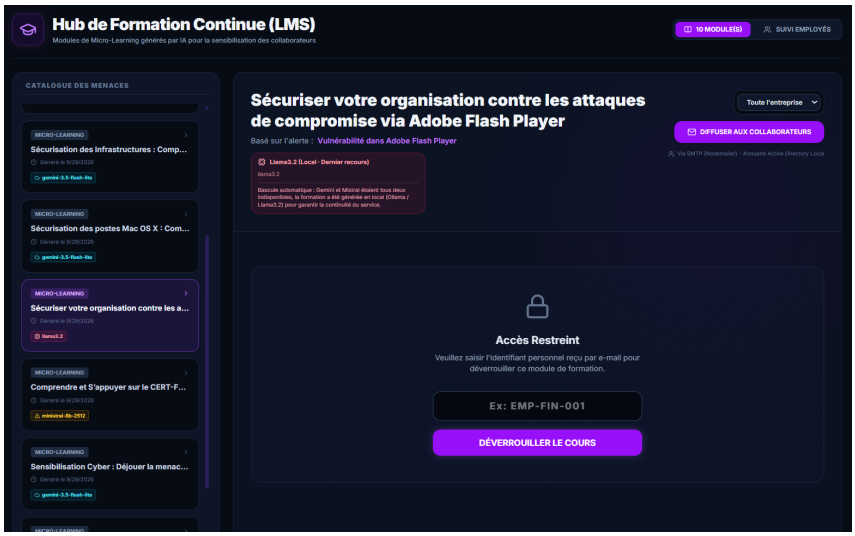


FIGURE 4.13 – Module de cours généré par Mistral

La figure 4.14 documente le troisième palier de la cascade : Gemini et Mistral étant tous deux indisponibles, le bandeau signale la bascule vers Llama 3.2 exécuté localement, qui garantit la continuité du service même sans accès aux API distantes.

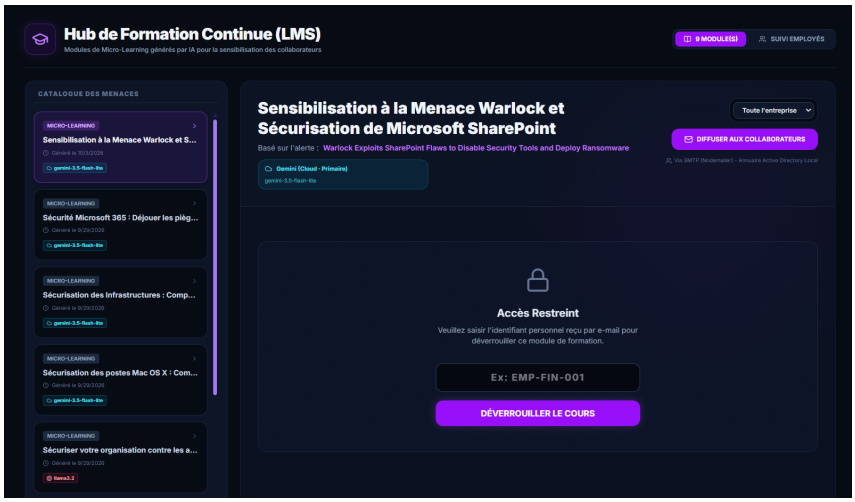


FIGURE 4.14 – Module de cours généré par Ollama Local

La figure 4.15 montre l'e-mail reçu par le collaborateur ciblé : il rappelle la politique Zero Trust ayant déclenché l'assignation, fournit l'identifiant de connexion personnel et pointe directement vers le portail LMS via un lien d'accès.

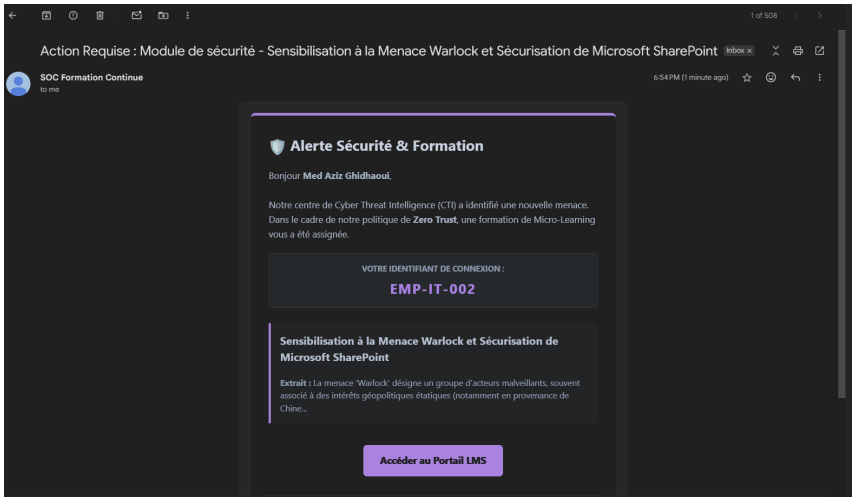


FIGURE 4.15 – E-mail de notification personnalisé envoyé à l'employé

La figure 4.16 montre le point d'entrée du portail LMS pour le module « Sensibilisation à la Menace Warlock » : l'accès au contenu pédagogique reste verrouillé tant que le collaborateur n'a pas saisi l'identifiant personnel reçu par e-mail, garantissant que seul le destinataire ciblé par la campagne peut déverrouiller le cours.

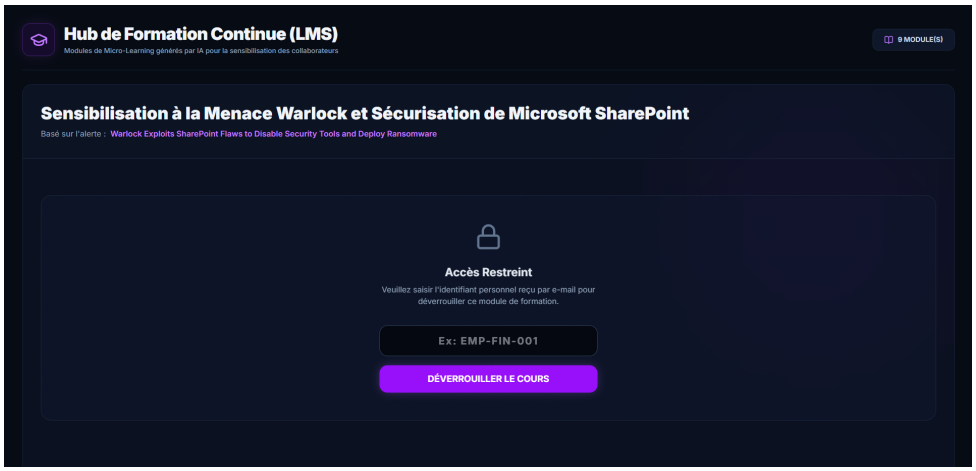


FIGURE 4.16 – Portail LMS : connexion par identifiant employé et QCM

La figure 4.17 présente le registre de conformité consulté depuis le Hub de Formation : chaque validation de QCM y est journalisée avec l'employé concerné, le module suivi, le score obtenu et la date, avec un filtrage possible par département ou par recherche libre.

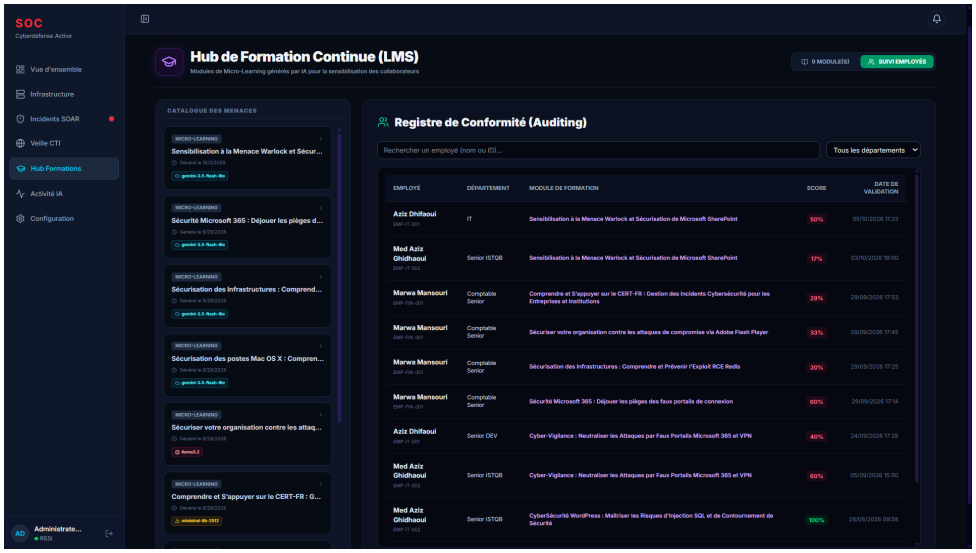


FIGURE 4.17 – Enregistrement du score dans la collection `FormationRecord`

4.3.5 Validation

Le tableau 4.7 présente les scénarios de test du sprint.

TABLE 4.7 – Plan de validation du sprint 5.

Test	Scénario	Résultat attendu	Résultat observé
T5.1	Génération sur un article `AWARENESS` réel	Module conforme (3 chapitres, 5 à 7 questions)	Figure 4.18

Test	Scénario	Résultat attendu	Résultat observé
T5.2	Diffusion ciblée à un département	E-mails reçus par les employés du département uniquement	Figure 4.19
T5.3	Connexion avec un identifiant inconnu	Accès refusé (401)	Figure 4.20
T5.4	Connexion avec un identifiant n'est pas autorisé	Accès non autorisé	Figure 4.21

La figure 4.18 montre le point de départ de la chaîne pédagogique : pour un article classé `AWARENESS` (ici une campagne de phishing visant les identifiants Microsoft 365 et Okta), l'analyste déclenche la génération du module de sensibilisation d'un simple clic, ce qui amorce la cascade de modèles présentée aux figures 4.12 à 4.14.



FIGURE 4.18 – Déclenchement de la génération d'un module sur un article `AWARENESS`

La figure 4.19 montre l'étape de diffusion : une fois le module déverrouillé, l'analyste sélectionne le département visé, ici IT, et le bouton « Campagne diffusée » confirme l'envoi des e-mails personnalisés (via Nodemailer) aux seuls collaborateurs du périmètre ciblé.

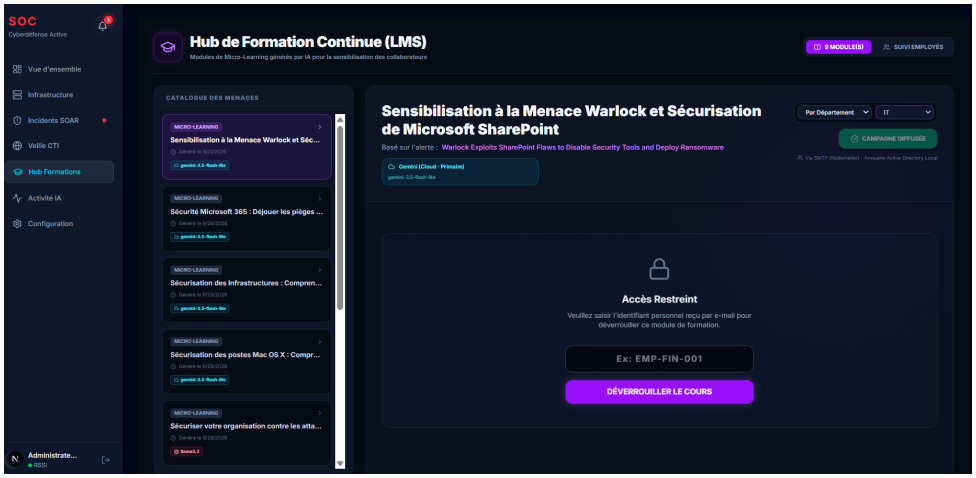


FIGURE 4.19 – Diffusion du formation ciblée

La figure 4.20 illustre le rejet d'une tentative de connexion avec un identifiant employé inexistant : une notification « Échec de connexion – Identifiant introuvable ou invalide » s'affiche.

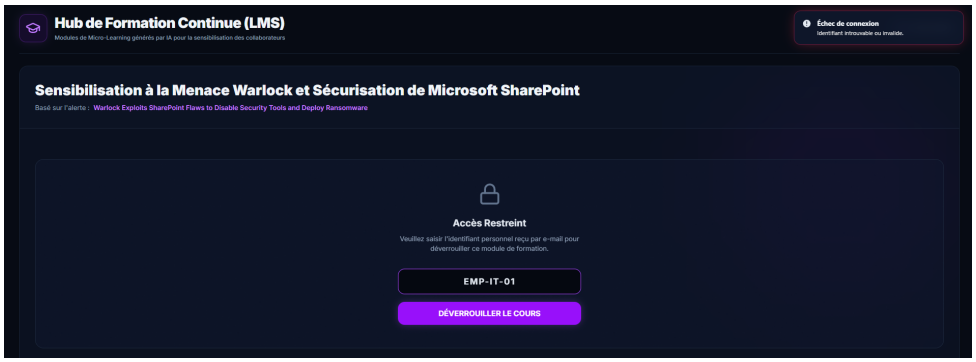


FIGURE 4.20 – Connexion avec ID inconnu

La figure 4.21 illustre le second niveau de contrôle d'éligibilité : un identifiant valide, appartenant à un collaborateur dont le département n'est pas ciblé par la campagne de diffusion, est distingué d'un identifiant simplement inexistant par un message explicite, « Accès non autorisé – Identifiant valide, mais vous n'êtes pas éligible à ce module de formation ».

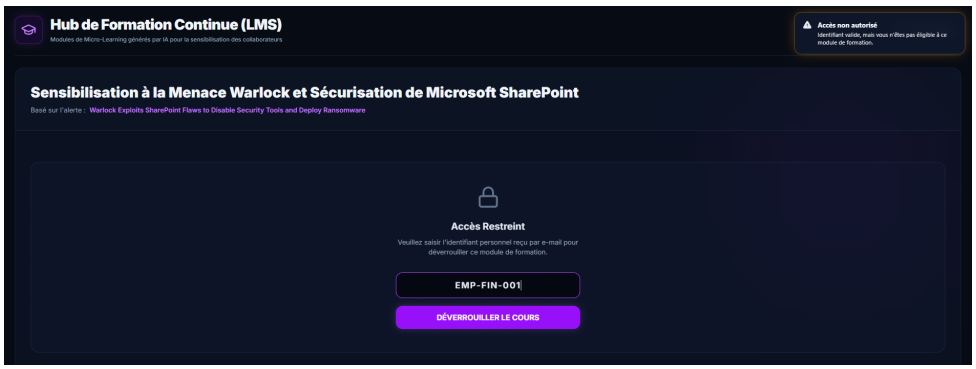


FIGURE 4.21 – Connexion avec ID non autorisé

4.3.6 Conclusion du sprint 5

Ce sprint a livré une plateforme de sensibilisation automatisée, alimentée par le triage CTI du sprint 4 et traçable par enregistrement individuel des résultats. Le dernier sprint fédère l'ensemble des briques de détection et de renseignement dans un dashboard SOC et ferme la boucle de réponse.

4.4 Sprint 6 : Orchestration SOAR multi-paliers, agent post-attaque et dashboard SOC

Le dernier sprint centralise le pilotage du SOC, analyse les incidents a posteriori et automatise la neutralisation des menaces, conformément aux principes de gestion d'incident du NIST [90]. Il constitue l'aboutissement de la chaîne détection–analyse–réponse et révèle une distinction architecturale importante : la corrélation et la décision autonome (`soar.py`) sont séparées de l'exécution supervisée par un humain (`agent_soar.py`).

4.4.1 Backlog du sprint 6

Le tableau 4.8 liste les user stories retenues.

TABLE 4.8 – Backlog du sprint 6.

ID	User story	Critère d'acceptation	Priorité
US6.1	En tant qu'analyste, je veux un dashboard temps réel (Next.js 14 / Node.js / Socket.IO) affichant les alertes et l'état des composants sans rechargement de page.	Alerte visible en moins de 2 s après l'émission Kafka.	Haute
US6.2	En tant qu'analyste, je veux que le SOAR corrèle les alertes ML et Suricata sur une fenêtre de 30 s afin de distinguer une menace confirmée d'une alerte isolée.	Une corrélation ML+Suricata sur la même IP déclenche le palier le plus élevé.	Haute
US6.3	En tant qu'analyste, je veux un rapport automatique de l'incident (score de confiance, technique MITRE ATTCK, recommandations) généré par `agent_ia.py`.	Rapport JSON structuré publié sur `incident-reports`.	Haute
US6.4	En tant qu'analyste, je veux un score composite de menace agrégeant Suricata, ML et LLM.	Score calculé pour chaque incident qualifié et affiché sur le dashboard.	Haute
US6.5	En tant qu'organisation, je veux un mode auto-pilote (mode nuit) bloquant automatiquement l'IP source lorsqu'aucun administrateur n'est connecté et que le score composite est $\geq 85\,\%$.	Blocage sans délai en l'absence d'analyste, notification par e-mail.	Haute

ID	User story	Critère d'acceptation	Priorité
US6.6	En tant qu'administrateur, je veux pouvoir déclencher ou ignorer manuellement un blocage depuis le dashboard lorsque je suis connecté.	Commande relayée à l'agent d'exécution et accusé de réception affiché.	Haute
US6.7	En tant qu'organisation, je veux qu'une IP de la liste blanche ne soit jamais bloquée automatiquement, même en cas d'alerte (politique Zero Trust *fail-open*).	Aucun blocage sur une IP whitelistée ; alerte tracée et signalée comme telle.	Haute
US6.8	En tant qu'administrateur, je veux un bouclier anti-DDoS déclenchable à distance (SYN cookies et limitation de débit) en cas de saturation volumétrique.	Règles appliquées sur `VM-Edge` en moins de 10 s par IP cible.	Moyenne
US6.9	En tant qu'administrateur, je veux un watchdog contrôlant la santé des machines virtuelles toutes les 15 secondes.	Panne détectée et statut `Hors Ligne` propagé au dashboard.	Moyenne

4.4.2 Analyse Fonctionnelle

Le cas d'utilisation principal de ce Sprint est *Neutraliser une Menace Corrélée*, où l'orchestrateur `soar.py` reçoit les alertes ML et Suricata, applique la politique Zero Trust (liste blanche), corrèle les deux sources sur une fenêtre temporelle de 30 s et décide d'un blocage autonome ou d'une simple notification, avant que l'agent IA post-attaque ne produise le rapport structuré affiché sur le dashboard.

Le tableau 4.9 décrit le cas « Neutraliser une menace corrélée » (chemin autonome, `soar.py`)

TABLE 4.9 – Description textuelle du cas d'utilisation « Neutraliser une menace corrélée »

Cas d'utilisation	Neutraliser une menace corrélée
Acteurs	`soar.py` (principal), `agent_ia.py`, backend (secondaire)
Pré-conditions	• Connexion SSH par clés vers `VM-Edge` établie. • Bus d'événements Kafka opérationnel. • Liste blanche chargée en mémoire.

Scénario nominal	1. `soar.py` reçoit une alerte ML ou Suricata et purge le cache d'IP en cours d'analyse (TTL de 300 s). 2. Si l'IP source figure dans la liste blanche, une alerte de contournement Zero Trust est publiée directement sur `incident-reports`, sans passer par l'IA (scénario A1). 3. Sinon, `soar.py` vérifie la corrélation temporelle (fenêtre de 30 s) entre une alerte ML et une alerte Suricata sur la même IP. 4. En cas de corrélation, ou de signature Suricata marquée `[CRITIQUE]`, l'IP est bloquée directement par SSH (`iptables DROP`) et l'agent IA est notifié via `alerts-for-llm` avec `auto_blocked=true`. 5. En l'absence de corrélation, seule une notification à l'agent IA est envoyée (`auto_blocked=false`), sans blocage. 6. `agent_ia.py` produit le rapport structuré et le publie sur `incident-reports`. 7. Le backend calcule le score composite, met à jour l'incident et diffuse `soar_incident_updated`.
Scénarios alternatifs	(A1) **IP en liste blanche :** bypass direct, blocage refusé, alerte « mouvement latéral » tracée. (A2) **Ollama indisponible :** rapport de secours (gabarit JSON) publié par `agent_ia.py`. (A3) **Alerte répétée :** si une alerte de la même catégorie a déjà été notifiée dans les 60 dernières secondes, la notification est supprimée (anti-saturation LLM).
Post-conditions	• L'IP est bloquée ou non, selon le palier de menace atteint. • L'incident est systématiquement tracé et rendu visible sur le dashboard.

4.4.3 Conception

Cette section détaille l'architecture globale, la dynamique de bout en bout, la logique de décision et le modèle de données.

4.4.3.1 Architecture du sprint

La figure 4.22 présente l'architecture complète : topics Kafka, agents SOAR, backend et dashboard. Le tableau 4.10 rappelle les topics Kafka effectivement mobilisés ; il convient de noter qu'aucun topic Kafka dédié n'est utilisé pour les commandes de l'administrateur, celles-ci transitant exclusivement par Socket.IO.

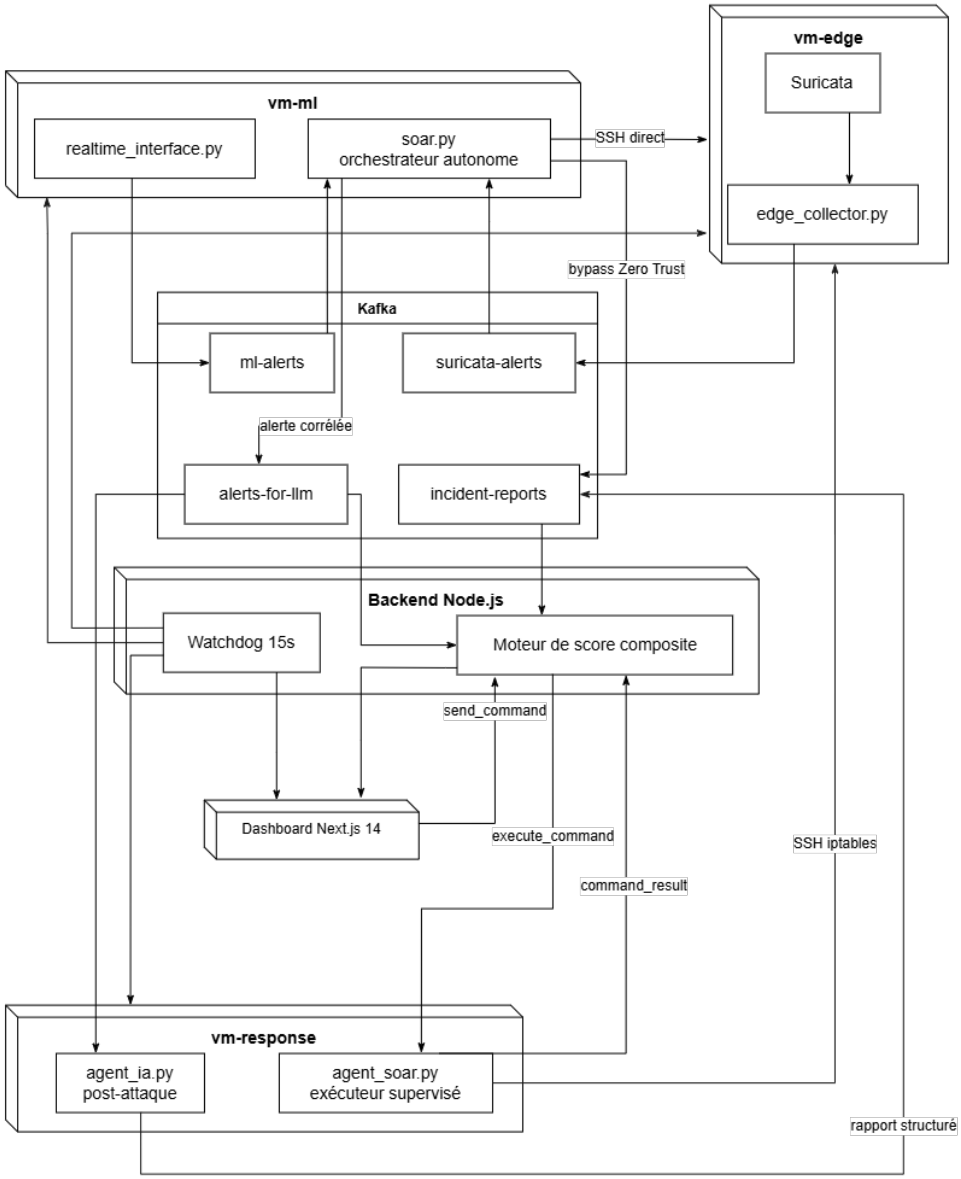


FIGURE 4.22 – Architecture du sprint 6

TABLE 4.10 – Topics Kafka mobilisés par le sprint 6.

Topic	**Producteur**	**Consommateur**	**Rôle**
`ml-alerts`	`realtime_-interface.py`	`soar.py`	Anomalie ML confirmée
`suricata-alerts`	Sonde Suricata (via Filebeat)	`soar.py`	Alerte par signature
`alerts-for-llm`	`soar.py`	`agent_ia.py`, `server.js`	Alerte corrélée à analyser / à afficher

Suite à la page suivante...

Topic	Producteur	Consommateur	Rôle
`incident-reports`	`agent_ia.py`, `soar.py` (bypass Zero Trust)	`server.js`	Rapport structuré ou alerte de contournement

4.4.3.2 Deux moteurs de remédiation distincts

L'implémentation distingue deux composants portant des responsabilités différentes, résumés dans le tableau 4.11 : `soar.py`, hébergé sur `vm-ml`, corrèle les alertes et décide en autonomie complète (sans intervention humaine) ; `agent_soar.py`, hébergé sur `vm-response`, est un simple exécuteur qui reste en écoute Socket.IO et n'agit que sur ordre explicite du backend, qu'il émane d'un administrateur ou du mode auto-pilote.

TABLE 4.11 – Distinction entre `soar.py` (orchestrateur autonome) et `agent_soar.py` (exécuteur supervisé).

Critère	`soar.py` (vm-ml)	`agent_soar.py` (vm-response)
Déclencheur	Consommation directe des topics Kafka `ml-alerts`/`suricata-alerts`	Événement Socket.IO `execute_command` émis par `server.js`
Décision	Corrélation, seuils de sévérité, logique de vote	Aucune ; exécution de l'ordre reçu
Origine du déclenchement	Entièrement autonome (aucune supervision humaine)	Action d'un administrateur *ou* décision auto-pilote du backend
Action	Blocage SSH direct (`bloquer_ip`) ou notification de l'agent IA	Blocage SSH sur ordre, avec accusé `command_result`

4.4.3.3 Score composite de menace

Le niveau de risque affiché sur le dashboard agrège les trois sources de détection selon la pondération de l'équation (4.1), calculée côté backend (`server.js`) à réception du rapport de l'agent IA :

$$S_{\text{composite}} = 0{,}4 \cdot S_{\text{Suricata}} + 0{,}3 \cdot S_{\text{ML}} + 0{,}3 \cdot S_{\text{LLM}} \tag{4.1}$$

La pondération privilégie la détection par signature, dont la précision est la plus élevée, tout en intégrant l'écart comportemental mesuré par l'erreur de reconstruction du LSTMVAE [38, 37] et l'évaluation cognitive du LLM (`score_confiance`). C'est ce score composite, et non un seuil de sévérité brut, qui conditionne le déclenchement du mode auto-pilote.

4.4.3.4 Notification en deux temps

Afin de ne pas exposer l'administrateur à un délai perçu de latence pendant la génération du rapport IA (de l'ordre de quelques secondes à une trentaine de secondes selon la charge), le backend notifie le dashboard *avant* que l'analyse cognitive ne soit terminée : un incident provisoire est créé et diffusé (`soar_incident_incoming`) dès réception sur `alerts-for-llm`, avec un résumé d'attente ; il est ensuite complété et rediffusé (`soar_incident_updated`) une fois le rapport structuré disponible sur `incident-reports`. Ce choix privilégie la réactivité perçue sur l'exhaustivité immédiate du diagnostic, cohérent avec l'objectif de défense 24/7 à faible latence.

4.4.3.5 Diagrammes de séquence

La figure 4.23 décrit le flux de la détection au dashboard, et la figure 4.24 la logique de décision entre les paliers autonome et supervisé.

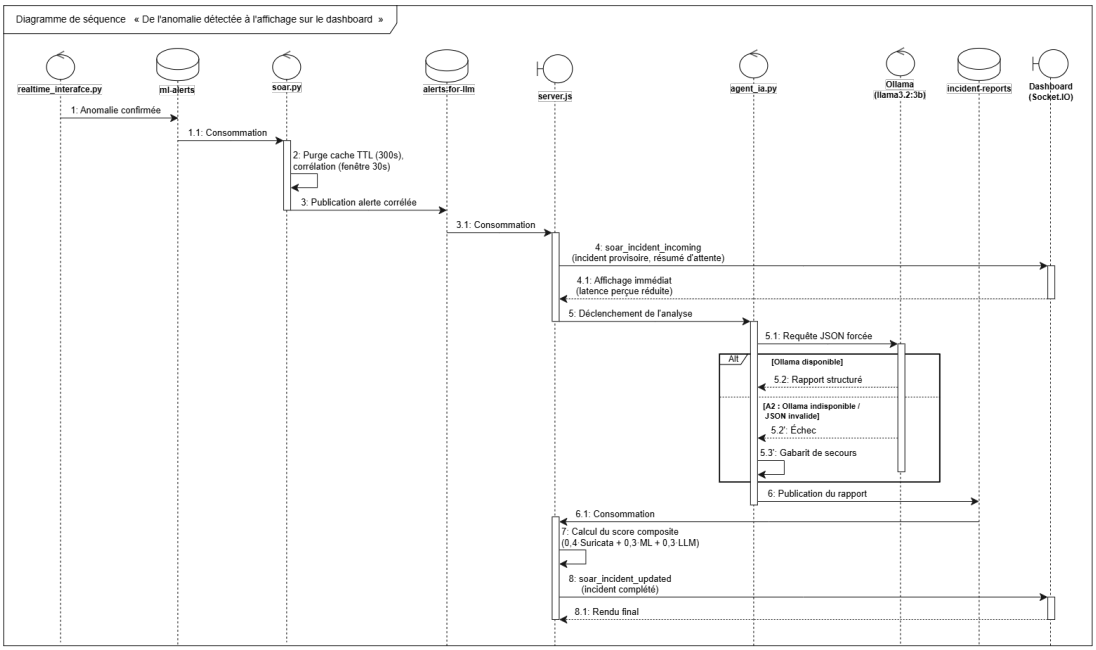


FIGURE 4.23 – Diagramme de séquence « de l'anomalie détectée à l'affichage sur le dashboard »

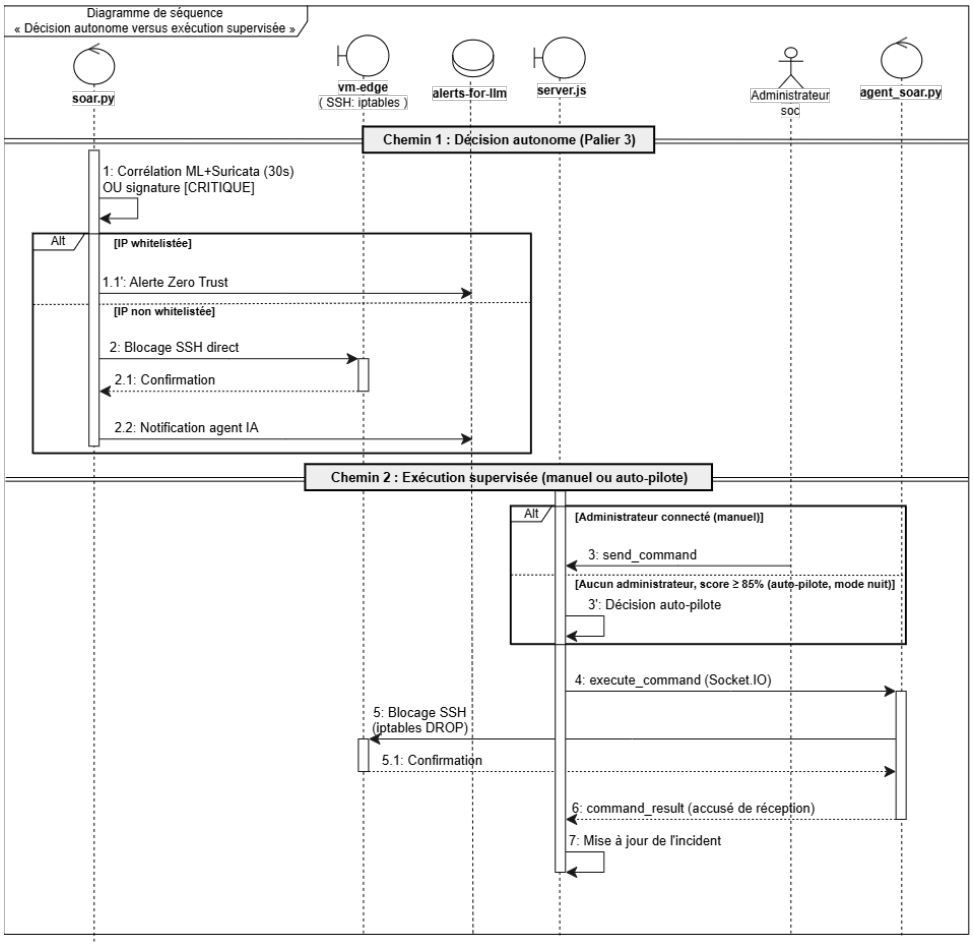


FIGURE 4.24 – Diagramme de séquence « décision autonome versus exécution supervisée »

4.4.3.6 Diagramme d'états de la réponse

Le cycle de vie d'un incident, de sa détection à sa clôture, est modélisé par la figure 4.25.

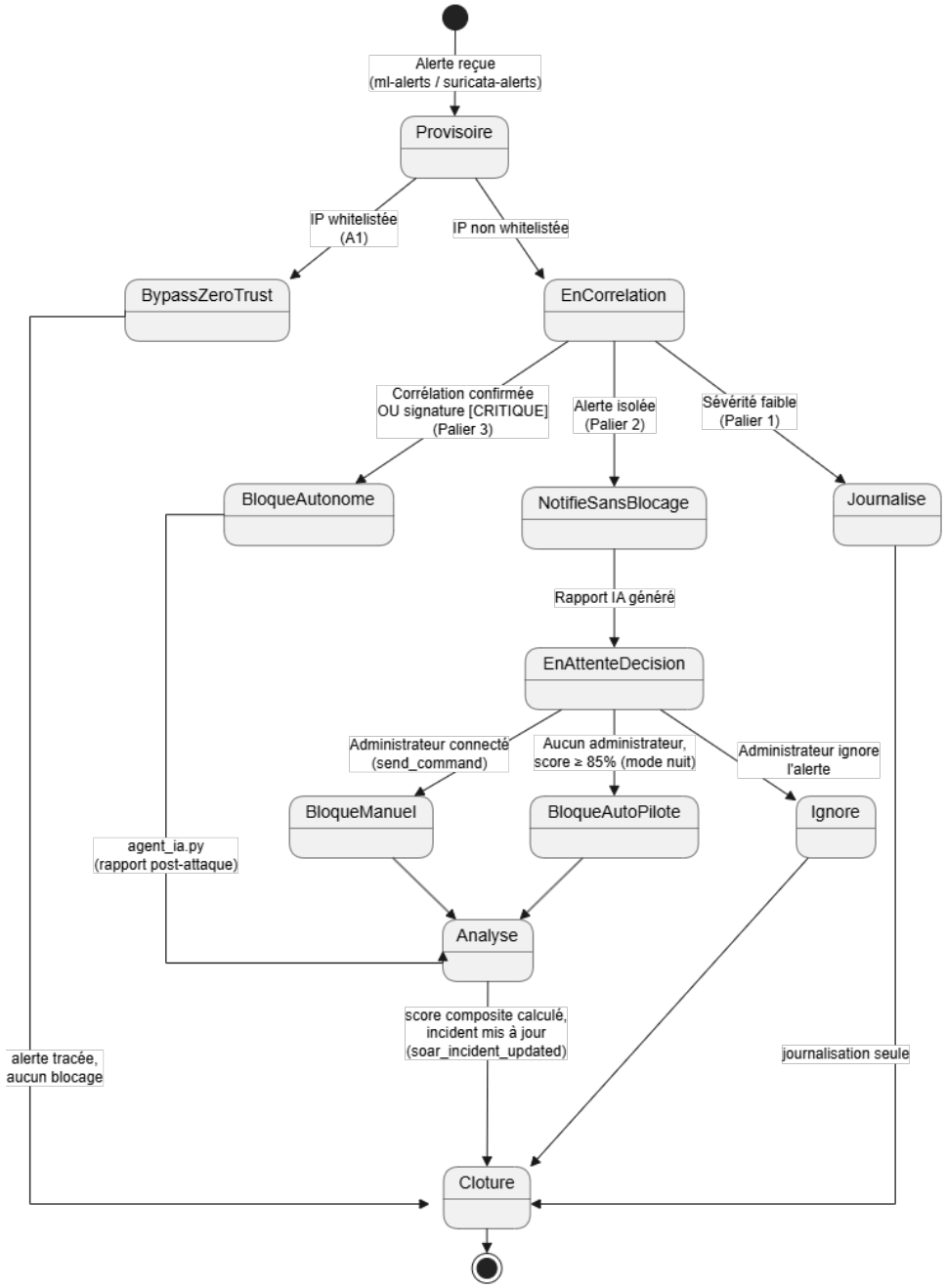


FIGURE 4.25 – Diagramme d'états d'un incident SOAR

4.4.4 Réalisation

Cette section présente l'implémentation du moteur de corrélation, de l'agent post-attaque, du dashboard et des mécanismes de résilience.

4.4.4.1 Moteur de corrélation autonome (`soar.py`)

Le moteur de corrélation maintient en mémoire l'horodatage de la dernière alerte ML et Suricata par IP, ainsi qu'un cache des IP en cours d'analyse par l'agent IA, purgé par TTL (300 s) pour permettre une re-notification après expiration. Une boucle de reconnexion automatique

(nouvelle tentative toutes les 5 s) enveloppe la connexion Kafka afin qu'une exception transitoire n'interrompe pas durablement le service.

4.4.4.2 Agent IA post-attaque

L'agent `agent_ia.py` consomme `alerts-for-llm`, interroge Ollama (`llama3.2:3b`, sortie JSON forcée) et publie sur `incident-reports` [?, 91]. Le rapport respecte un format structuré (listing 4.1), les techniques étant référencées selon MITRE ATTCK [92]. En cas d'indisponibilité du LLM ou d'échec de parsing JSON, un gabarit de secours préserve la continuité du pipeline.

Listing 4.1 – Format de diagnostic structuré publié par l'agent IA (exemple illustratif).

```
{
  "titre_incident": "Balayage de ports suivi d'une tentative SSH",
  "resume_executif": "...",
  "niveau_severite": "Critique",
  "score_confiance": 92,
  "mitre_attack_technique": "T1021.001",
  "analyse_technique": "...",
  "recommandations_actions": [
    "Action immediate 1",
    "Action de remediation 2"
  ]
}
```

4.4.4.3 Politique Zero Trust et corrélation anti-faux-positif

Deux mécanismes distincts protègent les actifs critiques listés dans la liste blanche partagée (`actifs_critiques.json`) : d'une part, un bypass inconditionnel côté `soar.py`, qui refuse tout blocage automatique d'une IP whitelistée et publie directement une alerte « mouvement latéral » sans passer par l'agent IA (politique *fail-open*) ; d'autre part, un filtre de corrélation anti-faux-positif côté backend, qui ignore une alerte émise par un actif whitelisté si une attaque externe réelle a été détectée dans les 180 dernières secondes, cette anomalie étant alors considérée comme du bruit collatéral plutôt qu'un mouvement latéral avéré.

4.4.4.4 Bouclier anti-DDoS

En cas de détection volumétrique, `soar.py` peut déclencher à distance, par SSH, l'activation de *SYN cookies* et d'une règle de limitation de débit sur `VM-Edge`, puis notifie l'agent IA pour traçabilité de l'événement.

4.4.4.5 Mode auto-pilote et exécution supervisée

Le tableau 4.12 résume les trois niveaux de traitement internes à `soar.py`, complétés par le mode auto-pilote piloté depuis le backend. Le déclenchement sans délai du blocage autonome est

un choix de conception délibéré : en défense 24/7, tout délai d'attente expose le système à des attaques parallèles pendant la fenêtre d'inaction.

TABLE 4.12 – Paliers de traitement du SOAR et mode auto-pilote.

Palier	**Condition**	**Action**
1 – Journalisation	Alerte Suricata de sévérité faible (informationnelle)	Journalisée uniquement, aucune notification
2 – Notification	Alerte ML isolée, ou Suricata de sévérité ≤ 2	Notification de l'agent IA, aucun blocage
3 – Blocage autonome	Corrélation ML+Suricata (fenêtre 30 s), ou signature `[CRITIQUE]`	Blocage SSH immédiat par `soar.py` et notification de l'agent IA
Auto-pilote (mode nuit)	Aucun administrateur actif et score composite $\geq 85\,\%$	Commande `execute_command` relayée à `agent_soar.py` ; blocage sans délai
Manuel	Administrateur actif, décision explicite (`send_command`)	Commande relayée à `agent_soar.py` ; accusé `command_result`

4.4.4.6 Dashboard SOC, authentification et watchdog

Le dashboard (Next.js 14, TypeScript, Tailwind CSS) adopte un thème sombre de type *glassmorphism* ; il écoute les événements Socket.IO `soar_incident_incoming` et `soar_incident_updated`, et émet `send_command` lors des actions de l'administrateur [93]. L'accès aux routes d'administration sensibles (whitelist, résultats de formation, diffusion) est protégé par un jeton JWT valide 8 heures, vérifié par un middleware qui contrôle également le rôle RSSI de l'utilisateur. Le watchdog contrôle l'ensemble des nœuds enregistrés toutes les 15 secondes et propage un statut `Hors Ligne` en cas d'absence de battement de cœur.

4.4.4.7 Interfaces et résultats

Les captures suivantes présentent les principales interfaces du SOC.

La figure 4.26 présente la page d'authentification du tableau de bord SOC, point d'entrée unique de l'interface d'administration : l'accès aux vues d'ensemble, aux incidents, à la veille CTI et au Hub de formation est conditionné à la saisie d'un identifiant et d'un mot de passe, protégeant ainsi les fonctions de pilotage du système des utilisateurs non autorisés.

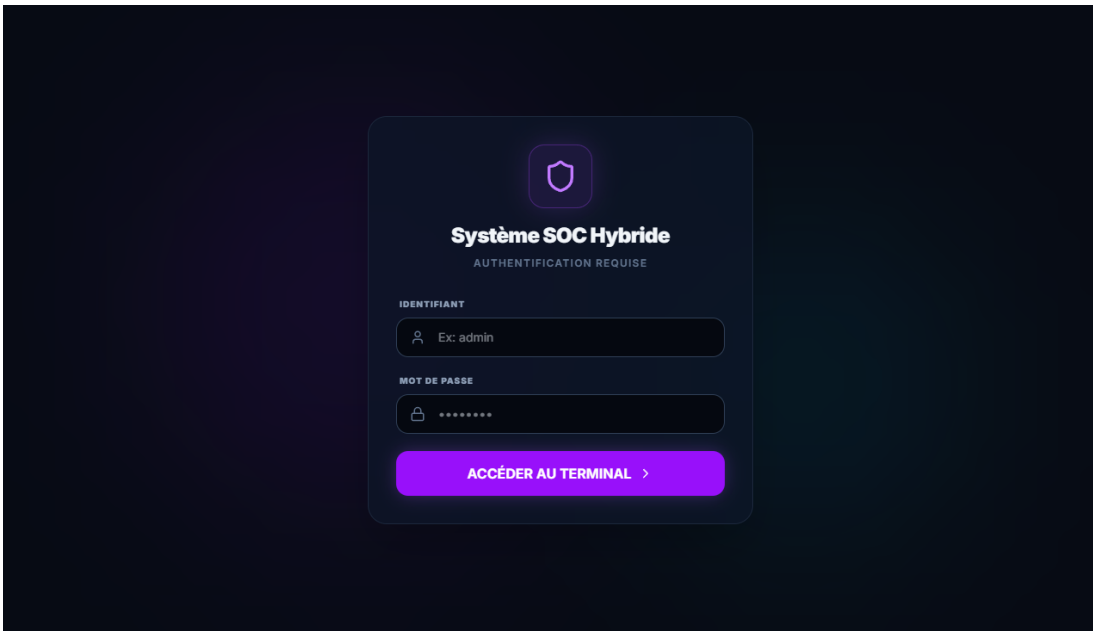


FIGURE 4.26 – Page `Login`

La figure 4.27 présente le rapport de relève affiché à la connexion de l'administrateur : il résume les événements survenus pendant la déconnexion (attaques bloquées, alertes critiques, nouveaux bulletins CTI) et détaille les incidents traités en mode autonome – ici trois attaques distinctes contre la même source, toutes bannies par le palier d'auto-pilote nocturne – avant que l'analyste n'acquitte le rapport pour démarrer sa session.

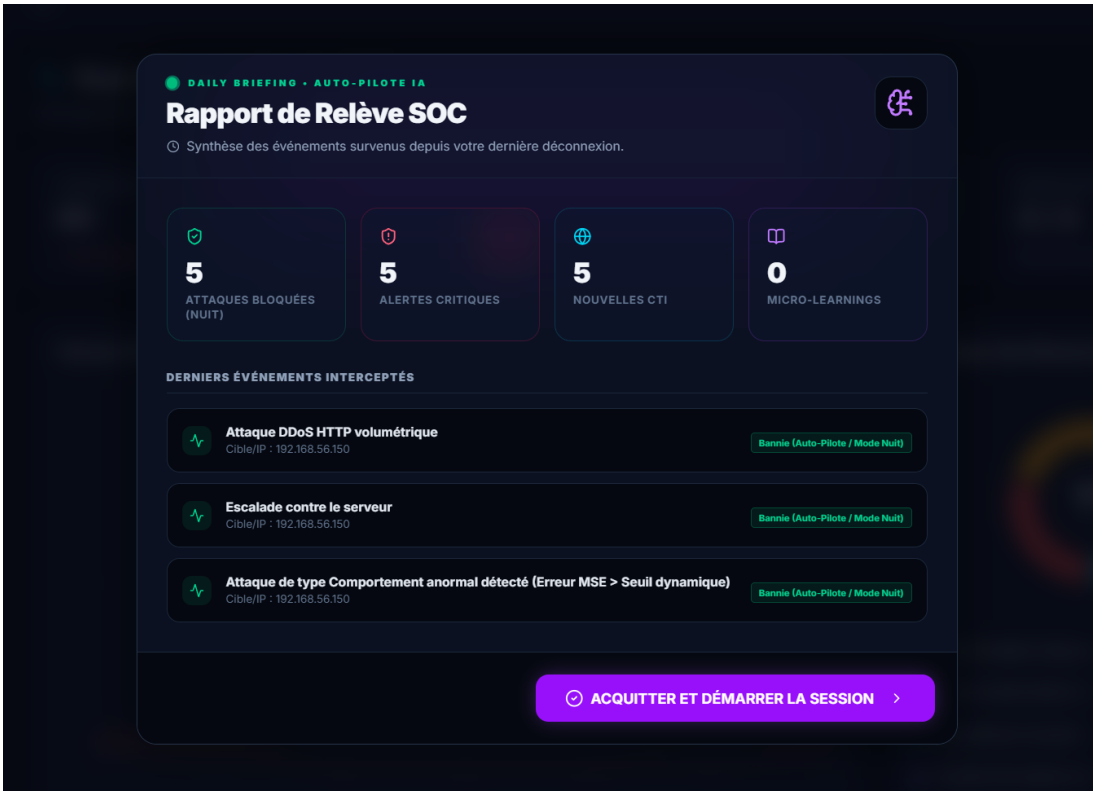


FIGURE 4.27 – Page `Rapport`

La figure 4.28 présente la page d'accueil du tableau de bord, qui agrège l'état du système en un coup d'œil : les indicateurs clés (incidents totaux, confiance moyenne du moteur de détection, taux de remédiation automatique, nœuds actifs), la répartition hebdomadaire des attaques et leur typologie, ainsi que trois panneaux de synthèse propres aux fonctions ajoutées en Release 2 – la répartition des générations par palier de la cascade de résilience IA, le nombre de bulletins de veille CTI en attente de validation, et le taux de réussite du parcours de sensibilisation.

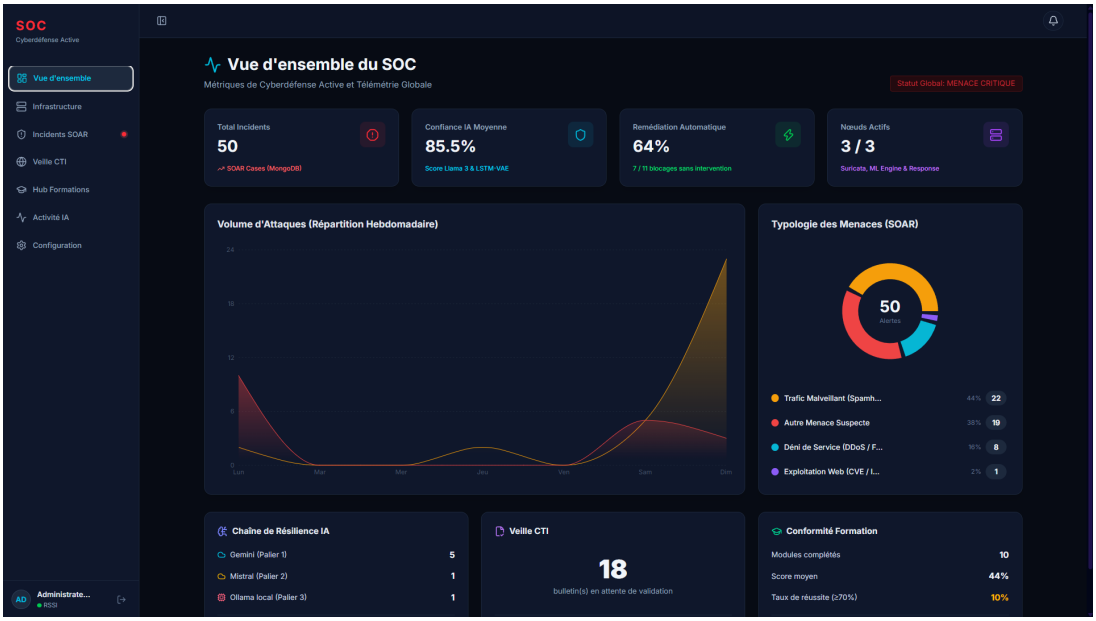


FIGURE 4.28 – Vue globale du dashboard SOC

La figure 4.29 présente le diagnostic complet d'un incident résolu en mode autonome : une attaque par usurpation d'adresses multiples (`DDoS Spoofing Multi-IP`), détectée par le bouclier anti-DDoS, est classée critique avec la technique MITRE ATT&CK correspondante (T1498), accompagnée d'un résumé exécutif, d'une analyse technique et d'un playbook de remédiation suggéré. Le bouton « Exporter l'Audit PDF » permet de générer le rapport correspondant pour archivage ou transmission.

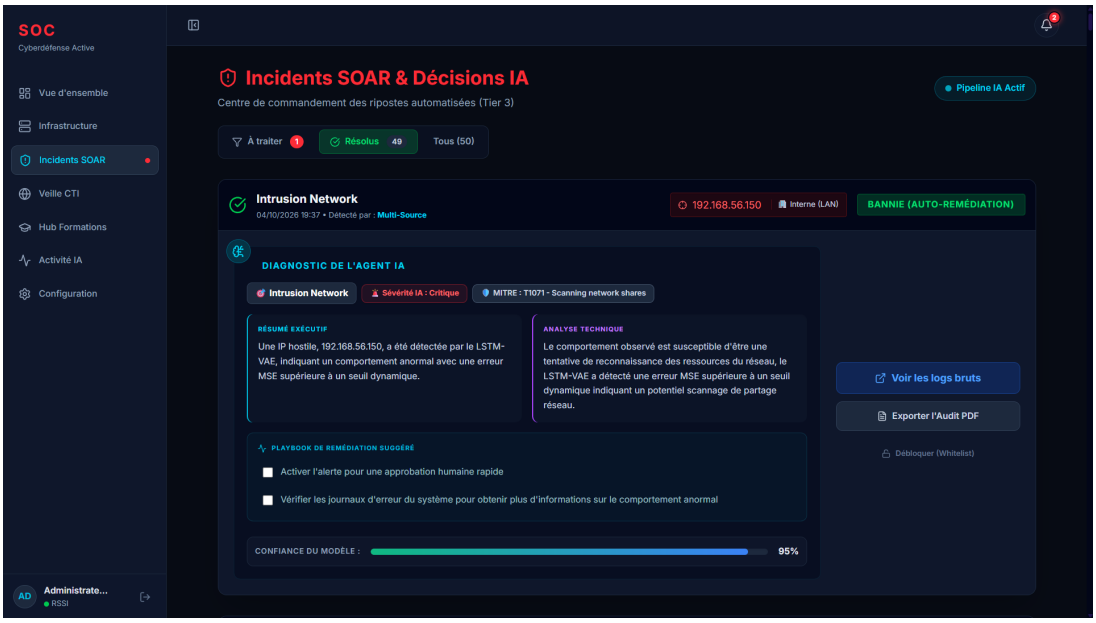


FIGURE 4.29 – Page `/incidents` et export PDF du rapport

La figure 4.30 présente la supervision de l'infrastructure : les trois nœuds (`VM-Capteur-Suricata`, `ML`, `VM-Response`) sont surveillés individuellement, avec leur consommation CPU et RAM ainsi que l'état de chacun de leurs services actifs, détecté soit par le statut `systemd`, soit par la présence du processus correspondant. Le watchdog marque un nœud hors ligne dès que son agent de télémétrie cesse d'émettre pendant plus de 15 secondes, ce qui alerte l'opérateur sans attendre une inspection manuelle.



FIGURE 4.30 – Page `/infrastructure` : état des machines virtuelles (watchdog)

La figure 4.31 illustre la voie supervisée de la réponse : après validation de l'analyste, le *bouclier SOAR humain* reçoit du tableau de bord un ordre de remédiation structuré (action, IP cible, capteur source, identifiant d'incident), puis exécute la sanction par SSH sur `VM-Edge` et

confirme le bannissement de l'adresse, sans qu'aucune règle `iptables` ne soit directement visible dans ce journal.

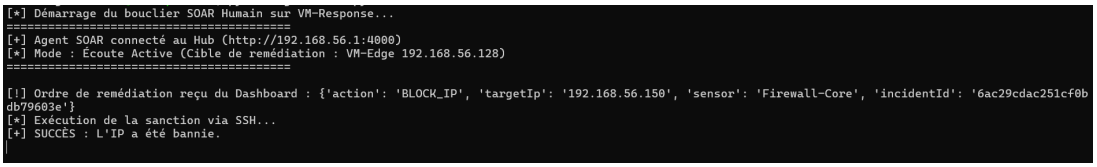


FIGURE 4.31 – Règle `iptables` appliquée par l'agent d'exécution sur la cible

4.4.5 Validation

Le tableau 4.13 présente les scénarios validant la boucle de réponse fermée. La route de simulation `/api/simulate-attack` a été utilisée pour injecter des incidents de test de bout en bout sans dépendre d'une attaque réelle.

TABLE 4.13 – Plan de validation du sprint 6.

Test	**Scénario**	**Résultat attendu**	**Résultat observé**
T6.1	Attaque simulée, administrateur connecté	Alerte temps réel, décision manuelle possible via `send_command`	Figure 4.32
T6.2	Attaque simulée, aucun administrateur connecté	E-mail envoyé et blocage immédiat si score $\geq 85\,\%$	Figure4.33
T6.3	Alertes ML et Suricata corrélées sur la même IP en moins de 30 s	Blocage autonome (palier 3), une seule règle iptables	Figure 4.34
T6.4	Coupure puis reprise de la connexion Kafka	Reconnexion automatique sans intervention manuelle	Figure 4.35
T6.5	Arrêt d'un nœud supervisé	Panne détectée par le watchdog en ≤ 15 s	Figure 4.36

La figure 4.32 illustre le scénario avec administrateur connecté : une attaque de force brute SSH, provenant de l'adresse `45.33.32.156` et détectée par Suricata (NIDS), déclenche une alerte système critique affichée en superposition sur la vue d'ensemble, offrant un accès direct au centre de commandement pour une décision manuelle.

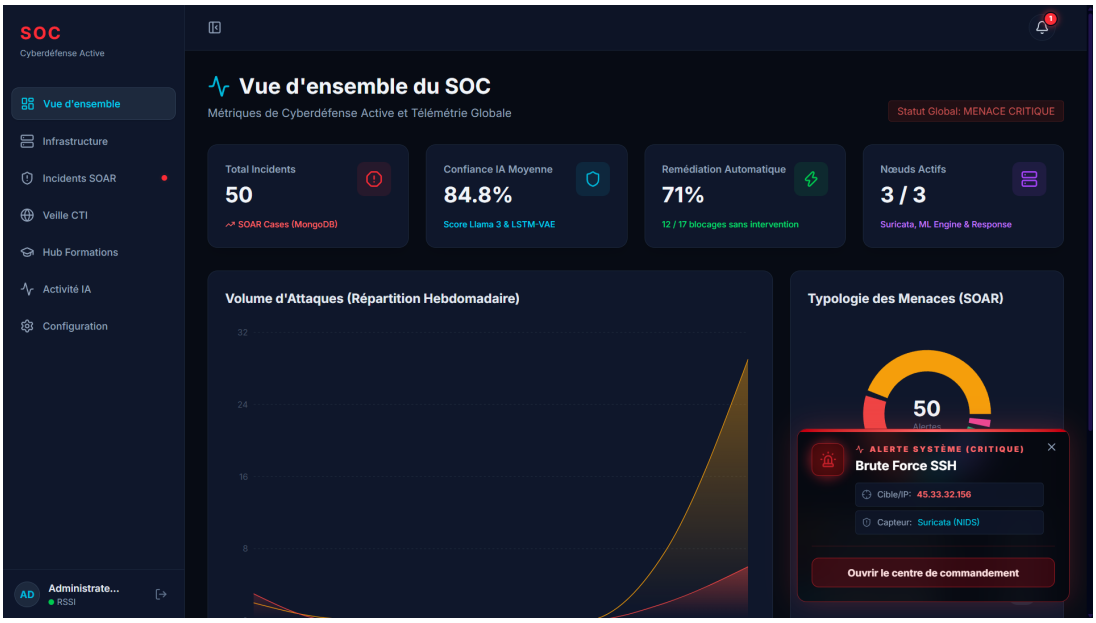


FIGURE 4.32 – Attaque simulée, administrateur connecté

La figure 4.33 illustre le scénario sans administrateur connecté : le journal du backend trace la déconnexion du dernier administrateur, la détection externe de l'IP `192.168.56.150`, puis le basculement en Mode Nuit dès qu'un score composite de 91 % dépasse le seuil d'auto-blocage, avec escalade par e-mail faute d'opérateur disponible. L'e-mail reçu confirme l'incident qualifié – IP malveillante, score composé, temps de génération du rapport et statut final « Bannie (Auto-Pilote / Mode Nuit) ».



FIGURE 4.33 – Attaque simulée, aucun administrateur connecté (Mode Nuit)

La figure 4.34 assemble les trois sources qui concourent à la décision de blocage autonome : les alertes Suricata sur l'IP ciblée, le diagnostic du LSTM-VAE (caractéristiques en violation, erreur de reconstruction, vote et persistance), et le journal du SOAR qui corrèle les deux premiers

paliers (sévérité Suricata, puis corrélation ML+Suricata en priorité haute) avant de déclencher le blocage physique sur `VM-Edge` et de transmettre le contexte à l'agent de diagnostic.

Suricata

```
2026-10-05 23:45:09,844 [INFO] [🚨 ALERTE] Attaque DDoS HTTP Volumétrique (src=192.168.56.150)
2026-10-05 23:45:10,353 [INFO] [🚨 ALERTE] ET INFO Spring Boot Actuator Health Check Request (src=192.168.56.150)
2026-10-05 23:45:10,457 [INFO] [🚨 ALERTE] SURICATA Applayer Detect protocol only one direction (src=192.168.56.150)
2026-10-05 23:45:10,457 [INFO] [🚨 ALERTE] SURICATA Applayer Detect protocol only one direction (src=192.168.56.150)
```

LSTM-VAE

```
2026-10-05 23:45:16,364 [INFO] [DIAG] IP=192.168.56.150 | Features en violation : ['Bwd Packet Length Min=3.3687(>P98=1.7334)', 'Min Packet Length=2.9827(>P98=1.2294)', 'SYN Flag Count=895
.5784(>P98=360.1122)', 'PSH Flag Count=562.1184(>P98=511.8539)', 'Fwd PSH Flags=895.7755(>P98=360.0901)', 'ACK Flag Count=428.5653(>P98=428.3677)', 'Bwd Packet Length Max=433.3007(>P98=10.
7327)']
2026-10-05 23:45:16,364 [INFO] IP=192.168.56.150 | MSE_global=203.4924 (Seuil=68.12) | Votes=7/16 features | paquets_2s=6 | Anomalies récentes=3/5 | latence=6.344ms
2026-10-05 23:45:16,364 [CRITICAL] *** AUTO-BLOCK DÉCLENCHÉ *** Isolation de l'IP malveillante : 192.168.56.150 | MSE=203.4924 (seuil=68.1181) | latence=6.344ms
2026-10-05 23:45:16,370 [INFO] Updated metadata: ClusterMetadata(brokers: 1, topics: 1, coordinators: 0)
2026-10-05 23:45:17,363 [INFO] [DIAG] IP=192.168.56.150 | Features en violation : ['Bwd Packet Length Min=3.6716(>P98=1.7334)', 'Min Packet Length=3.2925(>P98=1.2294)', 'SYN Flag Count=895
.5785(>P98=360.1122)', 'PSH Flag Count=562.1184(>P98=511.8539)', 'Fwd PSH Flags=895.7755(>P98=360.0901)', 'ACK Flag Count=471.9984(>P98=428.3677)', 'Bwd Packet Length Max=442.1634(>P98=10.
7327)']
```

SOAR

```
2026-10-05 23:45:15,524 [INFO] [PALIER 2] Alerte Suricata sévérité 2 pour IP=192.168.56.150
2026-10-05 23:45:16,382 [WARNING] [PALIER 3] Corrélation ML+Suricata pour IP=192.168.56.150 - priorité HAUTE
2026-10-05 23:45:17,468 [WARNING] IP BLOQUÉE physiquement sur VM-Edge : 192.168.56.150
2026-10-05 23:45:17,473 [INFO] Contexte pour l'IP 192.168.56.150 transmis à l'Agent IA (Auto-bloqué: True).
```

FIGURE 4.34 – Résultats des tests de validation du sprint 6

La figure 4.35 illustre la résilience face à une coupure du courtier Kafka : les tentatives de connexion échouent (`KafkaTimeoutError`), l'orchestrateur retente automatiquement sans interrompre son exécution, puis la connexion est rétablie et les métadonnées du cluster sont de nouveau synchronisées, sans qu'aucune intervention manuelle n'ait été nécessaire.

```
2026-10-06 02:21:23,815 [WARNING] Bootstrap bootstrap-0 not ready; waiting 21.05 secs
2026-10-06 02:21:44,882 [ERROR] Bootstrap failed: KafkaTimeoutError: Unable to bootstrap from 192.168.56.130:9092
2026-10-06 02:21:44,882 [ERROR] Erreur critique dans la boucle principale : KafkaTimeoutError: Unable to bootstrap from 192.168.56.130:9092. Nouvelle tentative de connexi
on dans 5s...
2026-10-06 02:21:49,883 [INFO] IO loop starting (client_id=kafka-python-3.0.8)
2026-10-06 02:21:49,884 [INFO] Attempting bootstrap to bootstrap-0 at 192.168.56.130:9092 (rack None)
2026-10-06 02:21:49,884 [INFO] Initializing connection for node_id bootstrap-0 at 192.168.56.130:9092 (rack=None)
2026-10-06 02:21:49,884 [INFO] <KafkaTCPTransport [192.168.56.130:9092<-52616]>: connected to <socket.socket fd=30, family=2, type=1, proto=6, laddr=('192.168.56.130', 52
616), raddr=('192.168.56.130', 9092)>
2026-10-06 02:21:49,890 [INFO] <KafkaConnection node_id=bootstrap-0 host=[192.168.56.130:9092<-52616] broker_version=unknown (initializing)>: Broker version identified as
 4.3
2026-10-06 02:21:49,891 [INFO] <KafkaConnection node_id=bootstrap-0 host=[192.168.56.130:9092<-52616] broker_version=(4, 3) (connected)>: Connected
2026-10-06 02:21:49,900 [INFO] Updated metadata: ClusterMetadata(brokers: 1, topics: 0, coordinators: 0)
```

FIGURE 4.35 – Coupure kafka et reprise

La figure 4.36 illustre la détection d'une panne de nœud par le watchdog : à l'arrêt de `VM-Response` (visible dans la fenêtre VMware Workstation en arrière-plan), la carte correspondante passe à l'état « Hors Ligne », ses indicateurs CPU et RAM retombent à 0 %, et ses services actifs (Agent SOAR, Agent Rapport) basculent au rouge, tandis que les deux autres nœuds restent opérationnels.

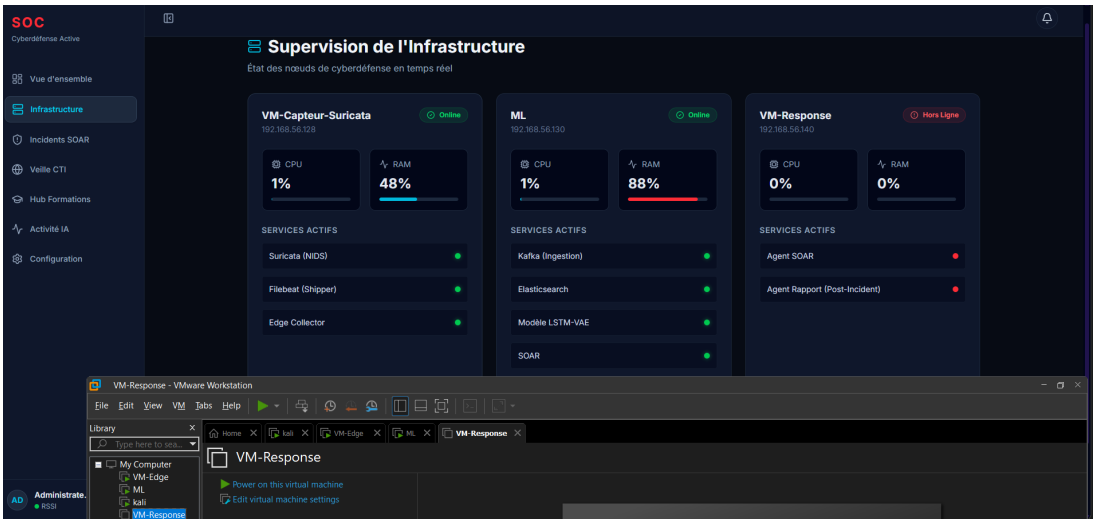


FIGURE 4.36 – Résultats des tests de validation du sprint 6

4.4.6 Conclusion du sprint 6

Ce sprint a livré l'application SOC centralisée, l'agent post-attaque et la boucle de réponse SOAR fermée, en clarifiant la séparation entre décision autonome et exécution supervisée. Il achève ainsi la chaîne détection–analyse–réponse de NG-IDPS.

Conclusion

Cette seconde release a complété le système NG-IDPS : le sprint 4 anticipe les menaces par un triage de renseignement hybride, le sprint 5 réduit le risque humain par une sensibilisation ciblée, et le sprint 6 unifie supervision, analyse cognitive et réponse automatisée au sein d'une orchestration SOAR à la fois autonome et supervisable par un analyste. Le chapitre suivant dresse le bilan général du projet et ses perspectives.

CONCLUSION GÉNÉRALE

Ce projet de fin d'études avait pour ambition de concevoir, de réaliser et de valider un système de détection et de prévention des intrusions réseau capable de fonctionner de manière autonome dans un environnement local et isolé, au profit d'organisations qui ne disposent ni d'équipes de supervision permanentes ni de plateformes commerciales intégrées. Le système NG-IDPS qui en résulte associe, au sein d'une chaîne unique, une détection par signatures, une détection comportementale non supervisée, un renseignement sur les menaces, une orchestration de la réponse et un tableau de bord de supervision, le tout reposant sur des composants libres et sur des modèles de langage auto-hébergés pour ses fonctions critiques.

La phase préparatoire a permis de traduire la problématique en sept objectifs fonctionnels et cinq objectifs non fonctionnels, de planifier le travail en six sprints selon la méthodologie Scrum et de définir une architecture répartie en nœuds spécialisés : capture, apprentissage, réponse et supervision.

La première *Release*, consacrée à la détection, a mis en place la capture du trafic par une sonde Suricata, un pipeline à double chemin (indexation pour l'investigation, flux temps réel pour l'analyse) et un moteur de détection comportementale fondé sur un autoencodeur variationnel à mémoire longue (LSTM-VAE). Ce dernier fonctionne avec une latence d'inférence de 2 à 4 millisecondes par séquence, très en deçà de la cible de 15 millisecondes. Son seuil a été calibré en comparant quatre points de fonctionnement ; le point retenu produit un rappel de 51,89 % pour 9,23 % de séquences signalées à tort, une aire sous la courbe ROC de 0,79 et une aire sous la courbe précision-rappel de 0,84. Ces faux positifs sont absorbés par une règle de persistance temporelle avant toute action automatique.

La seconde *Release*, consacrée à la prévention et à la réponse, a ajouté quatre capacités : une veille sur les menaces (CTI) qui agrège des sources ouvertes, trie les articles et propose des règles de détection candidates soumises à validation ; une chaîne de sensibilisation qui transforme les menaces à caractère humain en parcours de formation ciblés, avec un suivi de conformité ; un orchestrateur de réponse à plusieurs paliers, capable d'isoler une source malveillante de façon autonome ou après validation d'un analyste ; et un tableau de bord SOC présentant les incidents, leur diagnostic structuré par un modèle de langage local, l'état de l'infrastructure et un rapport d'audit exportable.

La validation ne s'est pas limitée à des mesures sur un jeu de données de référence. Le système a été confronté à des attaques réellement exécutées depuis une machine dédiée (balayages de ports, identification de services, force brute SSH, scan applicatif web, inondation SYN), dont les six campagnes rejouées dans l'ordre chronologique ont toutes été détectées, avec un taux de 0,18 % de fenêtres bénignes signalées à tort sur le trafic réel des nœuds de l'infrastructure. Cette démarche a surtout permis d'identifier puis de corriger des défauts concrets que l'implémentation seule ne révélait pas : une sélection de caractéristiques défavorable à certaines classes d'attaques, un fichier de règles non référencé par la sonde, des faux positifs provoqués par le trafic du pipeline lui-même, et, du côté des agents d'intelligence artificielle, des sorties désorganisées, des règles recopiées d'un jeu de référence, des étiquettes de source qui biaisaient le classement et des droits d'accès insuffisants pour l'agent de déploiement.

Ces résultats permettent de répondre aux cinq défis posés en introduction. Le défi de la latence est atteint : l'inférence compilée, le tampon glissant de dix pas et les pré-filtres volumétriques maintiennent le temps de calcul entre 2 et 4 millisecondes par séquence, pour une décision d'isolation émise environ dix secondes après le début des attaques testées, délai dominé par le remplissage de la fenêtre d'observation plutôt que par le calcul lui-même. Le défi des faux positifs est également atteint : la calibration du seuil, la règle de persistance sur cinq évaluations et la neutralisation des actifs connus par liste blanche ramènent le taux de séquences signalées à tort à 9,23 % hors ligne et à 0,18 % sur le trafic réel de l'infrastructure. Le défi de la détection de comportements inconnus est atteint grâce à la complémentarité entre la détection par signatures et la détection non supervisée, qui couvrent ensemble aussi bien les attaques connues que les comportements inédits. Le défi de l'accessibilité est atteint : l'emploi de composants libres et d'un modèle de langage local pour l'analyse des incidents et la veille, les modèles distants n'étant sollicités que pour la génération de contenus pédagogiques et toujours avec un repli local, rend la chaîne complète exécutable sans licence ni service obligatoire. Le défi de la continuité de la réponse, enfin, est atteint : l'orchestrateur à plusieurs paliers isole une source malveillante de façon autonome en l'absence d'opérateur, produit des rapports d'audit exportables et détecte la panne d'un nœud en moins de quinze secondes grâce à la surveillance continue de l'infrastructure.

Au regard des objectifs fixés au premier chapitre, les sept objectifs fonctionnels ont tous été réalisés. Parmi les objectifs non fonctionnels, la latence d'inférence, l'autonomie opérationnelle et la maîtrise de l'infrastructure sont pleinement atteintes. La performance de détection et la fiabilité du système le sont dans les limites de l'évaluation menée, que les paragraphes suivants précisent.

Ce travail comporte plusieurs limites qu'il convient d'assumer. La première concerne la portée de l'évaluation en conditions réelles : les attaques réelles ne sont qu'au nombre de six, exécutées dans un laboratoire à site unique, et un taux de détection de 6 sur 6 reste statistiquement compatible, à 95 % de confiance, avec un taux réel compris entre 61 et 100 %. Le jeu de données de référence, issu de 2017, ne reflète par ailleurs pas toutes les pratiques d'attaque actuelles ; ces résultats attestent donc le fonctionnement de bout en bout de la chaîne, sans valeur statistique généralisable. La deuxième limite tient à un écart entre entraînement et production : certaines caractéristiques, notamment les drapeaux TCP, sont approximées en production à partir des

agrégats de flux et non comptées paquet par paquet comme dans les données d'entraînement. Le dispositif compense cet écart par une neutralisation réservée aux adresses de la liste blanche, de sorte que le comportement du modèle sur un hôte inconnu au trafic ordinaire n'a pas été évalué, et que le rappel hors ligne reste limité sur le balayage de ports. La troisième limite porte sur la fiabilité des modèles de langage de petite taille : le modèle local de trois milliards de paramètres produit des diagnostics utiles mais perfectibles, avec parfois une technique MITRE ATT&CK inexacte, des recommandations de remédiation inégales, des scores de fiabilité aberrants ou des règles recopiées d'un jeu de référence ; ces sorties demeurent des propositions, et la validation humaine, le contrôle de structure ainsi que le test à blanc par le moteur Suricata restent indispensables, ce dernier ne vérifiant au demeurant que la syntaxe d'une règle et non sa pertinence. La quatrième limite concerne la sécurité du plan de contrôle interne : dans le laboratoire isolé où le système a été développé, les commandes de remédiation transmises par le canal temps réel ne sont pas authentifiées et le bus de messages n'est ni authentifié ni chiffré, une configuration qui n'est pas acceptable hors d'un réseau fermé. La dernière limite touche au périmètre réseau et à la capacité du système : le blocage repose sur l'adresse IPv4 source, les adresses IPv6 de portée locale étant écartées, la protection de la couche liaison ne couvre qu'un nœud et qu'une adresse, et le passage à l'échelle, qu'il s'agisse d'un courtier de messages unique ou d'une inférence exécutée sur processeur, n'a pas été évalué.

Plusieurs évolutions permettraient de prolonger ce travail. La première consisterait à rétablir la parité entre entraînement et production, en calculant de véritables compteurs de drapeaux à la périphérie ou en recalibrant et en réentraînant le modèle sur du trafic bénin local capturé avec le même extracteur, ce qui rendrait la neutralisation actuelle inutile. La deuxième consisterait à élargir l'évaluation à des campagnes plus nombreuses et plus variées, incluant des attaques lentes, des tentatives d'évasion et un trafic bénin diversifié, afin d'obtenir des intervalles de confiance resserrés. La troisième consisterait à renforcer la validation des règles générées par un test sur trafic rejoué avant déploiement, en complément du test à blanc syntaxique déjà en place, ainsi que par un contrôle de leur pertinence vis-à-vis de l'article source. La quatrième consisterait à sécuriser le plan de contrôle par l'authentification des clients et des commandes, le chiffrement et l'authentification du bus de messages, et la signature des règles déployées. La dernière consisterait à exploiter les décisions des analystes, qu'il s'agisse d'approbations ou de faux positifs déclarés, pour recalibrer périodiquement le seuil de détection et réentraîner le modèle.

Sur le plan personnel, ce projet nous a conduit à articuler des compétences habituellement séparées : réseaux et sécurité, apprentissage automatique, ingénierie logicielle distribuée et conduite de projet agile. Il nous a surtout appris que la valeur d'un système de sécurité se mesure autant à la rigueur avec laquelle on le met à l'épreuve qu'à ses performances annoncées. En ce sens, NG-IDPS constitue une base crédible et améliorable pour les organisations qui cherchent à acquérir une capacité de cyberdéfense proactive tout en conservant la maîtrise de leurs données.

WEBOGRAPHIE

[1] Manfred Vielberth, Fabian Böhm, Ines Fichtinger, and Günther Pernul. Security operations center : A systematic study and open challenges. *IEEE Access*, 8 :227756–227779, 2020.

[2] ISO/IEC/IEEE. ISO/IEC/IEEE 29148 :2018 — systems and software engineering — life cycle processes — requirements engineering, 2018.

[3] Object Management Group. Unified modeling language (UML), version 2.5.1. Specification formal/17-12-05, 2017.

[4] ISO/IEC. ISO/IEC 25010 :2011 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models, 2011.

[5] Mike Cohn. *User Stories Applied : For Agile Software Development*. Addison-Wesley, 2004.

[6] Ken Schwaber and Jeff Sutherland. The Scrum guide : The definitive guide to scrum : The rules of the game. https://scrumguides.org, 2020.

[7] Evangelos Triantaphyllou. *Multi-Criteria Decision Making Methods : A Comparative Study*, volume 44 of *Applied Optimization*. Kluwer Academic Publishers, 2000.

[8] Gerald J. Popek and Robert P. Goldberg. Formal requirements for virtualizable third generation architectures. *Communications of the ACM*, 17(7) :412–421, 1974.

[9] VMware by Broadcom. VMware Workstation Pro : documentation officielle. https://www.vmware.com/products/desktop-hypervisor/workstation-and-fusion.

[10] Oracle. Oracle VirtualBox : documentation officielle. https://www.virtualbox.org.

[11] Avi Kivity, Yaniv Kamay, Dor Laor, Uri Lublin, and Anthony Liguori. kvm : the Linux virtual machine monitor. In *Proceedings of the Linux Symposium*, volume 1, pages 225–230, 2007.

[12] Proxmox Server Solutions GmbH. Proxmox Virtual Environment : documentation officielle. https://www.proxmox.com.

[13] Canonical. Ubuntu release cycle and support lifecycle. https://ubuntu.com/about/release-cycle.

[14] Martin Roesch. Snort : Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)*, pages 229–238, 1999.

[15] Vern Paxson. Bro : a system for detecting network intruders in real-time. *Computer Networks*, 31(23–24) :2435–2463, 1999.

[16] Open Information Security Foundation (OISF). Suricata user guide. https://docs.suricata.io.

[17] Eugene Albin and Neil C. Rowe. A realistic experimental comparison of the Suricata and Snort intrusion-detection systems. In *Proceedings of the 8th International Symposium on Frontiers of Information Systems and Network Applications (FINA)*, Fukuoka, Japan, 2012. IEEE.

[18] Syed Ali Raza Shah and Biju Issac. Performance comparison of intrusion detection systems and application of machine learning to Snort system. *Future Generation Computer Systems*, 80 :157–170, 2018.

[19] Cisco Talos. Snort 3 : documentation officielle. https://www.snort.org/snort3.

[20] The Zeek Project. Zeek documentation. https://docs.zeek.org.

[21] Clinton Gormley and Zachary Tong. *Elasticsearch : The Definitive Guide*. O'Reilly Media, 2015.

[22] Elastic. Elastic stack documentation (Elasticsearch, Logstash, Kibana, Beats). https://www.elastic.co/docs.

[23] OpenSearch Project. OpenSearch documentation. https://opensearch.org/docs/latest/.

[24] Graylog. Graylog documentation. https://go2docs.graylog.org.

[25] Jay Kreps, Neha Narkhede, Jun Rao, et al. Kafka : A a distributed messaging system for log processing. In *Proceedings of the NetDB*, volume 11, pages 1–7, 2011.

[26] Amazon Web Services. Amazon Simple Queue Service (SQS) developer guide. https://docs.aws.amazon.com/sqs/.

[27] Apache Software Foundation. Apache Kafka documentation. https://kafka.apache.org/documentation/.

[28] Apache Software Foundation. Apache Pulsar documentation. https://pulsar.apache.org/docs/.

[29] Broadcom / VMware Tanzu. RabbitMQ documentation. https://www.rabbitmq.com/docs.

[30] Redis. Redis streams : documentation. https://redis.io/docs/latest/develop/data-types/streams/.

[31] Philippe Dobbelaere and Kyumars Sheykh Esmaili. Kafka versus RabbitMQ : A comparative study of two industry reference publish/subscribe implementations. In *Proceedings of the 11th ACM International Conference on Distributed and Event-based Systems (DEBS)*, pages 227–238. ACM, 2017.

[32] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection : A survey. *ACM Computing Surveys*, 41(3) :1–58, 2009.

[33] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2) :1153–1176, 2016.

[34] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422. IEEE, 2008.

[35] Bernhard Schölkopf, John C. Platt, John Shawe-Taylor, Alex J. Smola, and Robert C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7) :1443–1471, 2001.

[36] Lukas Ruff, Robert Vandermeulen, Nico Goernitz, Lucas Deecke, Shoaib Ahmed Siddiqui, Alexander Binder, Emmanuel Müller, and Marius Kloft. Deep one-class classification. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pages 4393–4402, 2018.

[37] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8) :1735–1780, 1997.

[38] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *Proceedings of the 2nd International Conference on Learning Representations (ICLR)*, 2014. arXiv :1312.6114.

[39] Daehyung Park, Yuuna Hoshi, and Charles C. Kemp. A multimodal anomaly detector for robot-assisted feeding using an LSTM-based variational autoencoder. *IEEE Robotics and Automation Letters*, 3(3) :1544–1551, 2018.

[40] Jinwon An and Sungzoon Cho. Variational autoencoder based anomaly detection using reconstruction probability. *Special Lecture on IE*, 2(1) :1–18, 2015.

[41] Martín Abadi et al. Tensorflow : A system for large-scale machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, pages 265–283, 2016.

[42] Adam Paszke et al. PyTorch : An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035, 2019.

[43] Fabian Pedregosa et al. Scikit-learn : Machine learning in Python. *Journal of Machine Learning Research*, 12 :2825–2830, 2011.

[44] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the KDD CUP 99 data set. In *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, pages 1–6. IEEE, 2009.

[45] Nour Moustafa and Jill Slay. UNSW-NB15 : a comprehensive data set for network intrusion detection systems. In *2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6. IEEE, 2015.

[46] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018.

[47] Canadian Institute for Cybersecurity. Intrusion detection evaluation dataset (CIC-IDS2017). https://www.unb.ca/cic/datasets/ids-2017.html.

[48] Gints Engelen, Vera Rimmer, and Wouter Joosen. Troubleshooting an intrusion detection dataset : the CICIDS2017 case study. In *2021 IEEE Security and Privacy Workshops (SPW)*, pages 7–12. IEEE, 2021.

[49] Maxime Lanvin, Pierre-François Gimenez, Yufei Han, Frédéric Majorczyk, Ludovic Mé, and Eric Totel. Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes. In *Risks and Security of Internet and Systems (CRiSIS 2022)*, volume 13857 of *Lecture Notes in Computer Science*, pages 18–33. Springer, 2023.

[50] OpenAI. GPT-4 technical report. arXiv preprint arXiv :2303.08774, 2023.

[51] Aaron Grattafiori et al. The Llama 3 herd of models. arXiv preprint arXiv :2407.21783, 2024.

[52] Albert Q. Jiang et al. Mistral 7B. arXiv preprint arXiv :2310.06825, 2023.

[53] Ollama. Ollama : documentation et api rest. https://github.com/ollama/ollama.

[54] Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626. ACM, 2023.

[55] Georgi Gerganov and contributors. llama.cpp : LLM inference in C/C++. https://github.com/ggml-org/llama.cpp.

[56] Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, pages 9459–9474, 2020.

[57] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3) :535–547, 2021.

[58] Jianguo Wang et al. Milvus : A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, pages 2614–2627. ACM, 2021.

[59] Chroma. Chroma documentation. https://docs.trychroma.com.

[60] Zach Nussbaum, John X. Morris, Brandon Duderstadt, and Andriy Mulyar. Nomic embed : Training a reproducible long context text embedder. arXiv preprint arXiv :2402.01613, 2024.

[61] Pydantic. Pydantic documentation. https://docs.pydantic.dev.

[62] Netfilter Core Team. The netfilter.org project : iptables and nftables. https://www.netfilter.org.

[63] Tatu Ylönen and Chris Lonvick. The secure shell (SSH) protocol architecture. RFC 4251, Internet Engineering Task Force, 2006.

[64] Paramiko. Paramiko : a Python implementation of SSHv2. https://www.paramiko.org.

[65] Red Hat. Ansible documentation. https://docs.ansible.com.

[66] Shuffle. Shuffle : open source security automation platform. https://shuffler.io.

[67] StrangeBee. TheHive and Cortex : incident response and analysis platform. https://strangebee.com/thehive/.

[68] Stefan Tilkov and Steve Vinoski. Node.js : Using JavaScript to build high-performance network programs. *IEEE Internet Computing*, 14(6) :80–83, 2010.

[69] Django Software Foundation. Django documentation. https://docs.djangoproject.com.

[70] Sebastián Ramírez. FastAPI documentation. https://fastapi.tiangolo.com.

[71] Ian Fette and Alexey Melnikov. The WebSocket protocol. RFC 6455, Internet Engineering Task Force, 2011.

[72] Socket.IO. Socket.IO documentation, v4. https://socket.io/docs/v4/.

[73] Vercel. Next.js documentation. https://nextjs.org/docs.

[74] Vue.js. Vue.js guide. https://vuejs.org/guide/.

[75] Google. Angular documentation. https://angular.dev.

[76] Tailwind Labs. Tailwind CSS documentation. https://tailwindcss.com/docs.

[77] Rick Cattell. Scalable SQL and NoSQL data stores. *ACM SIGMOD Record*, 39(4) :12–27, 2011.

[78] MongoDB, Inc. MongoDB manual. https://www.mongodb.com/docs/.

[79] PostgreSQL Global Development Group. PostgreSQL documentation. https://www.postgresql.org/docs/.

[80] D. Richard Hipp and contributors. SQLite documentation. https://www.sqlite.org/docs.html.

[81] GitHub. Planning and tracking with GitHub Projects. https://docs.github.com/en/issues/planning-and-tracking-with-projects.

[82] Daniela Brauckhoff and Christos Xenakis. Impact of packet drop on network intrusion detection. In *Information Security Theory and Practices. Security and Privacy of Pervasive Systems and Smart Devices (WISTP 2010)*, pages 202–217. Springer, 2010.

[83] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319, 2014.

[84] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8) :1735–1780, 1997.

[85] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv :1312.6114*, 2013.

[86] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018. Dataset CIC-IDS-2017.

[87] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn : Machine learning in Python. *Journal of Machine Learning Research*, 12 :2825–2830, 2011.

[88] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8) :861–874, 2006.

[89] Gemini Team, Google DeepMind. Gemini : A family of highly capable multimodal models. arXiv preprint arXiv :2312.11805, 2023.

[90] Paul Cichonski, Tom Millar, Tim Grance, and Karen Scarfone. Computer security incident handling guide. Technical Report NIST Special Publication 800-61 Revision 2, National Institute of Standards and Technology (NIST), 2012.

[91] Ollama. *Ollama : Get up and running with large language models locally*, 2024. Consulté en 2026.

[92] Blake E. Strom, Andy Applebaum, Doug P. Miller, Kathryn C. Nickels, Adam G. Pennington, and Cody B. Thomas. MITRE ATT&CK : Design and philosophy. Technical report, The MITRE Corporation, 2018.

[93] Socket.IO. Socket.io : documentation officielle. https://socket.io/docs/v4/, 2024.

Ce mémoire présente la conception et l'implémentation d'une plateforme autonome de cyberdéfense hybride, associant détection déterministe par signatures et détection comportementale par apprentissage profond. L'architecture repose sur trois plans d'exécution interdépendants : un *Data Plane* assurant une double voie d'acquisition — un chemin critique temps réel via Apache Kafka (mode KRaft) alimentant l'inférence, et un chemin parallèle Filebeat/Logstash/Elasticsearch dédié à l'audit et à la supervision humaine ; un *Control Plane* hébergeant un modèle LSTM-VAE non-supervisé, entraîné sur le corpus CIC-IDS-2017, dont le seuil de détection dynamique ($\mu + 3\sigma = 0{,}006754$) a été validé empiriquement avec un taux de faux positifs de 1,93 % sur un jour non vu à l'entraînement ; un *Response Plane* matérialisé par un orchestrateur SOAR appliquant une logique de décision à trois paliers (alerte simple, blocage automatique par persistance ou sévérité, blocage prioritaire par corrélation multi-source), avec exécution distante sécurisée du confinement réseau via SSH.

Les expérimentations conduites en conditions réelles démontrent une latence d'inférence de 1,245 ms, largement conforme à la contrainte de temps réel fixée, ainsi qu'une réduction de la latence de détection de bout en bout d'un facteur treize (de 53 secondes à 3-4 secondes) grâce à l'optimisation des paramètres de gestion des flux réseau. Un scénario de simulation d'attaque *Zero-Day*, réalisé en désactivant intégralement la base de signatures Suricata, confirme la capacité du modèle comportemental à détecter une exfiltration de données en l'absence de toute connaissance a priori de la menace. Une analyse critique du décalage entre corpus d'entraînement public et trafic réel de l'infrastructure de déploiement est également présentée, ouvrant sur des perspectives d'adaptation de domaine.

Mots-clés : Cyberdéfense autonome, Détection d'intrusion, Zero-Day, LSTM-VAE, Apprentissage profond non-supervisé, SOAR, Architecture événementielle, Apache Kafka, MLOps.

This dissertation presents the design and implementation of an autonomous hybrid cyberdefense platform combining deterministic signature-based detection with deep learning-based behavioral detection. The architecture rests on three interdependent execution planes: a Data Plane providing a dual acquisition path — a real-time critical path via Apache Kafka (KRaft mode) feeding the inference engine, and a parallel Filebeat/Logstash/Elasticsearch path dedicated to audit and human oversight; a Control Plane hosting an unsupervised LSTM-VAE model trained on the CIC-IDS-2017 corpus, whose dynamic detection threshold ($\mu + 3\sigma = 0.006754$) was empirically validated with a 1.93% false-positive rate on an unseen day; and a Response Plane implemented as a SOAR orchestrator applying a three-tier decision logic (simple alert, automatic blocking by persistence or severity, priority blocking by multi-source correlation), with secure remote execution of network containment via SSH.

Real-world experiments demonstrate an inference latency of 1.245 ms, well within the real-time constraint, and a thirteen-fold reduction in end-to-end detection latency (from 53 seconds to 3-4 seconds) achieved through tuning of network flow-management parameters. A Zero-Day attack simulation, conducted by fully disabling the Suricata signature base, confirms the behavioral model's ability to detect a data exfiltration attempt with no prior knowledge of the threat. A critical analysis of the gap between the public training corpus and the real traffic of the deployment infrastructure is also presented, opening avenues for domain adaptation.

Keywords: Autonomous cyberdefense, Intrusion detection, Zero-Day, LSTM-VAE, Unsupervised deep learning, SOAR, Event-driven architecture, Apache Kafka, MLOps.

Rapport Plagiat - PFE - Aziz Dhifaoui

RAPPORT DE SIMILITUDE

9%	7%	5%	7%
INDICE DE SIMILITUDE	SOURCES INTERNET	PUBLICATIONS	COPIES DE L'ÉTUDIANT

SOURCES PRINCIPALES

1	Submitted to University of Carthage Copie de l'étudiant	1%
2	Submitted to Tunisian Consortium Copie de l'étudiant	1%
3	Submitted to University of Manouba Copie de l'étudiant	1%
4	Submitted to University of Monastir Copie de l'étudiant	<1%
5	arxiv.org Source Internet	<1%
6	export.arxiv.org Source Internet	<1%
7	Kuan-Cheng Chang, Po-Hsin Hsieh, Mei-Yao Wu, Yu-Chen Wang et al. "Usefulness of Machine-Learning-Based Detection and Classification of Cardiac Arrhythmias with 12-Lead Electrocardiograms", Canadian Journal of Cardiology, 2020 Publication	<1%
8	theses.hal.science Source Internet	<1%
9	hdl.handle.net Source Internet	<1%
10	Submitted to University of Sousse Copie de l'étudiant	<1%
11	tel.archives-ouvertes.fr Source Internet	<1%
12	backend.orbit.dtu.dk Source Internet	<1%
13	sdm.lbl.gov Source Internet	<1%
14	Submitted to Imperial College of Science, Technology and Medicine Copie de l'étudiant	<1%